

INFORMATIQUE • TLE TI

MODULE 1 : PROGRAMMATION

LEÇON N04

PHP AVANCÉ : STRUCTURES DE CONTRÔLE ET INTERFAÇAGE AVEC MYSQL

DURÉE CONSEILLÉE : 14 H

DANS CETTE LEÇON

Cette leçon approfondit PHP par l'étude des structures de contrôle (alternatives et boucles) et de la fonction date(), puis présente l'interfaçage avec MySQL via l'extension mysqli : connexion, exécution des requêtes d'insertion, de modification, de suppression et de sélection, et affichage des résultats. Elle aboutit à la réalisation d'une petite application web complète interagissant avec une base de données.

- À la fin de cette leçon, je sais utiliser les structures alternatives if, else, elseif et switch pour orienter l'exécution d'un script PHP
- À la fin de cette leçon, je sais utiliser les boucles for, foreach, while et do...while pour répéter des traitements
- À la fin de cette leçon, je sais afficher et formater la date et l'heure du serveur avec la fonction date()
- À la fin de cette leçon, je sais établir une connexion à un serveur MySQL avec mysqli_connect et mysqli_select_db, et gérer les erreurs avec die()
- À la fin de cette leçon, je sais exécuter des requêtes SQL d'insertion, de modification, de suppression et de sélection depuis PHP avec mysqli_query
- À la fin de cette leçon, je sais parcourir et afficher les résultats d'une requête avec mysqli_fetch_row, mysqli_fetch_assoc, mysqli_fetch_array et mysqli_num_rows



TÉLÉCHARGE ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

1. Structures alternatives : if, else, elseif, switch
2. Structures répétitives : for, foreach, while, do...while
3. La fonction date()
4. Connexion à MySQL avec l'extension mysqli
5. Exécution des requêtes SQL depuis PHP
6. Lecture et affichage des résultats
7. Synthèse : mini-application web PHP/MySQL
8. Activité d'intégration
9. Méthodes et exercices résolus
10. Exercices
11. Corrigés des exercices
12. Fiche bilan

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais utiliser les structures alternatives if, else, elseif et switch pour orienter l'exécution d'un script PHP
- À la fin de cette leçon, je sais utiliser les boucles for, foreach, while et do...while pour répéter des traitements
- À la fin de cette leçon, je sais afficher et formater la date et l'heure du serveur avec la fonction date()
- À la fin de cette leçon, je sais établir une connexion à un serveur MySQL avec mysqli_connect et mysqli_select_db, et gérer les erreurs avec die()
- À la fin de cette leçon, je sais exécuter des requêtes SQL d'insertion, de modification, de suppression et de sélection depuis PHP avec mysqli_query
- À la fin de cette leçon, je sais parcourir et afficher les résultats d'une requête avec mysqli_fetch_row, mysqli_fetch_assoc, mysqli_fetch_array et mysqli_num_rows
- À la fin de cette leçon, je sais fermer proprement une connexion avec mysqli_close
- À la fin de cette leçon, je sais produire une petite application web complète interagissant avec une base de données MySQL

PRÉREQUIS

- Notions de PHP de base vues en Première TI : variables, types, echo, formulaires HTML

et passage de données (GET/POST)

- Langage SQL de base : CREATE TABLE, SELECT, INSERT, UPDATE, DELETE (leçon sur les bases de données)
- Environnement de travail WAMP/XAMPP : serveur Apache, MySQL, phpMyAdmin
- Structure d'une table : champs, enregistrements, clé primaire
- Algorithmique : notion de condition et de boucle (Première)

1 STRUCTURES ALTERNATIVES : IF, ELSE, ELSEIF, SWITCH

À la cybercafétéria « Espace Jeunes » de Yaoundé, les inscriptions au concours de la Fonction publique se font encore dans un grand cahier cartonné : doublons, ratures, pages arrachées, et des recherches interminables quand un candidat revient réclamer sa fiche. Le propriétaire, séduit par les plateformes comme `impots.cm`, veut un petit site web où chaque candidat s'inscrit en ligne, où l'on peut modifier ou supprimer une fiche, et afficher la liste des inscrits du jour. Mais un tel site doit aussi prendre des décisions : si le candidat a moins de 21 ans, afficher un message ; si sa moyenne est supérieure à 10, afficher « Admis » ; sinon, proposer le rattrapage. Comment faire dialoguer une page PHP avec une base de données MySQL ? Comment répéter l'affichage de centaines de lignes sans écrire centaines de lignes de code ? Comment, surtout, faire en sorte qu'un programme choisisse quoi faire selon les données ? C'est tout l'objet de cette leçon, et nous commençons par la brique fondamentale de toute décision : les **structures alternatives**.

1.1 Rappel algorithmique : qu'est-ce qu'une structure alternative ?

Jusqu'ici, tes scripts PHP s'exécutaient de haut en bas, ligne après ligne, comme une recette de cuisine. Or un vrai programme doit pouvoir réagir : afficher « Connexion réussie » si le mot de passe est correct, et « Accès refusé » sinon. En algorithmique, tu connais déjà cette idée sous la forme :

```
Si condition Alors
    instructions 1
Sinon
    instructions 2
FinSi
```

DÉFINITION Structure alternative

Une **structure alternative** (ou structure conditionnelle) est une instruction qui exécute un bloc de code selon qu'une **condition** — une expression dont la valeur est vraie ou fausse — est vérifiée ou non. Elle permet au programme de choisir entre plusieurs chemins d'exécution.

Une condition est une expression booléenne : elle ne peut valoir que `true` ou `false`. Pour la construire, PHP met à ta disposition deux familles d'opérateurs.

PROPRIÉTÉ Opérateurs de comparaison et opérateurs logiques

Opérateurs de comparaison (comparent deux valeurs et renvoient un booléen) :

- `==` : égalité de valeur ; `!=` : différence ;

- <, > : strictement inférieur, strictement supérieur ;

- <=, >= : inférieur ou égal, supérieur ou égal.

Opérateurs logiques (combinent des conditions) :

- && : ET logique — vrai si les deux conditions sont vraies ;
- || : OU logique — vrai si au moins une condition est vraie ;
- ! : NON logique — inverse la valeur de la condition.

Exemple : `($age >= 18) && ($age <= 35)` est vrai si l'âge est compris entre 18 et 35 ans inclus.

1.2 La structure if : exécuter un bloc seulement si...

La forme la plus simple ne prévoit qu'un seul chemin : on exécute un bloc uniquement si la condition est vraie, sinon on le saute.

PROPRIÉTÉ Syntaxe de if

```
if (condition){  
    // instructions exécutées si la condition est vraie  
}  
// la suite du programme s'exécute dans tous les cas
```

La condition est placée entre parenthèses ; le bloc d'instructions est délimité par des accolades { }. Si la condition est fausse, PHP ignore le bloc et poursuit après l'accolade fermante.

EXEMPLE Contrôle d'âge à l'inscription

```
<?php  
$age = 19;  
if ($age < 18){  
    echo "Inscription refusée : vous devez être majeur." ;  
}  
echo "Bienvenue sur Espace Jeunes." ;  
?>
```

Ici, le message de refus n'apparaît que si `$age` est strictement inférieur à 18 ; le message de bienvenue s'affiche dans tous les cas.

ATTENTION Le piège mortel : = au lieu de ==

L'erreur la plus fréquente (et la plus redoutée au Bac) est de confondre l'**affectation** = et la **comparaison** ==. Le code suivant est faux :

```
if ($a = 5) { // ERREUR : on AFFECTE 5 à $a !  
    echo "a vaut 5";  
}
```

\$a = 5 range 5 dans \$a, puis PHP évalue la valeur affectée : 5 est considéré comme vrai. Le test est donc toujours vrai, quelle que soit la valeur initiale de \$a. La version correcte est `if ($a == 5)`. Retiens : un seul = pour ranger, deux == pour comparer.

1.3 if ... else : traiter le cas contraire

Le plus souvent, on veut faire quelque chose aussi quand la condition est fausse : c'est le rôle de else.

PROPRIÉTÉ Syntaxe de if ... else

```
if (condition){  
    // instructions si la condition est VRAIE  
} else {  
    // instructions si la condition est FAUSSE  
}
```

Un seul des deux blocs est exécuté, jamais les deux, jamais aucun : l'alternative est exclusive et complète.

EXEMPLE Décision d'admission binaire

```
<?php  
$moyenne = 9.75;  
if ($moyenne >= 10) {  
    echo "Admis au concours."  
} else {  
    echo "Non admis cette année."  
}  
?>
```

Avec `$moyenne = 9.75`, la condition `$moyenne >= 10` est fausse : c'est le bloc else qui s'exécute.

1.4 if ... elseif ... else : la cascade de conditions

La réalité offre rarement seulement deux issues. Au Baccalauréat, une moyenne de 15 n'a pas le même sens qu'une moyenne de 11 : il faut tester plusieurs cas ordonnés. On enchaîne alors des `elseif` (contraction de « else if »).

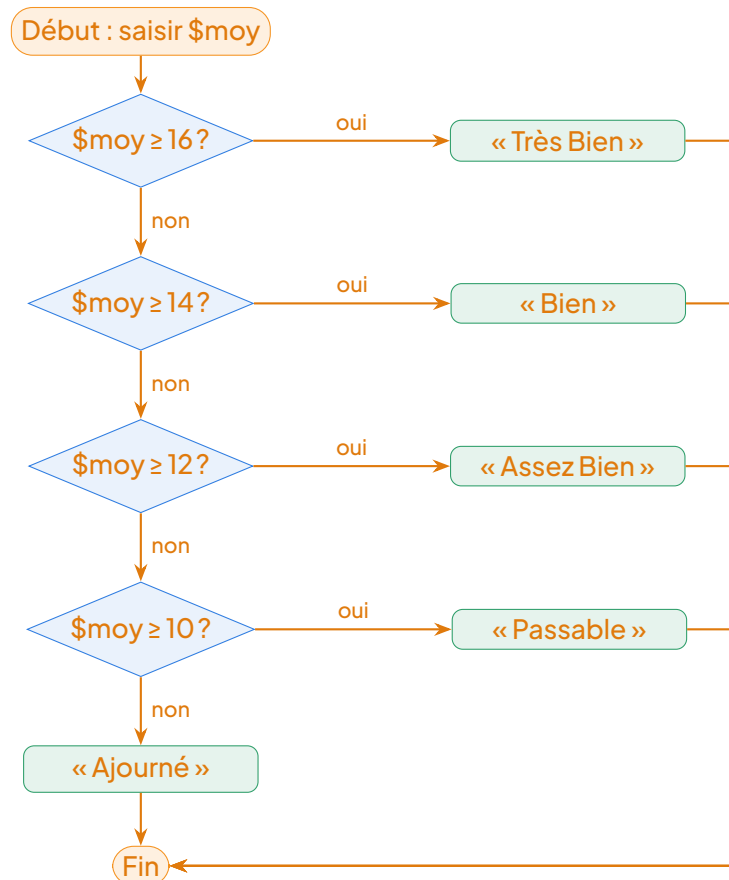
PROPRIÉTÉ Syntaxe et ordre d'évaluation de la cascade

```
if (condition1){  
    // si condition1 est vraie  
} elseif (condition2){  
    // sinon, si condition2 est vraie  
} elseif (condition3){  
    // sinon, si condition3 est vraie  
} else {  
    // sinon (toutes les conditions sont fausses)  
}
```

PHP évalue les conditions de haut en bas. Dès qu'une condition est vraie, son bloc est exécuté et tous les autres sont ignorés. Le `else final`, facultatif, ramasse tous les cas restants. L'ordre des tests est donc crucial : on place en général les conditions les plus restrictives en premier.

ATTENTION Ordre des tests : un bug silencieux

Si tu écris `if ($moy >= 10)` avant `elseif ($moy >= 14)`, un élève à 16 entrera dans le premier bloc et n'obtiendra jamais sa mention « Bien ». Le programme ne signale aucune erreur : il donne simplement un mauvais résultat. Teste toujours du plus exigeant vers le moins exigeant.



Arbre de décision de la cascade if/elseif/else pour les mentions du Bac : chaque losange n'est atteint que si tous les tests précédents ont échoué.

1.5 switch : le choix multiple sur une même variable

Quand il faut comparer une même variable à plusieurs valeurs constantes (un jour de la semaine, un code de filière, un choix de menu), la cascade de `elseif` devient lourde et répétitive. PHP propose alors une structure dédiée : `switch`.

PROPRIÉTÉ Syntaxe de switch

```
switch (variable) {  
    case valeur1 :  
        // instructions si variable == valeur1  
        break;  
    case valeur2 :  
        // instructions si variable == valeur2  
        break;  
    default :  
        // instructions si aucun cas ne correspond
```



```
| }
```

PHP compare la variable à chaque `case` par égalité (comparaison faible `==`). Le mot-clé `break` termine le `switch` : sans lui, l'exécution continue dans les cas suivants. Le `default`, facultatif, joue le rôle du `else final`.

EXEMPLE Menu de la cybercafétéria

```
<?php
$choix = 2;
switch($choix){
    case 1:
        echo "Nouvelle inscription";
        break;
    case 2:
        echo "Modifier une fiche";
        break;
    case 3:
        echo "Supprimer une fiche";
        break;
    case 4:
        echo "Liste des inscrits du jour";
        break;
    default:
        echo "Choix invalide.";
}
?>
```

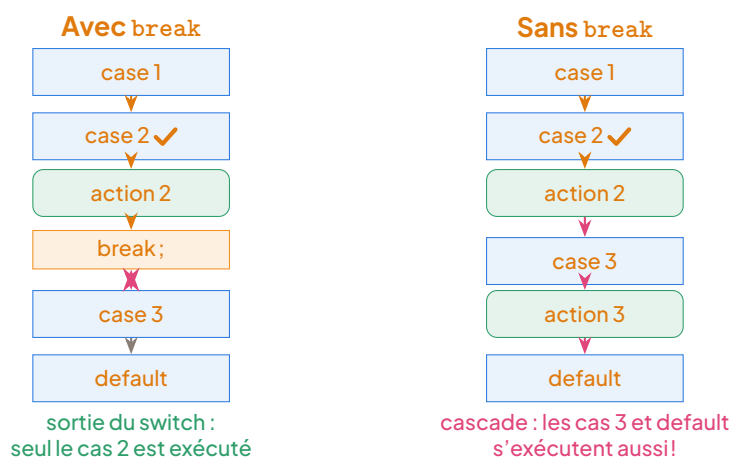
Avec `$choix = 2`, seul « Modifier une fiche » s'affiche.

ATTENTION Oublier le break : l'exécution en cascade

Sans `break`, PHP ne s'arrête pas à la fin du cas trouvé : il exécute tous les cas suivants jusqu'à la fin du `switch` (ou jusqu'au prochain `break`). C'est le comportement « en cascade » (fall-through).

```
<?php
$choix = 2;
switch($choix){
    case 2:
        echo "Modification ";
        // break oubliée ici !
    case 3:
        echo "Suppression ";
    default:
        echo "Invalide";
}
?>
```

Affiche : Modification Suppression Invalide. Ce comportement n'est utile que dans un cas précis : faire partager le même traitement à plusieurs valeurs en empilant des `case` vides (`case 1: case 2: ...`). Sinon, chaque `case` doit se terminer par `break;`.



Flux d'exécution d'un switch : à gauche, le `break` interrompt la structure ; à droite, son oubli laisse l'exécution « tomber » dans les cas suivants.

1.6 if/elseif ou switch : comment choisir ?

Les deux structures peuvent souvent exprimer la même décision, mais elles ne sont pas interchangeables.

MÉTHODE Choisir entre if/elseif et switch

1. Pose-toi la question : est-ce que je teste l'égalité d'une seule et même variable à plusieurs valeurs constantes (nombres, chaînes) ?
2. Si oui (menu, jour de la semaine, code filière...) : utilise `switch`, plus lisible et plus compact. N'oublie pas les `break;`.
3. Si non — intervalles (`$moy >= 14`), conditions composées avec `&&` ou `||`, comparaisons portant sur des variables différentes — : utilise la cascade `if/elseif/else`, en ordonnant les tests du plus restrictif au moins restrictif.

Critère	if / elseif / else	switch
Type de test	Toute condition (intervalles, ET/OU)	Égalité à des constantes uniquement
Variable testée	Peut changer à chaque test	Une seule variable pour tous les cas
Lisibilité	Bonne pour 2 à 4 cas	Excellente pour de nombreux cas
Risque classique	Mauvais ordre des tests	Oubli du break

1.7 Exemple complet : la mention au Baccalauréat

Mettons tout cela en œuvre dans le cas d'école qui tombe régulièrement à l'examen : afficher la mention d'un élève selon sa moyenne.

EXERCICE RÉSOLU 1 Mentions du Bac en PHP

Énoncé. Écrire un script PHP qui, à partir d'une variable `$moyenne` comprise entre 0 et 20, affiche la décision du jury : « Ajourné » si la moyenne est inférieure à 10 ; « Passable » de 10 à moins de 12 ; « Assez Bien » de 12 à moins de 14 ; « Bien » de 14 à moins de 16 ; « Très Bien » à partir de 16. Tester le script avec la moyenne 13,5.

SOLUTION

Solution. On utilise une cascade `if/elseif/else` car les tests portent sur des intervalles, ce que `switch` ne sait pas faire. On ordonne les conditions de la plus exigeante à la moins exigeante.

```
<?php
$moyenne = 13.5; // moyenne de l'eleve sur 20

if ($moyenne >= 16){
    $mention = "Tres Bien";
} elseif ($moyenne >= 14){
    $mention = "Bien";
} elseif ($moyenne >= 12){
    $mention = "Assez Bien";
} elseif ($moyenne >= 10){
    $mention = "Passable";
} else {
    $mention = "Ajourne";
}

echo "Moyenne : $moyenne /20 - Mention : $mention";
?>
```

Trace d'exécution avec `$moyenne = 13.5` :

- 13.5 >= 16 ? Faux.
- 13.5 >= 14 ? Faux.
- 13.5 >= 12 ? Vrai : `$mention` reçoit « Assez Bien » et les tests suivants sont ignorés.

Le script affiche : Moyenne : 13.5 /20 - Mention : Assez Bien. Remarque l'astuce

d'ordre : grâce à la cascade, il suffit d'écrire la borne inférieure de chaque intervalle ; la borne supérieure est garantie par l'échec des tests précédents.

Dans une application réelle comme celle d'Espace Jeunes, la moyenne proviendrait d'un formulaire HTML. On la récupère alors via le tableau `$_POST` :

EXEMPLE Décision à partir d'un formulaire

```
<?php
// Le formulaire HTML envoie un champ nomme "moyenne"
$moy = $_POST["moyenne"];

if ($moy >= 10) {
    echo "Admis(e) au concours. Felicitations !";
} elseif ($moy >= 8) {
    echo "Rattrapage : preparez l'epreuve orale.";
} else {
    echo "Ajourne(e). Retentez votre chance l'an prochain.";
}
?>
```

Avec une moyenne saisie de 8,5, le visiteur lit : « Rattrapage : préparez l'épreuve orale. »

LESAVAIS-TU ? L'opérateur ternaire : le if/else en une ligne

PHP propose une forme compacte du if/else, appelée **opérateur ternaire** : `condition ? valeurSiVrai : valeurSiFaux`. Par exemple :

```
$statut = ($moyenne >= 10) ? "Admis" : "Ajourne";
// equivalent a un if/else complet, en une seule ligne
```

Très apprécié des développeurs pour les affectations courtes, il est un complément utile à connaître pour le Bac. Son nom vient du fait qu'il prend trois opérandes. Attention toutefois : dès que le traitement dépasse une instruction simple, reviens au if/else classique, bien plus lisible.

À RETENIR L'essentiel de la section

- Une **structure alternative** exécute un bloc selon qu'une condition est vraie ou fausse.
- if seul : un cas ; if/else : deux cas exclusifs ; if/elseif/else : cascade évaluée de haut en bas, dont l'ordre est déterminant.
- switch compare une même variable à des valeurs constantes ; chaque case se termine

par `break;`, sous peine d'exécution en cascade.

- `==` compare, `=` affecte : `if ($a = 5)` est toujours vrai — l'erreur classique à bannir.
- Intervalles ou conditions composées $\Rightarrow$ `if/elseif` ; égalités multiples sur une variable $\Rightarrow$ `switch`.

EXERCICE 1 À toi de jouer

1. Écrire un script qui affiche si un candidat peut s'inscrire au concours : il faut avoir au moins 18 ans et être titulaire du Baccalauréat (variable booléenne `$bac`).
2. Écrire un `switch` qui affiche le nom du jour de la semaine à partir d'un numéro `$jour` de 1 (lundi) à 7 (dimanche), avec un message d'erreur pour toute autre valeur.
3. Réécrire la cascade des mentions du Bac en commençant par les plus petites moyennes. Quelle condition faut-il alors écrire à chaque étape ?

Maintenant que ton programme sait décider, il doit apprendre à répéter : afficher les 300 inscrits du jour sans écrire 300 fois la même instruction. C'est l'objet de la section suivante, consacrée aux structures répétitives.

2 STRUCTURES RÉPÉTITIVES : FOR, FOREACH, WHILE, DO...WHILE

Dans la section précédente, nous avons appris à faire des choix dans un script grâce aux structures alternatives (`if`, `elseif`, `else`, `switch`) : le programme emprunte un chemin ou un autre, mais chaque instruction n'est exécutée qu'une seule fois. Or, dans une application web réelle, on a très souvent besoin de répéter un traitement : afficher les 20 arrondissements d'une région, lister tous les élèves d'une classe de Terminale TI, calculer la moyenne de 15 notes, afficher toutes les lignes renvoyées par une requête MySQL. Recopier vingt fois la même instruction serait absurde, illisible et impossible à maintenir. C'est précisément le problème que résolvent les **structures répétitives**, encore appelées **boucles**.

DÉFINITION Structure répétitive (boucle)

Une **structure répétitive** (ou **boucle**) est une instruction qui répète l'exécution d'un bloc de code tant qu'une condition est vérifiée. Chaque exécution du bloc s'appelle une **itération**. Une boucle bien construite repose sur trois éléments :

- **l'initialisation** : on donne une valeur de départ à une variable de contrôle (le **compteur**) ;

- la **condition d'arrêt** : une expression booléenne testée à chaque passage ; tant qu'elle est vraie, la boucle continue ;
- l'**incrément** (ou modification du compteur) : on fait évoluer le compteur dans le corps de la boucle pour que la condition finisse par devenir fausse.

PHP propose quatre boucles : `for`, `while`, `do...while` et `foreach`. Étudions-les une à une.

2.1 La boucle `for` : quand le nombre d'itérations est connu

Intuition. Tu sais déjà combien de fois tu veux répéter : afficher les tables de multiplication de 1 à 10, numéroté les 20 arrondissements, calculer la somme des entiers de 1 à 100. Dans ce cas, la boucle `for` concentre les trois éléments de contrôle (initialisation, condition, incrément) sur une seule ligne : c'est la plus compacte et la plus sûre.

PROPRIÉTÉ Syntaxe de la boucle `for`

```
for ( initialisation ; condition ; incrementation ) {  
    // instructions repetees tant que la condition est vraie  
}
```

L'ordre d'exécution est le suivant :

1. l'initialisation est exécutée une seule fois, avant le premier tour ;
2. la condition est évaluée : si elle est fausse, la boucle s'arrête immédiatement (le corps peut donc ne jamais s'exécuter) ;
3. si elle est vraie, le corps est exécuté, puis l'incrément, puis on retourne à l'étape 2.

EXEMPLE Exemple chiffré : somme des entiers de 1 à 100 avec `for`

```
<?php  
    $somme = 0 ; // accumulateur, initialise a zero  
    for ($i = 1; $i <= 100; $i++) {  
        $somme = $somme + $i ; // on ajoute le compteur a chaque tour  
    }  
    echo "La somme de 1 a 100 vaut : $somme" ; // affiche 5050  
?>
```

Déroulé : `$i` vaut successivement 1, 2, 3, ..., 100. À chaque tour, on ajoute `$i` à `$somme`. Après le 100^e tour, `$i` devient 101, la condition `$i <= 100` devient fausse et la boucle s'arrête. Résultat affiché : 5050.

2.2 La boucle while : répéter tant qu'une condition est vraie

Intuition. Parfois, on ne sait pas à l'avance combien de tours seront nécessaires : on continue « tant que » quelque chose est vrai. Exemples : tant qu'il reste des lignes dans le résultat d'une requête MySQL, tant que le solde Mobile Money est insuffisant, tant que l'utilisateur saisit un mot de passe incorrect. La boucle `while` teste la condition en entrée : si elle est fausse dès le départ, le corps n'est jamais exécuté.

PROPRIÉTÉ Syntaxe de la boucle while

```
while (condition) {  
    // instructions repetees tant que la condition est vraie  
    // le compteur DOIT etre modifie ici  
}
```

La condition est testée avant chaque itération. Le nombre minimal d'exécutions du corps est donc zéro.

EXEMPLE La somme de 1 à 100 réécrite avec while

```
<?php  
$somme = 0;  
$i = 1;           // 1. initialisation (hors de la boucle)  
while ($i <= 100) { // 2. condition testee en entree  
    $somme = $somme + $i;  
    $i++;        // 3. incrementation (dans le corps)  
}  
echo "La somme de 1 a 100 vaut : $somme"; // affiche 5050  
?>
```

On retrouve exactement les trois éléments du `for`, mais dispersés : l'initialisation avant la boucle, la condition entre parenthèses, l'incrémentation dans le corps.

PROPRIÉTÉ Équivalence for / while (admise, justifiée sur exemple)

Toute boucle `for` peut se réécrire en `while`, et réciproquement. Les deux scripts précédents le prouvent : ils produisent le même résultat (5050) avec la même logique. La différence est syntaxique, pas fonctionnelle : le `for` regroupe les trois éléments de contrôle sur une ligne (plus lisible quand le nombre de tours est connu), le `while` les sépare (plus souple quand la condition dépend d'un événement imprévisible, comme la lecture d'une ligne de base de données).

2.3 La boucle `do...while` : exécuter au moins une fois

Intuition. Il existe des situations où le traitement doit être fait au moins une fois avant de décider de continuer : demander un mot de passe (on le demande une première fois, puis on redemande tant qu'il est faux), afficher un menu, jouer une partie avant de proposer de rejouer. La boucle `do...while` teste la condition en sortie, c'est-à-dire après le corps.

PROPRIÉTÉ Syntaxe de la boucle `do...while`

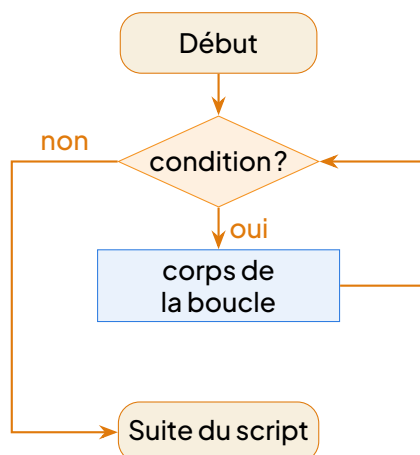
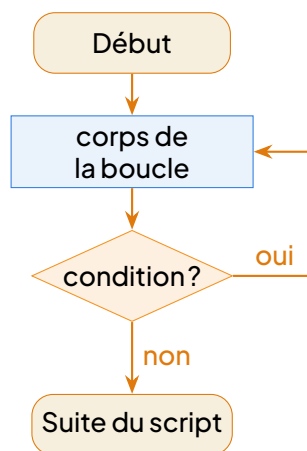
```
do{  
    // instructions executees AU MOINS une fois  
} while (condition); // attention au point-virgule final !
```

Le corps est exécuté une première fois sans aucun test, puis la condition est évaluée : si elle est vraie, on recommence ; sinon, on sort. Le nombre minimal d'exécutions est donc un. Ne jamais oublier le ; après la parenthèse fermante du `while`.

EXEMPLE Simulation d'une saisie contrôlée

```
<?php  
$essai = 0;  
$codeSecret = "1234"; // Code stocke en chaine de caracteres  
$saisies = array("0000", "1111", "1234"); // Simulations de saisies (chaines)  
  
do{  
    $essai++;  
    $saisie = $saisies[$essai - 1]; // Recuperation de la saisie simulee  
    echo "Essai $essai : code $saisie <br>";  
} while ($saisie !== $codeSecret && $essai < 3); // Comparaison stricte  
  
if ($saisie === $codeSecret){  
    echo "Acces autorise.";   
} else {  
    echo "Trop d'echecs. Compte verrouille.";   
}  
?>
```

Le corps s'exécute une première fois même si le premier code saisi est le bon : c'est toute la différence avec `while`. L'utilisation de chaînes de caractères pour les codes PIN évite le piège de l'interprétation octale des nombres commençant par 0 en PHP.

while : test en entrée**do...while : test en sortie**

Organigrammes comparés : avec `while`, le test précède le corps (0 exécution possible); avec `do...while`, le corps s'exécute d'abord (1 exécution minimum).

2.4 La boucle `foreach` : la reine du parcours de tableaux

Intuition. En PHP, les données viennent très souvent sous forme de tableaux : liste d'arrondissements, notes d'un élève, et surtout les résultats des requêtes MySQL que nous étudierons bientôt. Parcourir un tableau avec `for` et des indices est possible mais fastidieux (et impossible pour les tableaux associatifs dont les clés sont des chaînes). La boucle `foreach` est faite pour cela : elle visite automatiquement chaque élément, sans compteur à gérer.

PROPRIÉTÉ Les deux formes de `foreach`

```
// Forme 1 : on recupere seulement les valeurs
foreach ($tableau as $valeur) {
    // $valeur contient successivement chaque element
}

// Forme 2 : on recupere la cle ET la valeur
foreach ($tableau as $cle => $valeur) {
    // utile pour les tableaux associatifs
}
```

La boucle s'arrête d'elle-même à la fin du tableau : aucun risque de boucle infinie, aucun compteur à initialiser.

EXEMPLE Afficher les 20 arrondissements d'une région avec `foreach`

```
<?php
// Tableau indexé : arrondissements de la région du Centre (extrait)
$arrondissements = array(
    "Yaounde I", "Yaounde II", "Yaounde III", "Yaounde IV",
    "Yaounde V", "Yaounde VI", "Yaounde VII", "Mfoundi (rural)",
    "Mbankomo", "Okola", "Obala", "Nkolafamba",
    "Soa", "Akonolinga", "Mbalmayo", "Nkoteng",
    "Bafia", "Ntui", "Saa", "Esse"
);

echo "<h3>Les 20 arrondissements de la liste</h3>";
echo "<ol>";
foreach ($arrondissements as $ville) {
    echo "<li>$ville</li>"; // un élément de liste par arrondissement
}
echo "</ol>";

// Tableau associatif : région => chef-lieu
$chefsLieux = array(
    "Centre" => "Yaounde",
    "Littoral" => "Douala",
    "Ouest" => "Bafoussam",
    "Nord-Ouest" => "Bamenda"
);
foreach ($chefsLieux as $region => $capitale) {
    echo "Le chef-lieu de la région du $region est $capitale.<br>";
}
?>
```

La première boucle produit une liste numérotée HTML de 20 éléments ; la seconde exploite la forme `$cle => $valeur`, indispensable pour les tableaux associatifs.

LESAVAIS-TU ? foreach, la boucle reine du couple PHP/MySQL

Dans quelques sections, tu exécuteras des requêtes SQL depuis PHP grâce à l'extension `mysqli`. Chaque appel à `mysqli_fetch_assoc()` renvoie une ligne de résultat sous forme de tableau associatif (par exemple `$ligne["nom_eleve"]`). En pratique, on parcourt ces résultats avec un `while` (tant qu'il reste une ligne), puis on exploite chaque ligne comme un tableau — et dès que les résultats sont stockés dans un tableau PHP, c'est `foreach` qui prend le relais pour l'affichage. Maîtriser `foreach`, c'est déjà maîtriser la moitié de l'interfaçage PHP/MySQL.

2.5 Comparaison des quatre boucles et critères de choix

Boucle	Syntaxe	Position du test	Exécutions minimales	Cas d'usage typique
for	for (init ; cond ; incr)	En entrée	0	Nombre de tours connu à l'avance (tables, sommes, listes numérotées)
while	while (cond)	En entrée	0	Répéter tant qu'un événement imprévisible est vrai (lignes MySQL, saisie)
do...while	do {...} while (cond);	En sortie	1	Traitement à faire au moins une fois (menu, mot de passe)
foreach	foreach (\$tab as \$v)	Automatique	0	Parcours complet d'un tableau (indexé ou associatif)

Tableau récapitulatif des quatre boucles PHP : syntaxe, position du test, nombre minimal d'exécutions et cas d'usage.

MÉTHODE Choisir la bonne boucle

1. Tu parcoures un tableau ? → `foreach`, sans hésiter.
2. Tu connais le nombre exact de tours ? → `for`.
3. Tu répètes tant qu'une condition est vraie, sans connaître le nombre de tours ? → `while`.
4. Le corps doit s'exécuter au moins une fois avant tout test ? → `do...while`.

2.6 Le danger des boucles infinies

Une **boucle infinie** est une boucle dont la condition ne devient jamais fausse : le script tourne sans fin, consomme le processeur et finit par être tué par le serveur (ou bloque la page). Deux causes classiques : une condition mal conçue (toujours vraie) et l'**oubli de l'incrément** du compteur.

ATTENTION Erreur fréquente : oublier `$i++` dans un `while`

```
<?php
    $i = 1;
    while ($i <= 10) {
        echo "$i <br>";
        // OUBLI de $i++ : $i reste egal a 1 pour toujours
        // La condition $i <= 10 est TOUJOURS vraie :
        // boucle infinie qui bloque le serveur !
    }
?>
```

Sur un hébergement mutualisé, un tel script peut faire échouer toutes les pages du site. PHP finit par interrompre le script après le délai `max_execution_time` (30 secondes par défaut), mais le mal est fait : le visiteur n'a rien vu.

MÉTHODE Pour éviter toute boucle infinie, vérifier trois points

1. Initialisation : le compteur a-t-il une valeur de départ correcte avant la boucle ?
2. Condition d'arrêt : la condition peut-elle réellement devenir fausse un jour ? (Tester `$i <= 100` en incrémentant `$i` : oui. Tester `$i != 100` en partant de 1 par pas de 2 : non, car `$i` saute de 99 à 101!)
3. Modification du compteur : le corps de la boucle fait-il progresser le compteur vers la sortie (incrémentement, lecture de la ligne suivante, etc.) ?

EXERCICE RÉSOLU 2 Compter les élèves admis au Baccalauréat TI

Énoncé. Les notes sur 20 de 8 candidats sont stockées dans le tableau `$notes = array(12, 8, 15, 9, 10, 14, 7, 11)`. Un candidat est admis si sa note est supérieure ou égale à 10. Écrire un script PHP qui affiche chaque note avec la mention « Admis » ou « Refusé », puis le nombre total d'admis.

SOLUTION

Solution détaillée. On parcourt un tableau : la boucle adaptée est `foreach`. On utilise un compteur `$admis` initialisé à 0, incrémenté à chaque note admise grâce à une structure `if` (combinaison boucle + alternative, très fréquente au Baccalauréat).

```
<?php
    $notes = array(12, 8, 15, 9, 10, 14, 7, 11);
    $admis = 0;                // compteur d'admis
    $numero = 1;              // numero du candidat

    foreach ($notes as $note) {
        if ($note >= 10) {
            echo "Candidat $numero : $note/20 – Admis<br>";
        }
    }
?>
```

```
        $admis++;           // on incremente le compteur
    } else {
        echo "Candidat $numero : $note/20 – Refuse<br>";
    }
    $numero++;
}
echo "<strong>Nombre d'admis : $admis</strong>";
?>
```

Résultat affiché : les candidats 1, 3, 5, 6 et 8 sont admis ; le script conclut par « Nombre d'admis : 5 ». Remarque la structure générale : `foreach` pour le parcours, `if/else` pour la décision, compteur `$admis` pour l'accumulation — c'est le squelette de la plupart des traitements de données que tu rencontreras, y compris sur les résultats MySQL.

EXERCICE 2 À toi de jouer

1. Écris un script avec `for` qui affiche la table de multiplication de 7 (de 7×1 à 7×12).
2. Réécris ce script avec `while`, puis avec `do...while`. Que constates-tu si la condition de départ est fausse dans chacun des trois cas ?
3. Le tableau associatif `$operateurs = array("MTN" => "Mobile Money", "Orange" => "Orange Money", "Nexttel" => "Nexttel Possa")` associe un opérateur à son service de paiement. Affiche chaque couple sous la forme « MTN propose Mobile Money » à l'aide de `foreach`.

À RETENIR L'essentiel des structures répétitives

- Une boucle répète un bloc tant qu'une condition est vraie ; elle repose sur l'initialisation, la condition d'arrêt et l'incréméntation.
- `for` : nombre de tours connu ; `while` : test en entrée, 0 exécution possible ; `do...while` : test en sortie, 1 exécution minimum ; `foreach` : parcours automatique des tableaux.
- Toute boucle `for` peut se réécrire en `while` : la différence est syntaxique, pas fonctionnelle.
- Oublier l'incréméntation dans un `while` provoque une boucle infinie : toujours vérifier l'initialisation, la condition d'arrêt et la modification du compteur.
- `foreach` est la boucle reine de l'interfaçage PHP/MySQL : c'est elle qui parcourt les tableaux de résultats.

3 LA FONCTION DATE()

Dans la section précédente, tu as appris à répéter des traitements avec `for`, `foreach`, `while` et `do...while`. Ces boucles permettent, par exemple, d’afficher la liste des élèves inscrits à un cours. Mais une question se pose immédiatement dans toute application réelle : quand chaque élève s’est-il inscrit ? Le site `impots.cm` horodate chaque déclaration, l’application d’une startup camerounaise enregistre la date de chaque transaction Mobile Money, et ton projet scolaire de gestion de bibliothèque doit mémoriser la date d’emprunt de chaque livre. Toutes ces informations temporelles passent par une fonction PHP fondamentale : **`date()`**.

3.1 Rôle et syntaxe de la fonction `date()`

DÉFINITION La fonction `date()`

`date()` est une fonction prédéfinie de PHP qui retourne la date et l’heure courantes du serveur, formatées en une chaîne de caractères selon un modèle (format) fourni en paramètre.

```
string date(string $format, ?int $timestamp = null)
```

Le paramètre `$format` est une chaîne composée de codes de format (des lettres ayant chacune une signification précise) mélangés éventuellement à des caractères libres (slashes, deux-points, espaces, texte). Le second paramètre `$timestamp` est optionnel : s’il est omis, la fonction utilise l’heure courante (retournée par `time()`). La fonction retourne une chaîne que l’on peut afficher avec `echo` ou stocker dans une variable.

L’idée est simple : le serveur connaît en permanence la date et l’heure (grâce à son horloge système). La fonction `date()` ne fait que lire cette horloge et présenter le résultat comme tu le souhaites. Tu ne choisis pas la date, tu choisis seulement son **format d’affichage**.

EXEMPLE Premiers pas avec `date()`

```
<?php
// Affiche par exemple : 15/03/2025
echo date('d/m/Y');

echo "<br>";

// Affiche par exemple : 14:30:05
echo date('H:i:s');

echo "<br>";

// Stocke la date du jour dans une variable
$aujourd'hui = date('Y-m-d');
echo "Nous sommes le " . $aujourd'hui;
```

```
?>
```

Chaque lettre du format est remplacée par la valeur correspondante de l'horloge du serveur ; les caractères / et : sont recopiés tels quels.

À RETENIR Exemple chiffré de référence

Si l'horloge du serveur indique le 15 mars 2025 à 14 h 30, alors l'instruction :

```
echo date('d/m/Y à H:i');
```

affiche exactement : 15/03/2025 à 14 :30. Le mot « à » est du texte libre, recopié tel quel dans le résultat.

3.2 Les principaux codes de format

Le paramètre de `date()` est un mini-langage : chaque lettre a un sens. Il faut les connaître par cœur, car elles tombent régulièrement au Baccalauréat. Voici les codes essentiels au programme de Terminale TI.

PROPRIÉTÉ Codes de format de `date()`

Code	Signification	Exemple de sortie
d	Jour du mois, sur 2 chiffres (01 à 31)	15
j	Jour du mois, sans zéro initial (1 à 31)	15 (ou 5 le 5 du mois)
m	Mois, sur 2 chiffres (01 à 12)	03
M	Mois, abréviation anglaise sur 3 lettres	Mar
Y	Année sur 4 chiffres	2025
H	Heure, format 24 h, sur 2 chiffres (00 à 23)	14
i	Minutes, sur 2 chiffres (00 à 59)	30
s	Secondes, sur 2 chiffres (00 à 59)	05
l	Jour de la semaine, en toutes lettres (anglais)	Saturday

Tous les exemples de sortie correspondent au samedi 15 mars 2025 à 14 h 30 min 05 s.

Code	Format complet	Résultat affiché
d/m/Y	Date française classique	15/03/2025
H:i:s	Heure complète (24 h)	14:30:05
Y-m-d	Format SQL (champ DATE)	2025-03-15
l j M Y	Date en toutes lettres (anglais)	Saturday 15 Mar 2025
Y-m-d H:	Format SQL (champ DATETIME)	2025-03-15 14:30:05

Formats usuels de `date()` et leurs sorties pour le 15/03/2025 à 14 h 30 min 05 s.

ATTENTION Erreur fréquente : confondre m et i

Le piège classique du Bac : `m` désigne le mois et `i` les minutes. Ainsi, `date('H:m:s')` n'affiche pas l'heure avec les minutes : il affiche l'heure, puis le mois, puis les secondes ! Le 15 mars à 14 h 30, on obtiendrait 14:03:05 au lieu de 14:30:05. Retiens : mois = `m`, minutes = `i`. De même, `H` (majuscule) est l'heure sur 24 h, alors que `h` (minuscule) est l'heure sur 12 h (avec AM/PM).

3.3 Le problème du fuseau horaire

Voici une situation vécue par beaucoup de développeurs débutants : tu installes WampServer, tu écris `echo date('H:i');` ; à 10 h du matin à Yaoundé, et la page affiche... 09 h ou 11 h. Que s'est-il passé ?

La fonction `date()` lit l'heure du serveur dans le fuseau horaire configuré par PHP. Par défaut, cette configuration est souvent UTC (temps universel) ou le fuseau du pays d'origine de l'hébergeur. Or le Cameroun se trouve dans le fuseau UTC+1 (heure d'Afrique de l'Ouest, sans changement d'heure saisonnier). Si PHP est réglé sur UTC, toutes tes heures seront décalées d'une heure : un élève inscrit à 08 h 00 précises sera enregistré à 07 h 00. Pour un système de notes ou de présence, c'est inacceptable.

PHP fournit heureusement une fonction pour corriger cela : `date_default_timezone_set()`.

MÉTHODE Obtenir l'heure exacte du Cameroun

Pour que toutes les dates et heures de ton script correspondent à l'heure locale camerounaise :

1. Place, au tout début du script (avant tout appel à `date()`), l'instruction :

```
date_default_timezone_set('Africa/Douala');
```

2. Utilise ensuite `date()` normalement : elle retournera l'heure de Douala et Yaoundé (UTC+1).

3. Si ton projet comporte plusieurs pages, recopie cette ligne en tête de chaque script concerné (ou place-la dans un fichier `config.php` inclus partout via `require_once`).

EXEMPLE Heure locale correcte

```
<?php
// Réglage du fuseau horaire sur le Cameroun (UTC+1)
date_default_timezone_set('Africa/Douala');

// Désormais, date() retourne l'heure locale exacte
echo "Il est " . date('H:i:s') . " à Yaoundé.";
// Affiche par exemple : Il est 10 :15 :42 à Yaoundé.
?>
```

ATTENTION Oublier le fuseau horaire

Si tu oublies `date_default_timezone_set()`, PHP utilise le fuseau par défaut du serveur, qui n'est pas forcément celui du Cameroun, surtout si ton site est hébergé en Europe ou en Amérique. Conséquence : des horodatages faux, incohérents entre le serveur et les utilisateurs. En production, vérifie aussi la directive `date.timezone` du fichier `php.ini` pour une configuration globale.

3.4 Applications concrètes de `date()`

1. **Afficher la date du jour sur une page.** C'est l'usage le plus simple : un bandeau en haut du site de ton établissement, par exemple.

```
<?php
date_default_timezone_set('Africa/Douala');
echo "<p>Lycee de Nkol-Eton – Nous sommes le "
    . date('d/m/Y') . "</p>";
?>
```

2. **Horodatage d'une inscription.** Quand un utilisateur s'inscrit, on mémorise le moment exact de son inscription dans une variable, que l'on enregistrera ensuite dans la base de données.

```
<?php
date_default_timezone_set('Africa/Douala');

// Moment exact de l'inscription, au format DATETIME de MySQL
$date_inscription = date('Y-m-d H:i:s');
echo "Inscription enregistrée le " . $date_inscription;
// Exemple : Inscription enregistrée le 2025-03-15 14 :30 :05
?>
```

3. Calcul d'ancienneté. En combinant `date()` avec une date stockée, on peut calculer une durée. Par exemple, l'année courante moins l'année d'embauche donne l'ancienneté d'un employé :

```
<?php
$annee_embauche = 2019;
$anciennete = date('Y') - $annee_embauche;
// date('Y') retourne l'année courante, par ex. 2025 (chaîne convertie en entier)
echo "Anciennete : " . $anciennete . " ans"; // 6 ans
?>
```

3.5 Stocker une date dans MySQL

C'est ici que `date()` prend toute son importance pour la suite de la leçon (l'interfaçage avec MySQL). MySQL possède des types de colonnes spécialisés pour les dates :

Type MySQL	Contenu	Format attendu
DATE	Une date seule (jour, mois, année)	AAAA-MM-JJ
DATETIME	Une date avec l'heure	AAAA-MM-JJ HH:MM:SS

Le point capital : MySQL attend les dates au format **anglo-saxon** AAAA-MM-JJ (année d'abord), et non au format français JJ/MM/AAAA. Le format PHP correspondant est donc `date('Y-m-d')` pour un champ DATE et `date('Y-m-d H:i:s')` pour un champ DATETIME.

EXEMPLE Insérer la date du jour dans une table

Supposons une table `membre(id, nom, date_inscription)` où `date_inscription` est de type DATE :

```
<?php
date_default_timezone_set('Africa/Douala');

// Date du jour au format accepté par MySQL
$jour = date('Y-m-d'); // ex. 2025-03-15

// Construction de la requête d'insertion
$sql = "INSERT INTO membre (nom, date_inscription)
VALUES ('Amina', '$jour')";
// La requête générée est :
// INSERT INTO membre (nom, date_inscription)
// VALUES ('Amina', '2025-03-15')
?>
```

Nous verrons dans les sections suivantes comment exécuter cette requête avec `mysqli_query()`.

ATTENTION Faille de sécurité : Injection SQL

Dans l'exemple ci-dessus, la variable `$jour` (ou `$nom`) est insérée directement dans la chaîne de la requête SQL. Si la valeur provient d'une saisie utilisateur (formulaire `$_POST`), un attaquant pourrait injecter du code SQL malveillant (ex. `' OR '1'='1'`). C'est la faille n°1 du web : l'injection SQL.

- Ne jamais concaténer des variables non contrôlées dans une requête SQL.
- La parade standard avec `mysqli` est l'utilisation de requêtes préparées (`mysqli_prepare`, `bind_param`, `execute`), que nous verrons en détail dans la section « Exécution des requêtes ».
- En attendant, pour les exercices où les valeurs sont fixes (codées en dur dans le script), la concaténation est tolérée pour la simplicité pédagogique.

LESAVAIS-TU ? Et côté MySQL ?

MySQL possède ses propres fonctions de date : `NOW()` retourne la date et l'heure courantes, `CURDATE()` la date seule. On peut donc écrire directement `INSERT INTO membre (nom, date_inscription) VALUES ('Amina', CURDATE())` sans passer par PHP. Alors pourquoi utiliser `date()` côté PHP ? Parce qu'elle permet de contrôler et d'afficher la date avant l'enregistrement, et de la réutiliser dans le script (ex. dans un email de confirmation). Au Baccalauréat, la démarche attendue est le plus souvent : `$d = date('Y-m-d');` ; puis insertion de `$d` dans la requête.

EXERCICE RÉSOLU 3 Horodatage d'une inscription à un concours

Énoncé : le site d'un concours d'entrée dans une école de Douala doit, lors de l'inscription d'un candidat, afficher le message « Inscription de nom validée le JJ/MM/AAAA à HH:MM » et préparer la date au format MySQL pour un champ `DATETIME`. Le nom du candidat est "Biya Steve". Écris le script PHP complet (heure du Cameroun).

SOLUTION

Solution détaillée :

```
<?php
// Étape 1 : régler le fuseau horaire sur le Cameroun
date_default_timezone_set('Africa/Douala');

// Étape 2 : récupérer le nom du candidat
$nom = "Biya Steve";

// Étape 3 : préparer les deux formats de date
$date_affichee = date('d/m/Y à H:i'); // format français
$date_mysql = date('Y-m-d H:i:s'); // format DATETIME
```

```
// Étape 4 : afficher le message de confirmation
echo "Inscription de $nom validée le $date_affichee";
// Ex. : Inscription de Biya Steve validée le 15/03/2025 à 14 :30

// Étape 5 : préparer la requête SQL (exécution vue plus loin)
// ATTENTION : en situation réelle, utiliser une requête préparée !
$sql = "INSERT INTO candidat (nom, date_inscription)
VALUES ('$nom', '$date_mysql')";

?>
```

Points de vigilance : le fuseau horaire est réglé avant tout appel à `date()` ; on utilise bien `i` (minutes) et non `m` (mois) ; le format d’affichage français (d/m/Y) est distingué du format de stockage MySQL (Y-m-d H:i:s).

EXERCICE 3 À toi de jouer

Écris un script PHP qui :

1. règle le fuseau horaire sur le Cameroun ;
2. affiche « Bonjour, nous sommes le 15/03/2025 et il est 14 :30 » (avec les valeurs réelles du serveur) ;
3. stocke dans une variable `$aujourd'hui` la date du jour au format d’un champ DATE MySQL, puis construit la requête insérant un emprunt du livre "Les Misérables" à cette date dans une table `emprunt(titre, date_emprunt)`.

CORRIGÉ Exercice 1

```
<?php
// 1. Fuseau horaire du Cameroun
date_default_timezone_set('Africa/Douala');

// 2. Message d’accueil avec date et heure
echo "Bonjour, nous sommes le " . date('d/m/Y')
    . " et il est " . date('H:i');

// 3. Date au format DATE de MySQL + requête
$aujourd'hui = date('Y-m-d');
$sql = "INSERT INTO emprunt (titre, date_emprunt)
VALUES ('Les Miserables', '$aujourd'hui')";
echo $sql; // pour vérification

?>
```

Tu sais désormais obtenir, formater et préparer la date et l’heure du serveur. La prochaine étape est naturelle : ces dates doivent être enregistrées quelque part. C’est là qu’intervient la connexion

de PHP au SGBD MySQL grâce à l'extension `mysqli`, objet de la section suivante.

4 CONNEXION À MYSQL AVEC L'EXTENSION MYSQLI

Dans la section précédente, nous avons appris à manipuler la fonction `date()` pour produire dynamiquement la date et l'heure côté serveur. Tu sais désormais écrire des scripts PHP qui calculent, testent et bouclent. Mais une application web réelle — un site de gestion de notes, une boutique en ligne, un portail de paiement Mobile Money (MTN MoMo, Orange Money) — ne peut pas garder ses données dans de simples variables : celles-ci disparaissent dès que le script se termine. Il faut un serveur de données pour stocker les informations de façon permanente. C'est là qu'intervient **MySQL**, et c'est l'objet de cette section : apprendre à **connecter un script PHP à une base MySQL** grâce à l'extension `mysqli`.

4.1 Architecture client/serveur : qui parle à qui ?

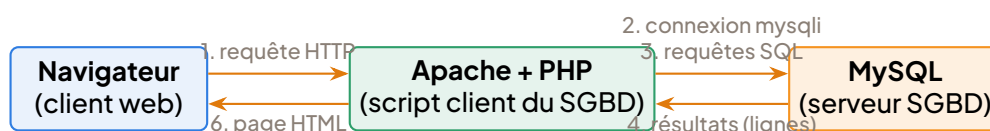
DÉFINITION SGBD et serveur MySQL

Un **SGBD** (Système de Gestion de Bases de Données) est un logiciel serveur qui stocke, organise et protège les données, et qui répond aux **requêtes** exprimées en langage SQL. **MySQL** est le SGBD libre le plus utilisé avec PHP ; il est installé automatiquement avec WampServer et XAMPP.

Lorsqu'un internaute visite une page dynamique, trois acteurs collaborent :

1. le **navigateur** (client web) envoie une requête HTTP au serveur web ;
2. le serveur **Apache** exécute le script **PHP**, qui joue alors le rôle de client du SGBD : il ouvre une connexion vers **MySQL**, envoie des requêtes SQL et récupère les résultats ;
3. PHP assemble la page **HTML** finale, qu'Apache renvoie au navigateur.

Point capital à bien comprendre : **le navigateur ne dialogue jamais directement avec MySQL**. Toutes les requêtes SQL transitent par le script PHP, qui est le seul à connaître les identifiants de connexion. C'est une garantie de sécurité : le code PHP n'est jamais envoyé au client, seul le HTML produit l'est.



Architecture client/serveur d'une application PHP/MySQL : le script PHP est l'intermédiaire obligé entre le navigateur et la base de données.

LESAVAIS-TU ? Le « i » de mysqli

L'extension `mysqli` signifie MySQL improved (« MySQL amélioré »). Elle a remplacé l'ancienne extension `mysql_` (sans le « i »), jugée obsolète et supprimée définitivement depuis PHP 7. Les anciens scripts utilisant `mysql_connect()` ne fonctionnent donc plus du tout sur les serveurs modernes. Au Baccalauréat TI, on exige l'écriture `mysqli` : un script avec `mysql_connect()` est considéré comme faux. Au Cameroun, la quasi-totalité des hébergements mutualisés (ceux utilisés par les startups et les sites d'administrations) tournent sous PHP 7 ou 8 : seul `mysqli` (ou PDO) y est accepté.

4.2 Établir la connexion : `mysqli_connect()`

Avant de poser la moindre question à MySQL, le script doit ouvrir une connexion, exactement comme on compose un numéro avant de parler au téléphone. Il faut fournir trois informations : où se trouve le serveur (l'hôte), qui se connecte (l'utilisateur) et son mot de passe.

DÉFINITION `mysqli_connect`

`mysqli_connect` est la fonction qui ouvre une connexion entre le script PHP et le serveur MySQL. Elle retourne un **objet** `mysqli` (identifiant de connexion représentant le canal de communication), que l'on conserve dans une variable pour toutes les opérations suivantes. En cas d'échec, elle retourne `false`.

PROPRIÉTÉ Syntaxe de `mysqli_connect`

```
$cnx = mysqli_connect(hote, utilisateur, mot_de_passe);
```

Sous WampServer ou XAMPP (configuration par défaut en salle informatique), les valeurs typiques sont :

- `hote` : `'localhost'` (MySQL tourne sur la même machine que PHP) ;
- `utilisateur` : `'root'` (super-utilisateur créé à l'installation) ;
- `mot_de_passe` : `''` (chaîne vide par défaut sous WAMP/XAMPP).

ATTENTION « root » sans mot de passe : uniquement en local !

La configuration `('localhost', 'root', '')` est acceptable uniquement sur le poste de développement (salle informatique du lycée, machine personnelle). Sur un serveur en production, utiliser `root` sans mot de passe est une faille de sécurité majeure : n'importe

qui lisant le script obtient un accès total à toutes les bases. En production, on crée un utilisateur MySQL dédié, avec un mot de passe fort et des droits limités à la seule base de l'application.

LESAVAIS-TU ? Paramètre « base de données » de `mysqli_connect`

La fonction `mysqli_connect` accepte un 4^e paramètre optionnel : le nom de la base de données à sélectionner immédiatement.

```
$cnx = mysqli_connect('localhost', 'root', '', 'ecole');
```

Cela évite l'appel séparé à `mysqli_select_db`. Toutefois, le programme officiel exige la maîtrise de `mysqli_select_db` comme étape distincte ; aux examens, privilégie la méthode en deux étapes (connexion puis sélection) pour faire apparaître explicitement la logique.

4.3 Gérer les erreurs de connexion : `die()` et `mysqli_connect_error()`

Que se passe-t-il si MySQL est éteint, si le mot de passe est faux ou si le serveur est injoignable ? `mysqli_connect` retourne `false`, et toutes les requêtes suivantes échoueraient en cascade avec des messages d'erreur peu lisibles. La bonne pratique consiste à **arrêter immédiatement le script** avec un message clair.

DÉFINITION `die`

`die(message)` est la fonction qui arrête immédiatement l'exécution du script en affichant le message passé en argument. Aucune ligne située après `die()` n'est exécutée. On l'utilise pour bloquer proprement le script lorsqu'une opération indispensable (comme la connexion) a échoué. Son alias `exit()` est strictement équivalent.

On combine `die()` avec l'opérateur `or` et la fonction `mysqli_connect_error()`, qui retourne une description textuelle de la dernière erreur de connexion (par exemple : Access denied for user 'root'@'localhost').

MÉTHODE Séquence type de connexion à une base MySQL

La connexion complète se fait toujours en deux étapes protégées par `die()`, suivies de la définition du jeu de caractères (indispensable pour les accents) :

1. Ouvrir la connexion au serveur et vérifier qu'elle a réussi :

```
$cnx = mysqli_connect('localhost', 'root', '')  
or die('Erreur de connexion : ' . mysqli_connect_error());
```

2. Sélectionner la base de données sur laquelle on va travailler :

```
mysqli_select_db($cnx, 'ma_base')  
or die('Base introuvable');
```

3. Définir le jeu de caractères (UTF-8) pour éviter les problèmes d'accents :

```
mysqli_set_charset($cnx, 'utf8mb4')  
or die('Erreur encodage : ' . mysqli_error($cnx));
```

Si ces trois lignes sont passées sans message d'erreur, la variable `$cnx` contient une connexion valide, correctement encodée, et prête à recevoir des requêtes SQL.

4.4 Choisir la base de données : `mysqli_select_db()`

Un serveur MySQL héberge en général plusieurs bases de données (sous WampServer, on en voit déjà plusieurs dès l'installation : `mysql`, `information_schema`, `performance_schema`, etc.). Se connecter au serveur ne suffit donc pas : il faut indiquer **sur quelle base** portent les requêtes.

PROPRIÉTÉ Syntaxe de `mysqli_select_db`

```
mysqli_select_db($cnx, 'nom_de_la_base');
```

Le premier paramètre est l'identifiant de connexion retourné par `mysqli_connect`, le second est le nom de la base (sensible à la casse sur certains serveurs Linux). La fonction retourne `true` en cas de succès, `false` sinon — d'où l'usage systématique de `or die(...)`.

ATTENTION Erreur fréquente : oublier `mysqli_select_db`

C'est le bug classique du débutant : la connexion réussit, mais toutes les requêtes échouent avec l'erreur « No database selected » (« aucune base sélectionnée »). La cause est presque toujours l'oubli de `mysqli_select_db($cnx, ...)` après `mysqli_connect`. Vérifie aussi que le nom de la base est exactement celui créé dans phpMyAdmin (respect des majuscules/minuscules et des underscores : `ecole` n'est pas `Ecole`).

4.5 Jeu de caractères : `mysqli_set_charset()`

Sans instruction explicite, la connexion peut utiliser un encodage par défaut (souvent `latin1`) qui corrompt les caractères accentués (é, è, à, ç, ô) lors des insertions ou des lectures. Au Cameroun, où les noms propres contiennent de nombreux accents (Bafoussam, Yaoundé, Nkong-samba, Mbappé), cette étape est **obligatoire**.

PROPRIÉTÉ Syntaxe de `mysqli_set_charset`

```
| mysqli_set_charset($cnx, 'utf8mb4');
```

Utilise `utf8mb4` (support complet d'Unicode, y compris émojis) plutôt que l'ancien `utf8`.
Place cet appel immédiatement après `mysqli_select_db`.

4.6 Fermer la connexion : `mysqli_close()`

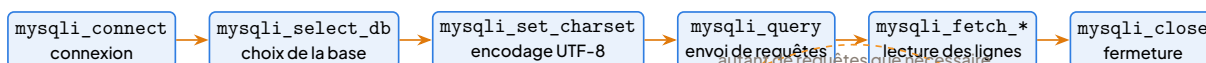
Une connexion ouverte consomme des ressources sur le serveur MySQL (mémoire, processus, verrous). Un bon script libère donc la connexion dès qu'il n'en a plus besoin, en toute fin de script :

PROPRIÉTÉ Syntaxe de `mysqli_close`

```
| mysqli_close($cnx);
```

La fonction ferme la connexion identifiée par `$cnx` et retourne `true` en cas de succès.
Après cet appel, la variable `$cnx` n'est plus utilisable pour des requêtes.

En pratique, PHP ferme automatiquement les connexions à la fin du script ; néanmoins, l'appel explicite à `mysqli_close()` est une **bonne pratique exigée dans les rédactions au Bac** : elle montre que tu maîtrises le cycle de vie complet de la connexion.



Cycle de vie complet d'une connexion MySQL dans un script PHP : connecter, sélectionner la base, définir l'encodage, interroger, lire, fermer.

4.7 Script type complet et réutilisable : `connexion.php`

Voici le script de connexion complet, commenté ligne par ligne, tel qu'on l'attend au Baccalauréat. Il se connecte au serveur, choisit la base `ecole`, force l'encodage UTF-8 et affiche un message de confirmation.

EXEMPLE Script `connexion.php` complet et commenté

```
| <?php
| // =====
| // connexion.php : connexion a la base "ecole"
```

```
// =====  
  
// 1. Ouverture de la connexion au serveur MySQL  
// En cas d'échec, die() stoppe le script avec un message clair  
$cnx = mysqli_connect('localhost', 'root', '')  
    or die('Erreur de connexion au serveur : ' . mysqli_connect_error());  
  
// 2. Selection de la base de donnees "ecole"  
mysqli_select_db($cnx, 'ecole')  
    or die('Base de donnees introuvable');  
  
// 3. Definition du jeu de caracteres (UTF-8 complet)  
mysqli_set_charset($cnx, 'utf8mb4')  
    or die('Erreur encodage : ' . mysqli_error($cnx));  
  
// 4. Si le script arrive ici, tout s'est bien passe  
echo 'Connexion reussie a la base ecole';  
  
// 5. Fermeture de la connexion (bonne pratique)  
mysqli_close($cnx);  
?>
```

Testé sous WampServer avec une base `ecole` existante, ce script affiche simplement : Connexion réussie à la base ecole. Si MySQL est arrêté, on obtient par exemple : Erreur de connexion au serveur : Connection refused. Si la base n'existe pas : Base de données introuvable. Si l'encodage échoue (très rare) : Erreur encodage : Chaque cas d'échec produit donc un message explicite, ce qui facilite énormément le débogage.

4.8 Bonnes pratiques : mutualiser la connexion avec `include` / `require`

Dans une application réelle, tous les scripts (liste des élèves, saisie des notes, bulletins, paiement Mobile Money...) ont besoin de la connexion. Recopier les mêmes lignes partout est une mauvaise idée : si le mot de passe change, il faudrait modifier des dizaines de fichiers. La solution professionnelle consiste à écrire la connexion **une seule fois** dans un fichier `connexion.php`, puis à l'inclure dans chaque script.

PROPRIÉTÉ Inclure le fichier de connexion

```
<?php  
// liste_eleves.php : affiche les eleves  
require('connexion.php'); // $cnx est maintenant disponible  
  
// ... requetes SQL avec $cnx ...  
  
mysqli_close($cnx);  
?>
```

- `include('connexion.php')` : en cas de fichier absent, émet un avertissement (Warning) et continue le script;

- `require('connexion.php')` : en cas de fichier absent, provoque une erreur fatale (Fatal error) et arrête le script.

Pour une connexion à la base, on préfère `require` : sans connexion, le reste du script n'a aucun sens.

Dans ce cas, le fichier `connexion.php` ne contient que les étapes 1 à 3 (connexion, sélection de la base, encodage), **sans** le `echo` de confirmation ni le `mysqli_close()` — la fermeture reste à la charge de chaque script appelant.

EXEMPLE Fichier `connexion.php` mutualisé (sans echo, sans close)

```
<?php
// connexion.php : a inclure avec require()
// Ne contient PAS mysqli_close()

$cnx = mysqli_connect('localhost', 'root', '')
    or die('Erreur de connexion : ' . mysqli_connect_error());

mysqli_select_db($cnx, 'ecole')
    or die('Base ecole introuvable');

mysqli_set_charset($cnx, 'utf8mb4')
    or die('Erreur encodage : ' . mysqli_error($cnx));
?>
```

EXERCICE RÉSOLU 4 Écrire la connexion à la base d'un cybercafé

Énoncé. Un cybercafé de Bafoussam gère ses postes clients dans une base MySQL nommée `cybercafe`, hébergée en local sous WampServer. Écris le script `connexion.php` mutualisé permettant de s'y connecter, avec gestion des erreurs et encodage UTF-8, puis le début du script `postes.php` qui l'utilise.

SOLUTION

Solution détaillée. On applique la séquence type en trois étapes protégées par `die()`, puis l'inclusion dans le script métier avec `require`.

```
<?php
// connexion.php (mutualise)
$cnx = mysqli_connect('localhost', 'root', '')
    or die('Erreur de connexion : ' . mysqli_connect_error());

mysqli_select_db($cnx, 'cybercafe')
    or die('Base cybercafe introuvable');
```

```
mysqli_set_charset($cnx, 'utf8mb4')
    or die('Erreur encodage : ' . mysqli_error($cnx));
?>
```

```
<?php
// postes.php
require('connexion.php'); // connexion, base, encodage prêts

echo 'Connexion réussie à la base cybercafe';

// ... ici : requetes SQL sur la table des postes ...

mysqli_close($cnx);    // liberation de la connexion
?>
```

Points de vigilance : ne pas oublier `mysqli_select_db` (sinon No database selected), ne pas oublier `mysqli_set_charset` (sinon accents corrompus), utiliser `require` plutôt qu'`include` pour la connexion, et fermer la connexion en fin de script métier.

EXPÉRIENCE TP guidé : tester sa première connexion PHP/MySQL

Objectif : vérifier expérimentalement le cycle complet de connexion sous WampServer.

Matériel/logiciels : un poste avec WampServer (ou XAMPP) installé, un navigateur.

Manipulation :

1. Lance WampServer et attends que l'icône soit verte (Apache et MySQL démarrés).
2. Ouvre phpMyAdmin (<http://localhost/phpmyadmin>), onglet Bases de données, et crée une base nommée `ecole` (interclassement `utf8mb4_general_ci`).
3. Dans le dossier `C:\wamp64\www`, crée un dossier `test`, puis le fichier `connexion.php` en recopiant le script complet de l'exemple ci-dessus (avec `echo` et `close`).
4. Dans le navigateur, saisis <http://localhost/test/connexion.php> et observe le message.
5. Arrête le service MySQL (clic gauche sur l'icône Wamp → MySQL → Arrêter le service) et recharge la page.
6. Redémarre MySQL, renomme la base en `ecole2` dans phpMyAdmin, et recharge encore la page.
7. (Optionnel) Modifie `connexion.php` pour mettre un mauvais mot de passe (`'pwd'`) et teste.

Observations : à l'étape 4, le message Connexion réussie à la base `ecole` s'affiche ; à l'étape 5, `die()` affiche l'erreur de connexion au serveur (ex. Connection refused) ; à l'étape 6, le message Base de données introuvable apparaît ; à l'étape 7, Access denied for user 'root'@'localhost'.

Interprétation : chaque étape de la séquence de connexion est protégée indépendam-

ment, ce qui permet d'identifier précisément l'origine d'une panne — compétence indispensable pour déboguer une application réelle.

À RETENIR L'essentiel de la connexion PHP/MySQL

- Le script PHP est le client du serveur MySQL ; le navigateur ne voit jamais ni la base ni le code PHP.
- `$cnx = mysqli_connect('localhost','root','') or die(...)` ouvre la connexion ; `mysqli_connect_error()` décrit l'erreur éventuelle.
- `mysqli_select_db($cnx, 'base') or die(...)` choisit la base — son oubli provoque **No database selected**.
- `mysqli_set_charset($cnx, 'utf8mb4')` garantit l'intégrité des accents (é, è, à, ç...).
- `mysqli_close($cnx)` libère la connexion en fin de script.
- On mutualise la connexion dans un fichier `connexion.php` inclus avec `require`.
- Au Bac : on écrit `mysqli`, jamais l'ancienne extension `mysql_` supprimée depuis PHP 7.

Maintenant que le canal de communication est ouvert, sécurisé et correctement encodé, il reste à l'exploiter : la section suivante montrera comment **exécuter des requêtes SQL** (insertion, modification, suppression, sélection) depuis PHP avec `mysqli_query()`, puis comment lire et afficher les résultats avec `mysqli_fetch_assoc()`, `mysqli_fetch_row()`, `mysqli_num_rows()`.

5 EXÉCUTION DES REQUÊTES SQL DEPUIS PHP

Dans la section précédente, nous avons appris à ouvrir une connexion au serveur MySQL avec `mysqli_connect()` et à choisir une base de travail avec `mysqli_select_db()`. Mais une connexion ouverte ne fait rien toute seule : c'est un simple **tuyau** entre ton script PHP et le serveur MySQL. Il faut maintenant faire circuler des ordres dans ce tuyau : insérer un élève, modifier une note, supprimer un candidat, consulter la liste des inscrits. C'est précisément le rôle de la fonction `mysqli_query()`, véritable cœur de l'interfaçage PHP/MySQL. C'est elle qui transforme ton site statique en véritable **application web** pilotée par une base de données.

5.1 La fonction `mysqli_query()` : envoyer un ordre au serveur

DÉFINITION La fonction `mysqli_query`

`mysqli_query()` est la fonction de l'extension **mysqli** qui envoie une requête SQL au serveur MySQL, sur la connexion et la base précédemment sélectionnées. Sa syntaxe géné-

rale est :

```
$resultat = mysqli_query($cnx, $sql);
```

où `$cnx` est l'identifiant de connexion retourné par `mysqli_connect()` et `$sql` une chaîne de caractères contenant la requête SQL.

PROPRIÉTÉ Valeur de retour de `mysqli_query`

La valeur retournée dépend du type de requête exécutée :

- pour INSERT, UPDATE, DELETE : la fonction retourne `true` en cas de succès, `false` en cas d'échec ;
- pour SELECT (et SHOW, DESCRIBE) : elle retourne un **jeu de résultats** (objet de type `mysqli_result`) en cas de succès, `false` en cas d'échec.

En cas d'échec, la fonction `die(mysqli_error($cnx))` permet d'arrêter le script en affichant le message d'erreur exact renvoyé par MySQL.

ATTENTION `false` n'est pas un jeu de résultats vide

Deux situations très différentes ne doivent pas être confondues :

- un SELECT qui ne trouve aucune ligne retourne un jeu de résultats vide (mais valide) : `mysqli_query()` a réussi ;
- un SELECT syntaxiquement faux (table inexistante, faute de frappe dans un nom de colonne) retourne `false`.

On teste donc toujours le retour avec `if ($resultat === false)` ou `if (!$resultat)` avant d'exploiter les données.

MÉTHODE Exécuter une requête SQL depuis PHP : la démarche en trois temps

Quelle que soit la requête, la démarche est toujours la même :

1. Construire la chaîne SQL dans une variable, par exemple `$sql = "INSERT INTO eleve ..."` ; — cela rend le script lisible et facilite le débogage (on peut afficher `$sql` pour vérifier la requête) ;
2. Envoyer la requête avec `$resultat = mysqli_query($cnx, $sql) ;`
3. Tester le résultat : si succès, afficher un message de confirmation ou exploiter le jeu de résultats ; sinon, arrêter proprement avec `die(mysqli_error($cnx))`.

5.2 Insertion : la requête INSERT alimentée par un formulaire

Intuition. Le cas le plus fréquent dans une application web camerounaise — inscription d'un élève, enregistrement d'un client Mobile Money, dépôt d'une candidature — consiste à récupérer les données saisies dans un formulaire HTML et à les enregistrer dans une table. Le tableau superglobal `$_POST` contient les valeurs envoyées par le formulaire ; il suffit de les insérer dans la chaîne SQL.

PROPRIÉTÉ Syntaxe de la requête INSERT

```
INSERT INTO nom_table (colonne1, colonne2, ...)  
VALUES (valeur1, valeur2, ...);
```

Depuis PHP, on construit cette chaîne en y injectant les variables issues de `$_POST` par concaténation ou interpolation.

ATTENTION Apostrophes obligatoires autour des valeurs textuelles

Dans une requête SQL, les valeurs de type texte (VARCHAR, CHAR, DATE...) doivent être entourées d'**apostrophes**, tandis que les valeurs numériques s'écrivent sans apostrophes :

```
$sql = "INSERT INTO eleve (nom, age) VALUES ('$nom', $age)";  
// Correct : 'Ngo Bell' entre apostrophes, 17 sans apostrophes
```

Oublier les apostrophes autour d'une chaîne provoque une erreur SQL (false retourné). C'est l'erreur la plus fréquente des débutants.

5.3 Modification : la requête UPDATE

Intuition. Un élève change de classe, un client modifie son numéro de téléphone : il faut mettre à jour des enregistrements existants sans les recréer.

PROPRIÉTÉ Syntaxe de la requête UPDATE

```
UPDATE nom_table  
SET colonne1 = valeur1, colonne2 = valeur2  
WHERE condition;
```

La clause `WHERE` désigne précisément les lignes à modifier, le plus souvent par la clé primaire (`WHERE id = 5`).

EXEMPLE Corriger la classe d'un élève

```
$sql = "UPDATE eleve SET classe = 'Tle TI' WHERE id = 12";  
if (mysqli_query($cnx, $sql)) {  
    echo "Enregistrement modifié avec succès."  
} else {  
    die("Erreur : " . mysqli_error($cnx));  
}
```

5.4 Suppression : la requête DELETE

Intuition. Un candidat se désiste, un produit est retiré du catalogue : on supprime la ligne correspondante de la table.

PROPRIÉTÉ Syntaxe de la requête DELETE

```
DELETE FROM nom_table WHERE condition;
```

ATTENTION Le danger du UPDATE ou DELETE sans WHERE

Sans clause WHERE, la requête s'applique à toutes les lignes de la table :

- UPDATE eleve SET classe = 'Tle C' re-classe tous les élèves de l'établissement;
- DELETE FROM eleve vide entièrement la table, sans demander de confirmation!

Règle d'or : un UPDATE ou un DELETE doit toujours comporter un WHERE ciblant la clé primaire. Avant d'exécuter une suppression en production, on teste d'abord la condition avec un SELECT : SELECT * FROM eleve WHERE id = 12 permet de vérifier qu'on vise bien la bonne ligne.

5.5 Sélection : la requête SELECT et le jeu de résultats

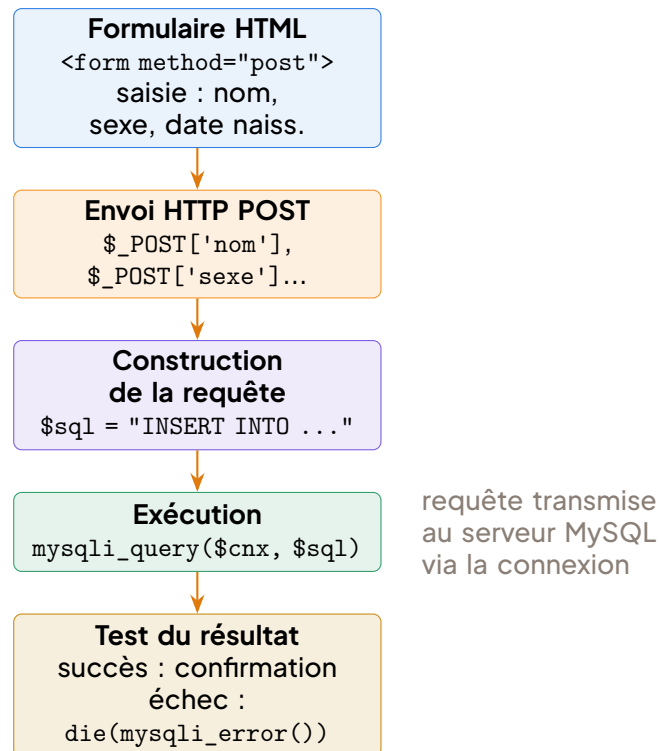
Contrairement aux trois requêtes précédentes, SELECT ne modifie rien : elle interroge la base et retourne un **jeu de résultats**, c'est-à-dire un tableau de lignes que le script parcourra ensuite (avec mysqli_fetch_assoc(), mysqli_num_rows(), etc. — ce sera l'objet de la section suivante).

```
$sql = "SELECT nom, classe FROM eleve WHERE classe = 'Tle TI';"  
$resultat = mysqli_query($cnx, $sql);  
if ($resultat === false) {  
    die("Erreur SQL : " . mysqli_error($cnx));  
}  
// $resultat est ici un jeu de résultats exploitable  
$nb = mysqli_num_rows($resultat);
```



```
| echo "$nb élève(s) trouvé(s).";
```

5.6 Le cycle complet : du formulaire au message de résultat



Cycle complet d'exécution d'une requête : du formulaire HTML au message de résultat.

5.7 Exemple guidé : script d'inscription d'un candidat

EXEMPLE Formulaire HTML d'inscription (fichier inscription.html)

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Inscription d'un candidat</title>
</head>
<body>
  <h2>Inscription au concours</h2>
  <form method="post" action="enregistrer.php">
    Nom : <input type="text" name="nom" required><br>
    Sexe :
    <select name="sexe">
      <option value="M">Masculin</option>
```

```
        <option value="F">Féminin</option>
    </select><br>
    Date de naissance : <input type="date" name="datnaiss" required><br>
    <input type="submit" value="Enregistrer">
</form>
</body>
</html>
```

EXEMPLE Script de traitement (fichier enregistrer.php)

```
<?php
// 1. Connexion au serveur MySQL
$cnx = mysqli_connect("localhost", "root", "")
    or die("Connexion impossible : " . mysqli_connect_error());

// 2. Sélection de la base
mysqli_select_db($cnx, "concours")
    or die("Base introuvable : " . mysqli_error($cnx));

// 3. Récupération des données du formulaire
$nom    = $_POST['nom'];
$sexe   = $_POST['sexe'];
$datnaiss = $_POST['datnaiss'];

// 4. Construction de la requête d'insertion
// (chaînes et dates entre apostrophes !)
$sql = "INSERT INTO candidat (nom, sexe, datnaiss)
    VALUES ('$nom', '$sexe', '$datnaiss')";

// 5. Envoi de la requête et test du résultat
if (mysqli_query($cnx, $sql)) {
    echo "l'enregistrement ajouté : bienvenue $nom !";
} else {
    die("Échec de l'insertion : " . mysqli_error($cnx));
}

// 6. Fermeture de la connexion
mysqli_close($cnx);
?>
```

Commentaires. Si l'utilisateur saisit Ngo Bell, M, 2006-04-12, la requête réellement envoyée à MySQL est :

```
INSERT INTO candidat (nom, sexe, datnaiss)
VALUES ('Ngo Bell', 'M', '2006-04-12');
```

La fonction retourne `true`, et le message « l'enregistrement ajouté : bienvenue Ngo Bell ! » s'affiche dans le navigateur.

EXERCICE RÉSOLU 5 Modification du numéro de téléphone d'un candidat

Énoncé. La table `candidat` possède les colonnes `id`, `nom`, `telephone`. Écrire le script PHP qui met à jour le téléphone du candidat d'`id` égal à 7 avec la valeur 699123456 reçue par `$_POST['tel']`, en affichant un message de succès ou l'erreur SQL.

SOLUTION

Solution détaillée. On suit la démarche en trois temps : construction, envoi, test.

```
<?php
$cnx = mysqli_connect("localhost", "root", "")
    or die("Connexion impossible");
mysqli_select_db($cnx, "concours")
    or die("Base introuvable");

$tel = $_POST['tel'];

// 1. Construction : WHERE obligatoire pour ne viser que l'id 7
$sql = "UPDATE candidat SET telephone = '$tel' WHERE id = 7";

// 2 et 3. Envoi et test
if (mysqli_query($cnx, $sql)) {
    echo "Téléphone mis à jour pour le candidat n° 7.";
} else {
    die("Erreur : " . mysqli_error($cnx));
}
mysqli_close($cnx);
?>
```

Sans la clause `WHERE id = 7`, tous les candidats auraient reçu le même numéro de téléphone : catastrophe garantie !

ATTENTION Injection SQL : le piège de la concaténation directe

Construire une requête en concaténant directement `$_POST` expose l'application aux **injections SQL** : un utilisateur malveillant peut saisir dans le champ `nom` la valeur `' OR '1'='1` et détourner la requête pour lire ou détruire des données. Les parades professionnelles sont :

- `mysqli_real_escape_string($cnx, $nom)` qui neutralise les caractères dangereux ;
- les **requêtes préparées** (`mysqli_prepare()`, `bind_param()`), qui séparent totalement le code SQL des données.

Ces techniques sont un complément utile à connaître, non exigible au programme de Terminale TI, mais indispensables dès que tu développeras une vraie application.

LESAVAIS-TU ? L'injection SQL, une faille bien réelle au Cameroun

L'injection SQL figure depuis plus de vingt ans dans le classement OWASP des failles web les plus critiques. Au Cameroun, l'ANTIC (Agence Nationale des Technologies de l'Information et de la Communication) sensibilise régulièrement les développeurs et administrations à ce risque : plusieurs sites institutionnels et plateformes locales ont été défigurés ou vidés de leurs données à cause de formulaires non protégés. Maîtriser `mysqli_query()` c'est bien ; savoir sécuriser ce qu'on lui envoie, c'est déjà penser comme un professionnel.

À RETENIR L'essentiel de la section

- `mysqli_query($cnx, $sql)` envoie une requête SQL au serveur : elle retourne `true/false` pour `INSERT`, `UPDATE`, `DELETE`, et un jeu de résultats (ou `false`) pour `SELECT`.
- La démarche type : construire la chaîne `$sql`, l'envoyer, tester le résultat avec `if` ou `die(mysqli_error($cnx))`.
- Les valeurs textuelles et les dates s'écrivent entre apostrophes dans la requête ; les nombres sans apostrophes.
- `UPDATE` et `DELETE` exigent une clause `WHERE` ciblant la clé primaire, sous peine de modifier ou supprimer toute la table.
- Les données d'un formulaire arrivent dans `$_POST` ; leur concaténation directe expose aux injections SQL (parade : `mysqli_real_escape_string()` ou requêtes préparées).

EXERCICE 4 Suppression contrôlée

Écrire un script `supprimer.php` qui reçoit `$_POST['id']`, supprime le candidat correspondant dans la table `candidat`, affiche « Candidat supprimé » en cas de succès, et l'erreur SQL sinon. Expliquer pourquoi il serait dangereux d'écrire `DELETE FROM candidat` sans `WHERE`.

6 LECTURE ET AFFICHAGE DES RÉSULTATS

Dans la section précédente, tu as appris à exécuter une requête SQL depuis PHP grâce à `mysqli_query()`. Tu as constaté une dissymétrie fondamentale : pour une requête `INSERT`, `UPDATE` ou `DELETE`, la fonction retourne simplement `true` ou `false`, car il n'y a rien à « lire ». Mais pour une requête `SELECT`, `mysqli_query()` retourne un objet particulier, appelé **jeu de résultats** (ou `result set`), qui contient toutes les lignes sélectionnées. Ce n'est **pas** un tableau PHP ordinaire : on ne peut pas écrire directement `$res[0]['nom']`. Il faut utiliser des fonctions spécialisées de la famille

`mysqli_fetch_*` pour extraire les lignes une à une. C'est précisément l'objet de cette section : transformer un jeu de résultats MySQL en données affichables dans une page web.

6.1 Nature du jeu de résultats : un tableau de lignes parcouru par un pointeur

DÉFINITION Jeu de résultats

Un **jeu de résultats** est la structure de données retournée par `mysqli_query()` après l'exécution d'une requête `SELECT`. Il se présente comme un tableau de lignes : chaque ligne correspond à un enregistrement de la table, et chaque ligne est elle-même composée de colonnes (les champs sélectionnés). Le jeu de résultats possède un **pointeur de ligne** interne, positionné initialement avant la première ligne.

L'image mentale à retenir est celle d'un **guichet de la CAMTEL** qui distribue des tickets : le jeu de résultats est la file d'attente des clients, et le pointeur est l'employé qui appelle les clients un par un. Chaque appel à une fonction `mysqli_fetch_*` « sert » le client suivant et fait avancer la file. Quand il n'y a plus personne, l'employé annonce « suivant : personne » — en PHP, c'est la valeur `NULL`.

Jeu de résultats : `SELECT nom, age FROM eleve`

nom	age
Amina	17
Biya	18
Cédric	17
Diane	19
Emma	18

NULL (fin du résultat)

← chaque appel de
`mysqli_fetch_assoc()`
avance d'une ligne

Le pointeur de ligne avance à chaque appel de
`mysqli_fetch_assoc()`, jusqu'à atteindre `NULL` en fin de résultat.

6.2 Compter les lignes : `mysqli_num_rows()`

Avant même de parcourir les résultats, il est souvent utile de savoir combien de lignes la requête a retournées : pour afficher « 5 élève(s) trouvé(s) », pour vérifier qu'une recherche n'est pas vide, ou pour décider d'afficher un message « aucun résultat ».

PROPRIÉTÉ Syntaxe de `mysqli_num_rows()`

`int mysqli_num_rows(mysqli_result $resultat)` retourne le nombre de lignes contenues

dans le jeu de résultats passé en paramètre. Elle s'applique uniquement aux résultats d'une requête `SELECT` (ou `SHOW`). Elle ne modifie pas le pointeur : on peut l'appeler avant ou après le parcours.

```
<?php
$cnx = mysqli_connect("localhost", "root", "")
    or die("Connexion impossible : " . mysqli_connect_error());
mysqli_select_db($cnx, "lycee") or die("Base introuvable");

$res = mysqli_query($cnx, "SELECT nom, age FROM eleve");
$nb = mysqli_num_rows($res);
echo $nb . " eleve(s) trouve(s).";
?>
```

ATTENTION `mysqli_num_rows()` ne fonctionne pas avec `INSERT/UPDATE/DELETE`

Pour une requête de modification, `mysqli_query()` retourne un booléen et non un jeu de résultats : appeler `mysqli_num_rows(true)` provoque une erreur. Pour connaître le nombre de lignes affectées par un `UPDATE` ou un `DELETE`, on utilise `mysqli_affected_rows($cnx)` — hors programme, mais utile en projet.

6.3 Extraire une ligne : les trois fonctions `mysqli_fetch_*`

PHP propose trois fonctions d'extraction, qui diffèrent uniquement par la **forme du tableau** retourné. Les trois partagent le même comportement : elles retournent la ligne courante, avancent le pointeur d'une ligne, et retournent `NULL` quand le résultat est épuisé.

1. `mysqli_fetch_row()` : tableau à indices numériques.

Chaque ligne est retournée sous la forme d'un tableau indexé par des entiers, dans l'ordre des champs du `SELECT` : `$ligne[0]` est le premier champ, `$ligne[1]` le deuxième, etc.

2. `mysqli_fetch_assoc()` : tableau associatif.

Chaque ligne est retournée sous la forme d'un tableau associatif dont les **clés sont les noms des champs** : `$ligne['nom']`, `$ligne['age']`. C'est la forme la plus lisible et la plus robuste.

DÉFINITION `mysqli_fetch_assoc()`

`array|null mysqli_fetch_assoc(mysqli_result $resultat)` retourne la ligne courante du jeu de résultats sous forme de **tableau associatif** (les clés sont les noms des champs de la requête), puis avance le pointeur d'une ligne. Elle retourne `NULL` lorsque toutes les lignes ont été parcourues.

3. `mysqli_fetch_array()` : les deux formes à la fois.

Chaque ligne est retournée dans un tableau qui contient à la fois les indices numériques et les clés associatives : on peut écrire indifféremment `$ligne[0]` ou `$ligne['nom']`. Pratique, mais le tableau est deux fois plus volumineux en mémoire.

Fonction	Accès aux champs	Usage typique
<code>mysqli_fetch_row()</code>	<code>\$ligne[0]</code> , <code>\$ligne[1]</code> ...	Rapide, mais fragile si l'ordre des champs change
<code>mysqli_fetch_assoc()</code>	<code>\$ligne['nom']</code> , <code>\$ligne['age']</code>	Recommandé : lisible et explicite
<code>mysqli_fetch_array()</code>	les deux formes	Polyvalent, mais redondant

ATTENTION Erreur classique : mélanger les indices

Avec `mysqli_fetch_row()`, les indices sont numériques : écrire `$ligne['nom']` produit l'erreur `Undefined array key "nom"` et affiche une chaîne vide. Réciproquement, avec `mysqli_fetch_assoc()`, `$ligne[0]` n'existe pas. Choisis une fonction et reste cohérent avec ses indices. Autre piège : les clés de `fetch_assoc` sont sensibles à la casse — `$ligne['Nom']` ne vaut pas `$ligne['nom']`.

LESAVAIS-TU ? `fetch_array` avec un mode

`mysqli_fetch_array()` accepte un deuxième paramètre optionnel : `mysqli_fetch_array($res, MYSQLI_ASSOC)` est strictement équivalent à `mysqli_fetch_assoc($res)`, et `MYSQLI_NUM` équivaut à `fetch_row`. Au Baccalauréat, `mysqli_fetch_assoc()` reste la forme la plus lisible et la plus attendue par les correcteurs : privilégie-la systématiquement.

6.4 Le schéma canonique de parcours : la boucle `while`

Comment afficher toutes les lignes alors qu'on ne sait pas combien la requête en retourne ? La réponse tient en une propriété remarquable des fonctions `fetch`.

PROPRIÉTÉ Arrêt naturel de la boucle — admise et illustrée

Chaque appel à une fonction `mysqli_fetch_*` avance le pointeur d'une ligne et retourne la ligne lue. Lorsque le pointeur a dépassé la dernière ligne, la fonction retourne `NULL`, valeur évaluée comme faux dans un test. Par conséquent, la boucle :

```
while ($ligne = mysqli_fetch_assoc($res)) { ... }
```

s'exécute exactement une fois par ligne et s'arrête naturellement en fin de résultat, sans test supplémentaire.

Comprends bien la subtilité : dans la condition du `while`, il y a une **affectation** (`=`) et non une comparaison (`==`). À chaque itération, PHP appelle `mysqli_fetch_assoc()`, range la ligne dans `$ligne`, puis évalue cette valeur : un tableau non vide vaut vrai, NULL vaut faux. C'est l'idiome le plus célèbre du PHP/MySQL.

MÉTHODE Schéma canonique d'affichage d'un SELECT

Pour afficher les résultats d'une requête de sélection, suis toujours les mêmes étapes :

1. Exécuter la requête : `$res = mysqli_query($cnx, $sql);` et vérifier que `$res` n'est pas `false`.
2. (Optionnel) Compter : `echo mysqli_num_rows($res) . ' résultat(s)';`
3. Parcourir : `while ($l = mysqli_fetch_assoc($res)) { echo $l['champ']; }`
4. Libérer et fermer : `mysqli_close($cnx);` en fin de script.

Ce schéma en quatre temps fonctionne pour toute requête `SELECT`, quelle que soit la table.

6.5 Affichage dans un tableau HTML généré dynamiquement

Le débouché naturel d'un jeu de résultats est un tableau HTML : les lignes de la base deviennent des lignes `<tr>`, les champs deviennent des cellules `<td>`. La technique consiste à **ouvrir** le tableau et à écrire sa ligne d'en-tête avant la boucle, à générer une ligne `<tr>` par itération, puis à **fermer** le tableau après la boucle.

EXEMPLE Liste des élèves de Terminale TI

Le script complet ci-dessous interroge la table `eleve` et affiche les cinq élèves dans un tableau HTML bordé, précédés du nombre de résultats.

```
<?php
// 1. Connexion au serveur MySQL puis choix de la base
$cnx = mysqli_connect("localhost", "root", "")
    or die("Connexion impossible : " . mysqli_connect_error());
mysqli_select_db($cnx, "lycee") or die("Base introuvable");

// 2. Execution de la requete de selection
$sql = "SELECT nom, age, moyenne FROM eleve ORDER BY nom";
$res = mysqli_query($cnx, $sql)
    or die("Erreur SQL : " . mysqli_error($cnx));

// 3. Nombre de lignes retournees
echo "<p>" . mysqli_num_rows($res) . " eleve(s) trouve(s).</p>";

// 4. Ouverture du tableau HTML et ligne d'en-tete
echo "<table border='1' cellspacing='0' cellpadding='5'>";
echo "<tr><th>Nom</th><th>Age</th><th>Moyenne</th></tr>";
```



```
// 5. Parcours canonique : une ligne HTML par enregistrement
while ($ligne = mysqli_fetch_assoc($res)) {
    echo "<tr>";
    echo "<td>". $ligne['nom'] . "</td>";
    echo "<td>". $ligne['age'] . "</td>";
    echo "<td>". $ligne['moyenne'] . "</td>";
    echo "</tr>";
}

// 6. Fermeture du tableau et de la connexion
echo "</table>";
mysqli_close($cnx);
?>
```

La page produite dans le navigateur ressemble à la maquette suivante :

http://localhost/lycee/liste_eleves.php

5 élève(s) trouvé(s).

Nom	Age	Moyenne
Amina	17	14,5
Biya	18	12,0
Cédric	17	15,5
Diane	19	11,5
Emma	18	16,0

Maquette du tableau HTML généré dynamiquement par la boucle
while : une ligne <tr> par enregistrement de la table eleve.

Observe la correspondance exacte : la ligne d'en-tête <th> est écrite **une seule fois** (hors boucle), chaque passage dans le while produit **exactement une ligne <tr>**, et le nombre de lignes du tableau HTML est égal à `mysqli_num_rows($res)`.

ATTENTION**Ouvrir et fermer le tableau hors de la boucle**

Une erreur très fréquente consiste à placer `echo "<table>";` à l'intérieur du while : le navigateur reçoit alors cinq tableaux d'une ligne au lieu d'un tableau de cinq lignes. Règle d'or : tout ce qui est répété va dans la boucle (les <tr>), tout ce qui est unique reste dehors (<table>, <th>, </table>).

EXERCICE RÉSOLU 6**Bulletins de la classe**

Énoncé. La table `eleve` de la base `lycee` contient les champs `nom`, `age` et `moyenne`. Écris un script PHP qui affiche, pour chaque élève, une ligne « Amina (17 ans) — moyenne : 14,5 », puis indique le nombre total d'élèves. Si la table est vide, le script doit afficher « Aucun élève enregistré ».

SOLUTION

Solution détaillée. On applique le schéma canonique, en ajoutant un test sur `mysqli_num_rows()` pour traiter le cas vide :

```
<?php
$cnx = mysqli_connect("localhost", "root", "")
    or die("Connexion impossible");
mysqli_select_db($cnx, "lycee") or die("Base introuvable");

$res = mysqli_query($cnx, "SELECT nom, age, moyenne FROM eleve")
    or die("Erreur SQL : " . mysqli_error($cnx));

if (mysqli_num_rows($res) == 0) {
    echo "Aucun eleve enregistré.";
} else {
    while ($l = mysqli_fetch_assoc($res)) {
        echo $l['nom'] . " (" . $l['age'] . " ans) – moyenne : "
            . $l['moyenne'] . "<br>";
    }
    echo "<p>Total : " . mysqli_num_rows($res) . " eleve(s).</p>";
}
mysqli_close($cnx);
?>
```

Points de vigilance : le test `== 0` se fait avant la boucle ; `mysqli_num_rows()` peut être rappelé après le parcours car il ne dépend pas du pointeur ; les clés `'nom'`, `'age'`, `'moyenne'` correspondent exactement aux champs du `SELECT`.

EXPÉRIENCE TP guidé : afficher les clients d'un cybercafé

Objectif. Produire une page web listant les clients enregistrés dans la base d'un cybercafé de Yaoundé. Logiciels. WampServer (ou XAMPP), un navigateur, un éditeur de texte.

1. Lance WampServer, ouvre phpMyAdmin et crée la base `cybercafe` avec la table `client` (`id` INT AUTO_INCREMENT PRIMARY KEY, `nom` VARCHAR(50), `forfait` VARCHAR(20), `solde` INT).
2. Insère au moins quatre clients (ex. « Ngo Bell », forfait « Journée », solde 500 FCFA).
3. Crée le fichier `clients.php` dans `www` et écris le script de connexion, la requête `SELECT * FROM client`, l'affichage du nombre de clients, puis le tableau HTML bordé avec les colonnes Nom, Forfait, Solde.
4. Ouvre `http://localhost/clients.php` et vérifie que le nombre affiché correspond aux

lignes du tableau.

5. Manipulation. Ajoute un client via phpMyAdmin, recharge la page : le tableau s'allonge sans modifier le code. C'est toute la puissance de la génération dynamique.
6. Interprétation. Supprime tous les clients : que se passe-t-il? Ajoute le test `mysqli_num_rows() == 0` pour afficher un message propre.

EXERCICE 5 Recherche par filière

La table `eleve` possède un champ `filiere`. Écris un script qui n'affiche que les élèves de la filière « TI » dans un tableau HTML, avec leur nombre. Que se passe-t-il si aucun élève n'est en TI? Propose une solution élégante.

CORRIGÉ Exercice : recherche par filière

On ajoute une clause `WHERE` à la requête, puis on teste le nombre de lignes avant d'ouvrir le tableau :

```
<?php
$cnx = mysqli_connect("localhost", "root", "") or die("Connexion impossible");
mysqli_select_db($cnx, "lycee") or die("Base introuvable");

$res = mysqli_query($cnx, "SELECT nom, age FROM eleve WHERE filiere='TI'")
      or die("Erreur SQL : " . mysqli_error($cnx));

$nb = mysqli_num_rows($res);
if ($nb == 0) {
    echo "Aucun eleve en filiere TI.";
} else {
    echo "<p>$nb eleve(s) en TI.</p>";
    echo "<table border='1'><tr><th>Nom</th><th>Age</th></tr>";
    while ($l = mysqli_fetch_assoc($res)) {
        echo "<tr><td>" . $l['nom'] . "</td><td>" . $l['age'] . "</td></tr>";
    }
    echo "</table>";
}
mysqli_close($cnx);
?>
```

Sans le test, un résultat vide produirait un tableau HTML réduit à sa seule ligne d'en-tête — d'où l'intérêt du message alternatif.

À RETENIR L'essentiel de la section

- Un `SELECT` retourne un **jeu de résultats** : un tableau de lignes parcouru par un pointeur interne.
- `mysqli_num_rows($res)` donne le nombre de lignes ; `mysqli_fetch_assoc($res)` retourne la ligne courante en tableau associatif et avance le pointeur ; `NULL` signale la fin.
- Le schéma canonique `while ($l = mysqli_fetch_assoc($res)) { ... }` parcourt toutes les lignes sans connaître leur nombre à l'avance.
- Pour un tableau HTML : ouverture et en-tête avant la boucle, un `<tr>` par itération, fermeture après.

Tu maîtrises désormais la chaîne complète « requête → résultat → affichage ». Il ne reste qu'à assembler toutes les briques dans la synthèse : une mini-application web PHP/MySQL complète.

7 SYNTHÈSE : MINI-APPLICATION WEB PHP/MYSQL

Dans les sections précédentes, tu as appris à structurer tes scripts avec les instructions conditionnelles et les boucles, à manipuler la date avec `date()`, à te connecter à MySQL via l'extension `mysqli` et à exécuter des requêtes `SELECT` en affichant les résultats avec `mysqli_fetch_assoc()`. Tu possèdes désormais toutes les briques élémentaires. Il est temps de les assembler pour construire une véritable **application web** complète, structurée et maintenable : la gestion des candidats d'un concours (par exemple l'entrée dans une école d'informatique à Yaoundé). Cette synthèse te montre comment un développeur professionnel organise son projet en plusieurs fichiers spécialisés qui collaborent.

7.1 Le cahier des charges : une application CRUD

DÉFINITION Application CRUD

Une **application CRUD** est une application qui permet les quatre opérations fondamentales sur les données d'une base :

- Create : créer un enregistrement (requête `INSERT`) ;
- Read : lire et afficher les enregistrements (requête `SELECT`) ;
- Update : modifier un enregistrement existant (requête `UPDATE`) ;
- Delete : supprimer un enregistrement (requête `DELETE`).

Notre mini-application « Gestion des candidats » devra permettre à un secrétaire de :

1. **ajouter** un candidat (nom, prénom, série, moyenne) via un formulaire ;

2. **lister** tous les candidats dans un tableau HTML ;
3. **modifier** les informations d'un candidat via un formulaire pré-rempli ;
4. **supprimer** un candidat après confirmation.

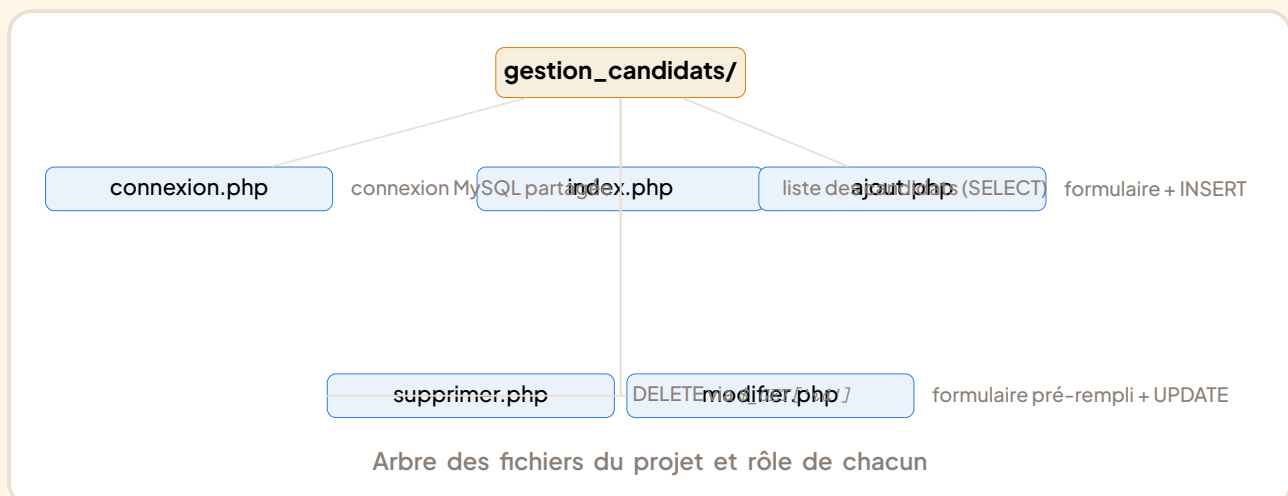
La base de données s'appellera `concours` et contiendra une seule table `candidats` :

```
CREATE DATABASE IF NOT EXISTS concours;
USE concours;

CREATE TABLE candidats (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(50) NOT NULL,
  prenom VARCHAR(50) NOT NULL,
  serie VARCHAR(10) NOT NULL,
  moyenne DECIMAL(4,2) NOT NULL,
  date_inscription DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

7.2 Architecture en fichiers du projet

Plutôt qu'un unique script monolithique, on découpe l'application en petits fichiers spécialisés, chacun ayant un rôle unique et bien défini. C'est plus lisible, plus facile à déboguer et à faire évoluer.



Le point central de cette architecture est `connexion.php` : il est inclus au début de chaque page grâce à `require`, ce qui évite de dupliquer le code de connexion et garantit une configuration unique.

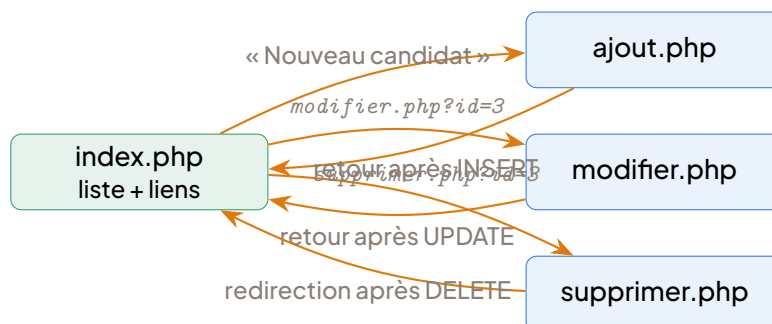
```
<?php
// connexion.php : connexion partagée à la base "concours"
$conn = mysqli_connect("localhost", "root", "")
  or die("Echec de connexion : " . mysqli_connect_error());

mysqli_select_db($conn, "concours")
  or die("Base introuvable : " . mysqli_error($conn));

mysqli_set_charset($conn, "utf8"); // gestion correcte des accents
?>
```

7.3 Navigation entre les pages et passage de l'identifiant

Comment la page `supprimer.php` sait-elle quel candidat supprimer ? Grâce à un paramètre passé dans l'URL : `supprimer.php?id=3`. Le lien est généré dynamiquement dans la liste (`index.php`), et la page cible récupère la valeur dans le tableau superglobal `$_GET`.



Navigation entre les pages de l'application

ATTENTION Toujours tester l'existence de `$_GET['id']` et `$_POST`

Si un visiteur tape directement `supprimer.php` sans paramètre, `$_GET['id']` n'existe pas : PHP émet un avertissement (Warning) et la requête `DELETE` devient dangereuse ou invalide. Règle d'or : avant d'utiliser un paramètre d'URL ou de formulaire, tester son existence avec `if (isset($_GET['id']))` ou `if (isset($_POST['nom']))`. C'est la base de la robustesse et de la sécurité.

7.4 La page `index.php` : lister avec `while` et générer les liens

EXEMPLE `index.php` commenté : liste des candidats

```

<?php
require "connexion.php"; // inclusion de la connexion partagée

// 1. Lecture de tous les candidats (tri alphabétique)
$resultat = mysqli_query($conn,
    "SELECT * FROM candidats ORDER BY nom")
    or die("Erreur SELECT : " . mysqli_error($conn));
?>
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>Candidats</title></head>
<body>
<h1>Liste des candidats</h1>
<p><a href="ajout.php">+ Nouveau candidat</a></p>
<table border="1" cellpadding="6">
<tr><th>Nom</th><th>Prenom</th><th>Serie</th>

```

```

        <th>Moyenne</th><th>Actions</th></tr>
<?php
// 2. while parcourt les lignes une par une
while ($ligne = mysqli_fetch_assoc($resultat)){
    echo "<tr>";
    echo "<td>". $ligne["nom"]. "</td>";
    echo "<td>". $ligne["prenom"]. "</td>";
    echo "<td>". $ligne["serie"]. "</td>";
    echo "<td>". $ligne["moyenne"]. "</td>";
    // 3. L'identifiant est passe dans l'URL de chaque lien d'action
    echo "<td>";
        <a href='modifier.php?id=' . $ligne["id"] . "'>Modifier</a> |
        <a href='supprimer.php?id=' . $ligne["id"] . "'>Supprimer</a>
    </td>";
    echo "</tr>";
}
?>
</table>
</body>
</html>

```

Observe la ligne des liens : pour le candidat d'id égal à 3, PHP génère le HTML `<a href='supprimer.php?id=3'>`. C'est le mécanisme clé qui lie l'interface (HTML) à la logique métier (PHP/SQL).

7.5 Ajout, suppression et modification

ajout.php regroupe l'affichage du formulaire (méthode GET) et son traitement (méthode POST) dans un même fichier. On teste `$_SERVER["REQUEST_METHOD"]` pour distinguer les deux phases.

```

<?php
require "connexion.php";

// Traitement du formulaire (méthode POST)
if ($_SERVER["REQUEST_METHOD"] === "POST"){
    // Récupération et nettoyage basique
    $nom = trim($_POST["nom"]);
    $prenom = trim($_POST["prenom"]);
    $serie = trim($_POST["serie"]);
    $moyenne = (float) $_POST["moyenne"]; // cast numérique

    // Requête paramétrée idéale en pro ; ici concaténation pour niveau Terminale
    $sql = "INSERT INTO candidats (nom, prenom, serie, moyenne)
    VALUES ('$nom', '$prenom', '$serie', $moyenne)";
    mysqli_query($conn, $sql)
    or die("Erreur INSERT : " . mysqli_error($conn));

    header("Location : index.php"); // PRG pattern : Post-Redirect-Get
    exit;
}
?>
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>Ajout candidat</title></head>
<body>
<h1>Nouveau candidat</h1>

```

```

<form method="post" action="ajout.php">
  Nom : <input type="text" name="nom" required><br><br>
  Prenom : <input type="text" name="prenom" required><br><br>
  Serie : <input type="text" name="serie" required><br><br>
  Moyenne : <input type="number" step="0.01" name="moyenne" required><br><br>
  <input type="submit" value="Enregistrer">
  <a href="index.php">Annuler</a>
</form>
</body>
</html>

```

supprimer.php est concis : il valide la présence de l'identifiant, le caste en entier (sécurité), exécute le DELETE et redirige vers la liste.

```

<?php
require "connexion.php";

// Sécurité : l'id existe-t-il et est-il numérique ?
if (isset($_GET["id"])){
    $id = (int) $_GET["id"]; // conversion en entier (neutralise injections)
    $sql = "DELETE FROM candidats WHERE id = $id";
    mysqli_query($conn, $sql)
    or die("Erreur DELETE : " . mysqli_error($conn));
}
// Redirection systématique (même si id absent)
header("Location : index.php");
exit;
?>

```

modifier.php combine les deux techniques : d'abord un SELECT ... WHERE id = ... pour pré-remplir le formulaire avec les valeurs actuelles (arrivée par GET), puis un UPDATE quand le formulaire est soumis (envoi par POST).

```

<?php
require "connexion.php";

// --- CAS 1 : Soumission du formulaire de modification (POST) ---
if ($_SERVER["REQUEST_METHOD"] === "POST" && isset($_POST["id"])) {
    $id = (int) $_POST["id"];
    $nom = trim($_POST["nom"]);
    $prenom = trim($_POST["prenom"]);
    $serie = trim($_POST["serie"]);
    $moyenne = (float) $_POST["moyenne"];

    $sql = "UPDATE candidats
        SET nom='$nom', prenom='$prenom',
            serie='$serie', moyenne=$moyenne
        WHERE id=$id";
    mysqli_query($conn, $sql)
    or die("Erreur UPDATE : " . mysqli_error($conn));

    header("Location : index.php");
    exit;
}

// --- CAS 2 : Arrivée sur la page via lien (GET) : pré-remplissage ---
if (isset($_GET["id"])){
    $id = (int) $_GET["id"];
    $res = mysqli_query($conn,

```



```

        "SELECT * FROM candidats WHERE id = $id")
    or die("Erreur SELECT : " . mysqli_error($conn));
    $c = mysqli_fetch_assoc($res);

    // Vérification cruciale : le candidat existe-t-il ?
    if (!$c) {
        die("Candidat introuvable (id=$id).");
    }
} else {
    die("Aucun candidat précisé.");
}
?>
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>Modifier candidat</title></head>
<body>
<h1>Modifier le candidat #<?php echo $c["id"]; ?></h1>
<form method="post" action="modifier.php">
    <!-- Champ caché : l'id voyage avec le formulaire sans être visible -->
    <input type="hidden" name="id" value="<?php echo $c["id"]; ?>">

    Nom : <input type="text" name="nom"
        value="<?php echo htmlspecialchars($c["nom"]); ?>" required><br><br>
    Prenom : <input type="text" name="prenom"
        value="<?php echo htmlspecialchars($c["prenom"]); ?>" required><br><br>
    Serie : <input type="text" name="serie"
        value="<?php echo htmlspecialchars($c["serie"]); ?>" required><br><br>
    Moyenne : <input type="number" step="0.01" name="moyenne"
        value="<?php echo $c["moyenne"]; ?>" required><br><br>
    <input type="submit" value="Mettre à jour">
    <a href="index.php">Annuler</a>
</form>
</body>
</html>

```

Remarque le champ caché `<input type="hidden" name="id" ...>` : il permet de transmettre l'identifiant au moment de la soumission POST, sans que l'utilisateur puisse le modifier facilement. C'est un classique du développement web. Note aussi l'usage de `htmlspecialchars()` dans les attributs `value` : cela protège l'affichage si les données contiennent des guillemets ou des caractères spéciaux.

ATTENTION Faible d'injection SQL — Rappel vital

Concaténer directement `$_POST["nom"]` dans une requête, comme ci-dessus pour rester lisible au niveau Terminale, expose l'application aux **injections SQL** : un utilisateur malveillant peut taper `' OR '1'='1` dans le formulaire et modifier/supprimer toute la table. En production, on utilise les requêtes préparées (`mysqli_prepare`, `bind_param`, `execute`) qui séparent le code SQL des données. Retiens l'idée : jamais confiance aveugle aux données venant de l'utilisateur.

7.6 Horodatage avec `date()` et stratégie de débogage

La fonction `date()` vue précédemment trouve ici une application naturelle : afficher en pied de

page l'heure de génération de la liste, ou tracer les opérations dans un journal (fichier log).

```
// Dans index.php, avant </body> par exemple
echo "<p style='text-align:center;color:gray;margin-top:20px;'>";
echo "Liste générée le " . date("d/m/Y") . " à " . date("H:i:s");
echo "</p>";
// Affiche : Liste générée le 14/03/2025 à 09:42:17
```

Pour les tests et le débogage, la démarche professionnelle est double :

- **Côté PHP** : chaque `mysqli_query()` est suivi de `or die(mysqli_error($conn))` pour afficher le message d'erreur exact de MySQL (syntaxe, contrainte, droit) au lieu d'une page blanche muette.
- **Côté phpMyAdmin** : après chaque ajout, modification ou suppression, on ouvre la table `candidats` dans phpMyAdmin (onglet Parcourir) pour vérifier visuellement que l'opération a bien eu lieu et que les données sont cohérentes.

MÉTHODE**Démarche de construction d'une application CRUD**

1. Créer la base et la table dans phpMyAdmin (ou par script SQL exécuté une fois).
2. Écrire `connexion.php` et le tester seul (page blanche = OK, message d'erreur = correction).
3. Écrire la liste : `SELECT` + boucle `while` dans `index.php`. Tester avec les données de test.
4. Ajouter l'insertion : formulaire + `INSERT` dans `ajout.php`. Tester l'ajout et vérifier dans phpMyAdmin.
5. Ajouter la modification (`SELECT` de pré-remplissage + `UPDATE`) dans `modifier.php`. Tester.
6. Ajouter la suppression (`DELETE` via `$_GET`) dans `supprimer.php`. Tester.
7. À chaque étape : tester immédiatement dans le navigateur ET dans phpMyAdmin avant de passer à la suivante.

EXPÉRIENCE**TP guidé : déployer « Gestion des candidats » sous WampServer**

Objectif : faire fonctionner l'application complète en local sur ta machine.

Matériel/logiciels : un poste Windows avec WampServer (ou XAMPP) installé, un navigateur web, phpMyAdmin (inclus).

Manipulation :

1. Démarre WampServer (icône verte dans la barre des tâches). Ouvre `http://localhost/phpmyadmin` et exécute le script SQL de création de la base `concours` et de la table `candidats` (onglet SQL).

2. Insère manuellement deux candidats de test dans phpMyAdmin (onglet Insérer de la table `candidats`).
3. Crée le dossier `C:\wamp64\www\gestion_candidats` et places-y les cinq fichiers PHP vus dans cette section (`connexion.php`, `index.php`, `ajout.php`, `modifier.php`, `supprimer.php`).
4. Ouvre `http://localhost/gestion_candidats/index.php` : la liste des deux candidats doit s'afficher avec les liens d'action.
5. Clique sur « Nouveau candidat », remplis le formulaire, valide. Vérifie dans phpMyAdmin que la ligne existe (onglet Parcourir).
6. Clique sur « Modifier » sur une ligne : le formulaire doit être pré-rempli. Change la moyenne, valide, vérifie la mise à jour dans phpMyAdmin.
7. Clique sur « Supprimer » sur une ligne, confirme si le navigateur le demande (lien direct ici), puis vérifie que la ligne a disparu dans phpMyAdmin.

Observations attendues : chaque action depuis le navigateur se répercute instantanément dans la base de données ; la liste `index.php` est régénérée à chaque affichage (requête `SELECT` fraîche), donc toujours à jour.

Interprétation : le navigateur (client) ne communique jamais directement avec MySQL ; c'est toujours PHP, exécuté sur le serveur web (Apache), qui joue l'intermédiaire — c'est l'architecture client-serveur à trois niveaux (Navigateur / Serveur PHP / SGBD MySQL).

EXERCICE RÉSOLU 7 Tracer une suppression complète

Énoncé : la table `candidats` contient une ligne d'id égal à 3 (nom : « Etoundi »). Dans `index.php`, décris le code HTML généré pour le lien « Supprimer » de cette ligne, puis explique pas à pas ce qui se passe quand l'utilisateur clique dessus.

SOLUTION

Solution : la boucle `while` de `index.php` génère pour cette ligne le HTML suivant :

```
<a href='supprimer.php?id=3'>Supprimer</a>
```

Au clic, le navigateur envoie une requête HTTP GET vers l'URL `supprimer.php?id=3`. Côté serveur, l'exécution suit ce scénario :

1. `require "connexion.php"` ouvre la connexion à MySQL (hôte `localhost`, utilisateur `root`, base `concours`);
2. La condition `isset($_GET["id"])` est vraie, donc `$id = (int) "3"` vaut l'entier 3;
3. PHP construit la chaîne de requête `DELETE FROM candidats WHERE id = 3` et l'envoie à MySQL via `mysqli_query()`;

4. MySQL localise la ligne d'identifiant 3 (clé primaire) et la supprime physiquement ;
5. L'instruction `header("Location: index.php")` envoie une redirection HTTP 302 au navigateur ;
6. Le navigateur recharge `index.php`, qui ré-exécute le `SELECT` : le candidat « Etoundi » n'apparaît plus.

LESAVAIS-TU ? Des petits fichiers aux grands frameworks

Cette architecture en petits fichiers spécialisés (`index`, `ajout`, `modifier`, `supprimer` partageant `connexion.php`) est l'ancêtre direct des **frameworks** modernes comme Laravel, Symfony ou CodeIgniter, qui organisent le code selon le motif architectural MVC (Modèle-Vue-Contrôleur). Les développeurs camerounais qui conçoivent des plateformes de paiement mobile (API MTN MoMo, Orange Money), des applications de gestion pour les PME de Douala ou des portails administratifs (`impots.cm`, `ANTIC`) utilisent ces frameworks quotidiennement — mais la logique CRUD que tu viens de construire à la main reste exactement la même sous le capot. Maîtriser ce « PHP natif », c'est comprendre ce que le framework automatise pour toi.

À RETENIR L'essentiel de la synthèse

- Une application CRUD couvre les quatre opérations de base : `INSERT`, `SELECT`, `UPDATE`, `DELETE`.
- On découpe le projet en fichiers spécialisés partageant un `connexion.php` inclus par `require` (factorisation, maintenance).
- L'identifiant d'un enregistrement voyage par l'URL (`?id=3`, lu dans `$_GET`) pour les liens (suppression, édition) ou par un champ caché `<input type="hidden">` (lu dans `$_POST`) pour les formulaires de modification.
- Le formulaire de modification est pré-rempli grâce à un `SELECT ... WHERE id = ...` exécuté à l'arrivée sur la page (GET).
- Toujours tester `isset($_GET['id'])` / `isset($_POST['...'])` avant usage, caster les identifiants en `(int)`, et vérifier chaque opération dans `phpMyAdmin`.
- `htmlspecialchars()` dans les `value` des `inputs` protège l'affichage (XSS basique) ; les requêtes préparées protègent la base (SQLi).

EXERCICE 6 À toi de jouer

Adapte l'application pour gérer la bibliothèque de ton lycée : table `livres(id, titre, auteur, annee, disponible)` où `disponible` est un `TINYINT(1)` (0 ou 1).

1. Écris le script SQL de création de la table.
2. Écris les pages `index.php` (liste) et `supprimer.php`.
3. Dans `index.php`, ajoute dans le tableau une colonne « Disponible » qui affiche « Oui » si `disponible=1` et « Non » sinon, grâce à une structure `if/else` à l'intérieur de la boucle `while`.

8 ACTIVITÉ D'INTÉGRATION

8.1 Objectifs de l'activité

À l'issue de cette activité d'intégration, tu seras capable de produire, en groupe, une **application web complète** interagissant avec une base de données **MySQL**. Cette activité mobilise l'ensemble des savoir-faire de la leçon :

- organiser un script PHP à l'aide des **structures de contrôle** (if, elseif, switch, while);
- horodater des données avec la fonction `date()` ;
- établir une **connexion sécurisée** à MySQL avec l'extension `mysqli` (`mysqli_connect`, `mysqli_select_db`, `die`);
- écrire et exécuter depuis PHP les quatre requêtes de manipulation des données : INSERT, SELECT, UPDATE, DELETE ;
- lire et afficher proprement les résultats avec `mysqli_fetch_assoc` et `mysqli_num_rows`.

8.2 Situation

La cybercafétéria « Espace Jeunes » de Nkolndongo, à Yaoundé, accueille chaque année des dizaines de lycéens venus s'inscrire au concours interne d'entrée en classe de Terminale TI du Lycée Bilingue voisin. Jusqu'ici, le gérant, M. Etoundi, enregistre les candidats dans un cahier : nom, sexe, date de naissance, épreuve choisie. Résultat : ratures, doublons, cahier égaré en pleine période d'inscription, et impossible de connaître rapidement le nombre d'inscrits par épreuve.

M. Etoundi fait appel à ta classe de Terminale TI pour numériser ce processus. Il formule les exigences suivantes :

1. Les inscriptions doivent être enregistrées dans une base MySQL nommée `concours`, contenant une table `candidat` de structure :

Champ	Type	Rôle
id	INT AUTO_INCREMENT PRIMARY KEY	Identifiant unique du candidat
nom	VARCHAR(60)	Nom et prénom
sexe	VARCHAR(1)	M ou F
date_naissance	DATE	Date de naissance (format AAAA-MM-JJ)
choix_epreuve	VARCHAR(20)	Algorithmique, Reseaux ou BDD
date_inscription	DATE	Remplie automatiquement à la date du jour

2. Le règlement du concours impose que seuls les candidats **nés à partir de 2005** peuvent s'inscrire : tout candidat né avant 2005 doit être refusé avec un message clair.

3. La date d'inscription ne doit pas être saisie à la main : elle est générée automatiquement par le serveur.
4. La liste des inscrits doit s'afficher dans un tableau HTML, avec le nombre total de candidats, et une couleur de ligne différente selon l'épreuve choisie.
5. Le gérant doit pouvoir **modifier** les informations d'un candidat (erreur de saisie fréquente sur les noms) et **supprimer** un candidat (désistement), après confirmation.

Par groupes de 3 à 4 élèves, tu vas concevoir l'application « **E-Candidat** », la tester avec 5 candidats fictifs, puis la présenter à la classe. Les solutions seront comparées à l'aide d'une grille d'évaluation : connexion sécurisée par `die()`, boucle de lecture correcte, clause `WHERE` présente dans les requêtes de modification et de suppression, affichage soigné.

8.3 Consignes

1. **Base de données.** Sous phpMyAdmin, crée la base `concours` puis la table `candidat` en respectant exactement la structure imposée. Insère manuellement un premier candidat pour vérifier la structure.
2. **Connexion.** Écris le fichier `connexion.php` qui établit la connexion au serveur MySQL (hôte `localhost`, utilisateur `root`, mot de passe vide sous WampServer) et sélectionne la base `concours`. Toute erreur doit arrêter le script avec un message explicite grâce à `die()`.
3. **Inscription.** Écris `inscription.php` : un formulaire HTML (nom, sexe, date de naissance, épreuve choisie) et le traitement PHP. À l'aide d'un `if/elseif`, refuse les candidats nés avant 2005. Si le candidat est accepté, exécute un `INSERT` en utilisant `date('Y-m-d')` pour la date d'inscription, puis affiche un message de confirmation.
4. **Liste des inscrits.** Écris `liste.php` : exécute un `SELECT`, affiche le nombre total d'inscrits avec `mysqli_num_rows`, puis parcourt les résultats avec une boucle `while` et `mysqli_fetch_assoc` pour remplir un tableau HTML. Utilise un `switch` sur `choix_epreuve` pour attribuer une couleur de fond à chaque ligne. Ajoute dans chaque ligne deux liens : `modifier.php?id=...` et `supprimer.php?id=...`.
5. **Modification.** Écris `modifier.php` : récupère `$_GET['id']`, pré-remplis le formulaire avec les données actuelles du candidat, puis exécute un `UPDATE` à la validation.
6. **Suppression.** Écris `supprimer.php` : affiche d'abord une demande de confirmation, puis exécute le `DELETE` ciblant l'identifiant transmis.
7. **Test et présentation.** Teste l'application avec 5 candidats fictifs (dont au moins un né avant 2005, qui doit être refusé). Présente votre application à la classe et évaluez les autres groupes avec la grille fournie.

8.4 Ressources à mobiliser

À RETENIR Boîte à outils de la leçon

- Structures de contrôle : `if / elseif / else` pour les conditions (contrôle de l'année de naissance), `switch` pour les cas multiples (couleur selon l'épreuve), `while` pour parcourir un résultat de requête.
- Fonction `date()` : `date('Y-m-d')` retourne la date du jour au format SQL; `date('Y', strtotime($d))` extrait l'année d'une date.
- Connexion : `$cnx = mysqli_connect(hote, user, mdp) or die(...)` puis `mysqli_select_db($cnx, 'base') or die(...)`.
- Exécution : `$res = mysqli_query($cnx, $sql);` pour un `SELECT`, `mysqli_num_rows($res)` donne le nombre de lignes.
- Lecture : `while ($ligne = mysqli_fetch_assoc($res))` donne un tableau associatif par ligne.
- Requêtes : `INSERT INTO ... VALUES`, `SELECT ... FROM`, `UPDATE ... SET ... WHERE id=...`, `DELETE FROM ... WHERE id=...` — la clause `WHERE` est indispensable pour ne modifier ou supprimer qu'une seule ligne.
- Fermeture : `mysqli_close($cnx)` en fin de script.

8.5 Démarche et corrigé commenté**EXERCICE RÉSOLU 8** Corrigé commenté de l'application « E-Candidat »

Produire l'application complète demandée par M. Etoundi.

SOLUTION

Étape 1 : création de la base et de la table. Sous phpMyAdmin (onglet SQL) :

```
CREATE DATABASE concours;
USE concours;
CREATE TABLE candidat (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(60) NOT NULL,
    sexe VARCHAR(1) NOT NULL,
    date_naissance DATE NOT NULL,
    choix_epreuve VARCHAR(20) NOT NULL,
    date_inscription DATE NOT NULL
);
```

Étape 2 : le fichier `connexion.php`. On utilise `die()` pour stopper proprement le script en cas d'échec, et on factorise la connexion dans un fichier inclus partout.

```
<?php
```



```
// connexion.php : connexion au serveur puis a la base
$cnx = mysqli_connect("localhost", "root", "")
    or die("Echec de connexion au serveur MySQL");
mysqli_select_db($cnx, "concours")
    or die("Base de donnees concours introuvable");
?>
```

Étape 3 : inscription.php. Le contrôle d'âge s'appuie sur l'extraction de l'année de naissance ; date('Y-m-d') horodate l'inscription.

```
<?php
require("connexion.php");
if (isset($_POST["envoyer"])){
    $nom = $_POST["nom"];
    $sexe = $_POST["sexe"];
    $naiss = $_POST["naissance"];
    $epreuve = $_POST["epreuve"];
    $annee = date("Y", strtotime($naiss)); // annee de naissance
    if ($annee < 2005){
        echo "Inscription refusee : candidat ne avant 2005.";
    } elseif ($nom == "" || $naiss == ""){
        echo "Veuillez remplir tous les champs.";
    } else {
        $jour = date("Y-m-d"); // horodatage automatique
        $sql = "INSERT INTO candidat
            (nom, sexe, date_naissance, choix_epreuve, date_inscription)
            VALUES ('$nom', '$sexe', '$naiss', '$epreuve', '$jour')";
        mysqli_query($cnx, $sql)
            or die("Erreur lors de l'inscription");
        echo "Candidat $nom inscrit avec succes le $jour.";
    }
}
?>
<form method="post" action="inscription.php">
    Nom : <input type="text" name="nom"><br>
    Sexe : <select name="sexe">
        <option value="M">M</option>
        <option value="F">F</option>
    </select><br>
    Date de naissance : <input type="date" name="naissance"><br>
    Epreuve : <select name="epreuve">
        <option value="Algorithmique">Algorithmique</option>
        <option value="Reseaux">Reseaux</option>
        <option value="BDD">BDD</option>
    </select><br>
    <input type="submit" name="envoyer" value="S'inscrire">
</form>
```

Étape 4 : liste.php. La boucle while lit ligne par ligne ; le switch choisit la couleur ; mysqli_num_rows affiche le total.

```
<?php
require("connexion.php");
$res = mysqli_query($cnx, "SELECT * FROM candidat")
    or die("Erreur de lecture");
$total = mysqli_num_rows($res);
```

```

echo "<h2>Liste des inscrits ($total candidats)</h2>";
echo "<table border='1' cellpadding='6'>";
echo "<tr><th>Nom</th><th>Sexe</th><th>Naissance</th>";
    <th>Epreuve</th><th>Inscrit le</th><th>Actions</th></tr>";
while ($c = mysqli_fetch_assoc($res)){
    switch ($c["choix_epreuve"]){
        case "Algorithmique" : $couleur = "#c8e6c9"; break; // vert
        case "Reseaux" :      $couleur = "#bbdefb"; break; // bleu
        case "BDD" :         $couleur = "#ffe0b2"; break; // orange
        default :             $couleur = "#ffffff";
    }
    echo "<tr style='background-color:$couleur'>";
    echo "<td>".$c["nom"]."</td><td>".$c["sexe"]."</td>";
    echo "<td>".$c["date_naissance"]."</td>";
    echo "<td>".$c["choix_epreuve"]."</td>";
    echo "<td>".$c["date_inscription"]."</td>";
    echo "<td><a href='modifier.php?id=".$c["id"]."'>Modifier</a>";
        <a href='supprimer.php?id=".$c["id"]."'>Supprimer</a></td>";
    echo "</tr>";
}
echo "</table>";
mysqli_close($cnx);
?>

```

Étape 5 : modifier.php. On pré-remplit le formulaire grâce à un SELECT sur l'identifiant reçu, puis on exécute l'UPDATE avec un WHERE obligatoire.

```

<?php
require("connexion.php");
$id = $_GET["id"];
if (isset($_POST["valider"])){
    $nom = $_POST["nom"]; $epreuve = $_POST["epreuve"];
    $sql = "UPDATE candidat SET nom='$nom', choix_epreuve='$epreuve'
        WHERE id=$id";
    mysqli_query($cnx, $sql) or die("Erreur de modification");
    header("Location : liste.php"); // retour a la liste
    exit;
}
$res = mysqli_query($cnx, "SELECT * FROM candidat WHERE id=$id");
$c = mysqli_fetch_assoc($res);
?>
<form method="post" action="modifier.php?id=<?php echo $id; ?>">
    Nom : <input type="text" name="nom"
        value="<?php echo $c['nom']; ?>"><br>
    Epreuve : <input type="text" name="epreuve"
        value="<?php echo $c['choix_epreuve']; ?>"><br>
    <input type="submit" name="valider" value="Enregistrer">
</form>

```

Étape 6 : supprimer.php. La confirmation évite toute suppression accidentelle ; le DELETE cible l'identifiant.

```

<?php
require("connexion.php");
$id = $_GET["id"];
if (isset($_POST["confirmer"])){

```

```
mysql_query($cnx, "DELETE FROM candidat WHERE id=$id")
    or die("Erreur de suppression");
header("Location : liste.php");
exit;
}
?>
<p>Confirmer la suppression du candidat numero
  <?php echo $id; ?> ?</p>
<form method="post" action="supprimer.php?id=<?php echo $id; ?>">
  <input type="submit" name="confirmer" value="Oui, supprimer">
  <a href="liste.php">Annuler</a>
</form>
```

Étape 7 : tests. Avec 5 candidats fictifs (ex. Awa née en 2006 épreuve BDD, Biko né en 2007 épreuve Réseaux, Carine née en 2005 épreuve Algorithmique, Dieudonné né en 2004 — refusé —, Essomba né en 2008 épreuve BDD), on vérifie : refus du candidat de 2004, total affiché de 4, couleurs correctes, modification du nom d'Essomba, suppression de Biko avec confirmation.

ATTENTION Pièges fréquents rencontrés pendant l'activité

- Oublier le **WHERE** dans **UPDATE** ou **DELETE** : toute la table est alors modifiée ou vidée. C'est l'erreur la plus grave de l'activité.
- Comparer la date de naissance entière à 2005 au lieu d'en extraire l'année avec `date('Y', strtotime(...))`.
- Concaténer `$_GET['id']` directement dans la requête sans vérification : c'est une porte ouverte à l'**injection SQL**. En production, on filtrerait avec `(int)` ou des requêtes préparées.
- Oublier `exit;` après `header("Location: ...")` : le script continue de s'exécuter.

8.6 Bilan de l'activité

Cette activité a permis d'assembler, dans un projet réel et contextualisé, toutes les compétences de la leçon. Elle a mis en évidence que :

- les **structures de contrôle** ne sont pas des notions isolées : `if/elseif` protège une règle métier (âge minimal), `switch` pilote l'affichage, `while` parcourt les résultats d'une requête ;
- `date()` automatise l'horodatage, ce qui supprime les erreurs de saisie manuelle constatées dans le cahier de M. Etoundi ;
- la connexion `mysqli` sécurisée par `die()` est le point de départ obligatoire de tout script d'accès aux données ;
- les quatre requêtes **INSERT**, **SELECT**, **UPDATE**, **DELETE** forment le cycle de vie complet d'une

donnée, et la clause `WHERE` est la condition de sécurité des deux dernières ;

- la comparaison des solutions sur la grille d'évaluation a développé l'esprit critique et le travail en équipe, compétences essentielles du développeur.

Tu disposes désormais d'un modèle complet d'application web PHP/MySQL, réutilisable pour tout projet de gestion : bibliothèque scolaire, notes d'examen, stock d'un cybercafé.

9 MÉTHODES ET EXERCICES RÉSOLUS

10 PHP AVANCÉ : STRUCTURES DE CONTRÔLE ET INTERFAÇAGE AVEC MYSQL

10.1 Structures alternatives : if, else, elseif, switch

MÉTHODE Choisir et rédiger une structure de contrôle PHP

1. Identifier le type de décision : une condition unique (if), deux cas exclusifs (if...else), plusieurs cas fondés sur des comparaisons (if...elseif...else), ou plusieurs cas fondés sur une seule valeur (switch).
2. Écrire la condition avec les bons opérateurs : comparaison == (égalité de valeur), === (égalité de valeur et de type), !=, <, <=, >, >=; logique && (ET), || (OU), ! (NON).
3. Ouvrir et fermer systématiquement les accolades { } dès la rédaction, même pour une seule instruction, et indenter le bloc de 4 espaces.
4. Pour un switch : un case par valeur possible, un break ; à la fin de chaque cas pour éviter la chute en cascade, et un default : pour traiter les valeurs imprévues.
5. Tester mentalement le script avec des valeurs limites (borne inférieure, borne supérieure, cas d'erreur, valeur null).

EXEMPLE Structure switch : menu d'une application de gestion de notes

Le switch est idéal pour tester une variable contre une liste de valeurs constantes (entiers, chaînes). Ici, le choix d'un menu principal.

```
<?php
$choix = "moyenne"; // Valeur simulée, en pratique issue de $_GET ou $_POST

switch ($choix) {
    case "saisie":
        echo "Affichage du formulaire de saisie des notes.";
        break;
    case "moyenne":
        echo "Calcul et affichage de la moyenne de la classe.";
        break;
    case "bulletin":
        echo "Génération du bulletin PDF.";
        break;
}
```

```
        default :  
            echo "Option inconnue : " . htmlspecialchars($choix);  
            break;  
    }  
?>
```

Remarque : `htmlspecialchars()` protège contre les failles XSS si `$choix` provient de l'utilisateur.

EXERCICE RÉSOLU 9 Moyenne et mention d'un candidat au BEPC

Une école de Yaoundé veut automatiser l'affichage des mentions au BEPC. Écrire un script PHP qui, à partir d'une moyenne stockée dans la variable `$moy`, affiche : « Refusé » si la moyenne est strictement inférieure à 10 ; « Passable » si elle est comprise entre 10 et 11,99 ; « Assez Bien » entre 12 et 13,99 ; « Bien » entre 14 et 15,99 ; « Très Bien » à partir de 16. Le script affichera ensuite la liste des 5 premiers numéros de table à l'aide d'une boucle.

SOLUTION

Raisonnement. Il s'agit de plusieurs cas fondés sur des intervalles (comparaisons), donc la structure adaptée est `if...elseif...else` et non `switch` (qui ne teste que l'égalité à une valeur). Les bornes doivent être testées dans l'ordre décroissant pour éviter les chevauchements (une moyenne de 16 validerait aussi `>= 14` si l'ordre était croissant). Pour la liste des numéros, le nombre d'itérations est connu (5) : on utilise `for`.

```
<?php  
    $moy = 13.5; // moyenne du candidat sur 20  
  
    // Structure alternative : tests dans l'ordre decroissant  
    if ($moy >= 16){  
        echo "Mention : Tres Bien<br>";  
    } elseif ($moy >= 14){  
        echo "Mention : Bien<br>";  
    } elseif ($moy >= 12){  
        echo "Mention : Assez Bien<br>";  
    } elseif ($moy >= 10){  
        echo "Mention : Passable<br>";  
    } else {  
        echo "Mention : Refuse<br>";  
    }  
  
    // Structure repetitive : nombre d'iterations connu  
    for ($i = 1; $i <= 5; $i++){  
        echo "Numero de table : " . $i . "<br>";  
    }  
?>
```

Vérification. Avec `$moy = 13.5` : le test `$moy >= 16` est faux, `$moy >= 14` est faux, `$moy >= 12` est vrai $\Rightarrow$ affichage « Mention : Assez Bien », ce qui est correct car $12 \leq 13,5 < 14$. La boucle affiche les numéros 1, 2, 3, 4, 5 puis s'arrête car la condition `$i <= 5` devient fausse.

pour `$i = 6`.

Conclusion. Le script affiche « Mention : Assez Bien » suivi des cinq numéros de table. L'ordre décroissant des tests garantit qu'aucune moyenne ne tombe dans deux cas à la fois.

10.2 Structures répétitives : for, foreach, while, do...while

MÉTHODE Choisir et rédiger une structure répétitive PHP

1. Identifier le type de répétition :

- Nombre d'itérations connu à l'avance $\Rightarrow$ `for` (ex. afficher 10 lignes).
- Parcours d'un tableau ou d'un jeu de résultats $\Rightarrow$ `foreach` (plus lisible, pas de gestion d'index).
- Condition d'arrêt basée sur une expression booléenne, test avant la première itération $\Rightarrow$ `while` (ex. lire un fichier tant que non fin).
- Condition d'arrêt, test après la première itération (au moins une exécution garantie) $\Rightarrow$ `do...while` (ex. menu interactif, saisie utilisateur).

2. Initialiser, tester, incrémenter : pour `for`, vérifier l'initialisation (`$i=0` ou `1`), la condition de continuation (`$i < $n` ou `$i <= $n`) et l'incrémentation (`$i++`).

3. Éviter les boucles infinies : s'assurer que la condition de sortie devient `false` à un moment (variable modifiée dans le corps de la boucle).

EXEMPLE Boucle `foreach` : parcours d'un tableau d'élèves

Le `foreach` est la structure privilégiée pour parcourir les tableaux associatifs (résultats de BDD, formulaires).

```
<?php
$eleves = [
    "24G001" => "Abena",
    "24G002" => "Biya",
    "24G003" => "Essomba"
];

echo "<ul>";
foreach ($eleves as $matricule => $nom) {
    echo "<li>[$matricule] $nom</li>";
}
echo "</ul>";
?>
```

EXEMPLE Boucle `do...while` : saisie contrôlée (simulation)

Utile pour imposer au moins une exécution, par exemple pour valider une saisie utilisateur (ici simulée).

```
<?php
    $note = -1; // Initialisation invalide pour forcer l'entree dans la boucle
    do {
        // En reel : $note = (float)readline("Note (0-20) : ");
        $note = 15; // Simulation d'une saisie valide
        if ($note < 0 || $note > 20) {
            echo "Note invalide, recommencez.<br>";
        }
    } while ($note < 0 || $note > 20);
    echo "Note valide enregistrée : $note";
?>
```

10.3 La fonction `date()`**MÉTHODE** Afficher une date ou une heure avec `date()`

1. Retenir la syntaxe générale : `date("format")` renvoie une chaîne de caractères représentant la date/heure courante du serveur.
2. Mémoriser les codes de format essentiels :

Code	Signification	Code	Signification
d	Jour sur 2 chiffres (01-31)	Y	Année sur 4 chiffres (2025)
m	Mois sur 2 chiffres (01-12)	H	Heure 24h (00-23)
i	Minutes (00-59)	s	Secondes (00-59)
l	Nom du jour (anglais)	w	Indice jour (0=Dim...6=Sam)

3. Composer le format avec des séparateurs libres : `date("d/m/Y H:i:s")` donne 12/05/2025 14:30:05.
4. Pour comparer des dates (ex. majorité), extraire l'année avec `date("Y")` (retourne chaîne, conversion auto en entier) ou utiliser `strtotime()` pour des calculs précis.
5. Attention au fuseau horaire : par défaut UTC. En local (WampServer), configurer `date.timezone = Africa/Douala` dans `php.ini` ou appeler `date_default_timezone_set("Africa/Douala");` en début de script.

EXERCICE RÉSOLU 10 Contrôle d'âge sur une plateforme d'inscription

Une startup camerounaise de e-learning n'accepte que les élèves âgés d'au moins 15 ans. Écrire un script PHP qui affiche la date du jour au format `jj/mm/aaaa`, calcule l'âge d'un élève né en 2007 et affiche « Inscription autorisée » ou « Inscription refusée ».

SOLUTION

Raisonnement. On récupère l'année courante avec `date("Y")`, qui renvoie une chaîne ; PHP la convertit automatiquement en entier lors de la soustraction. L'âge approché est année courante – année de naissance. On compare ensuite cet âge à 15 avec un `if...else`.

```
<?php
    date_default_timezone_set("Africa/Douala"); // Bonne pratique

    // 1. Affichage de la date du jour
    echo "Nous sommes le " . date("d/m/Y") . "<br>";

    // 2. Calcul de l'âge
    $anneeNaissance = 2007;
    $anneeCourante = (int)date("Y"); // Cast explicite pour clareté
    $age = $anneeCourante - $anneeNaissance; // 2025 - 2007 = 18

    echo "Age de l'eleve : " . $age . " ans<br>";

    // 3. Decision
    if ($age >= 15) {
        echo "Inscription autorisee";
    } else {
        echo "Inscription refusee";
    }
?>
```

Vérification. Si la date du serveur est le 12/05/2025 : la première ligne affiche « Nous sommes le 12/05/2025 » ; l'âge calculé vaut $2025 - 2007 = 18$; comme $18 \geq 15$, le test est vrai et le script affiche « Inscription autorisée ».

Conclusion. Le script affiche la date du jour, l'âge de 18 ans et le message « Inscription autorisée ». (En production, on comparerait les dates complètes jour/mois pour un âge exact au jour près.)

10.4 Connexion à MySQL avec l'extension mysqli

PROPRIÉTÉ Fonctions mysqli essentielles (style procédural)

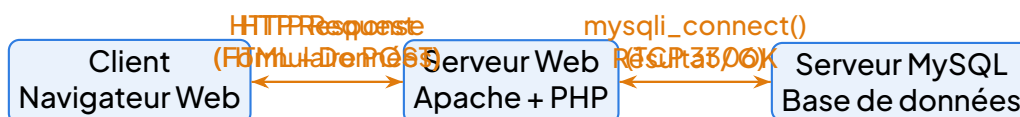
Fonction	Rôle
<code>mysqli_connect(hote, user, mdp, [base], [port])</code>	Ouvre une connexion, retourne un objet <code>\$cnx</code> ou <code>false</code> .
<code>mysqli_select_db(\$cnx, "nom_base")</code>	Sélectionne la base de données courante. Retourne <code>true/false</code> .
<code>mysqli_query(\$cnx, \$sql)</code>	Exécute une requête. Retourne : <code>false</code> (échec), <code>true</code> (INSERT/UPDATE/DELETE OK), ou objet résultat (SELECT OK).
<code>mysqli_fetch_assoc(\$res)</code>	Retourne la ligne suivante en tableau associatif <code>\$ligne["champ"]</code> , ou <code>false</code> si fin.
<code>mysqli_fetch_row(\$res)</code>	Retourne la ligne suivante en tableau indexé <code>\$ligne[0]</code> , <code>\$ligne[1]...</code>
<code>mysqli_fetch_array(\$res)</code>	Retourne les deux (indices numériques + associatifs).
<code>mysqli_num_rows(\$res)</code>	Retourne le nombre de lignes d'un résultat SELECT.
<code>mysqli_close(\$cnx)</code>	Ferme la connexion, libère les ressources serveur.
<code>mysqli_real_escape_string(\$cnx, \$str)</code>	Échappe les caractères spéciaux (protection injection SQL).
<code>die(message)</code>	Affiche <code>message</code> et arrête immédiatement le script.

MÉTHODE**Établir une connexion PHP–MySQL en 4 étapes (Squelette Bac)**

1. Ouvrir la connexion au serveur : `$cnx = mysqli_connect("localhost", "root", "");` (WampServer par défaut).
2. Sécuriser avec `die()` : tester la connexion et arrêter le script proprement en cas d'échec : `if (!$cnx) { die("..."); } ou forme idiomatique $cnx = ... or die("...");.`
3. Sélectionner la base : `mysqli_select_db($cnx, "nom_base") or die("Base introuvable");`.

4. Fermer la connexion en fin de script : `mysqli_close($cnx);`.

Réflexe Bac : Une connexion correcte contient toujours ces 4 appels, dans cet ordre : connect → test die → select_db → (requêtes) → close.



Architecture client-serveur PHP/MySQL : le script PHP (côté serveur) dialogue avec MySQL via l'extension mysqli sur le port 3306.

EXERCICE RÉSOLU 11 Connexion à la base d'un cybercafé

Le gérant d'un cybercafé de Douala veut connecter son site à la base MySQL `cybercafe` hébergée en local (utilisateur `root`, mot de passe vide). Écrire le script PHP de connexion complet, avec gestion des erreurs, qui affiche « Connexion réussie à la base cybercafe ».

SOLUTION

Raisonnement. On applique la démarche en 4 étapes : connexion au serveur, arrêt contrôlé en cas d'échec grâce à `die()`, sélection de la base, puis fermeture. La fonction `die()` affiche un message et stoppe immédiatement l'exécution du script, ce qui évite d'exécuter des requêtes sur une connexion inexistante.

```

<?php
// Etape 1 : connexion au serveur MySQL
$cnx = mysqli_connect("localhost", "root", "");

// Etape 2 : gestion de l'erreur de connexion
if (!$cnx) {
    die("Echec de connexion au serveur MySQL : " . mysqli_connect_error());
}

// Etape 3 : selection de la base de donnees
$dbase = mysqli_select_db($cnx, "cybercafe");
if (!$dbase) {
    die("Base de donnees introuvable : " . mysqli_error($cnx));
}

echo "Connexion reussie a la base cybercafe";

// Etape 4 : fermeture de la connexion
mysqli_close($cnx);
?>

```

Vérification. Si WampServer est démarré et que la base `cybercafe` existe, `mysqli_connect`

renvoie un objet connexion (valeur vraie), le test `!$cnx` est faux, la sélection réussit et le message de succès s'affiche. Si MySQL est arrêté, `$cnx` vaut `false` et `die()` affiche l'erreur puis stoppe le script.

Conclusion. Le script établit une connexion sécurisée en 4 étapes et affiche « Connexion réussie à la base cybercafe » ; toute défaillance est interceptée par `die()`.

10.5 Exécution des requêtes SQL depuis PHP : INSERT, UPDATE, DELETE

MÉTHODE Envoyer une requête de manipulation (INSERT, UPDATE, DELETE) depuis PHP

1. Écrire la requête SQL dans une chaîne PHP. Règle d'or : les valeurs textuelles et les dates sont entre apostrophes `'...'`, les nombres sans apostrophe. Les noms de tables/champs sans guillemets (ou backticks si mots-clés).
2. Protéger les données utilisateur : impératif avant d'insérer des variables dans le SQL. Utiliser `mysqli_real_escape_string($cnx, $variable)` sur chaque variable d'origine externe (`$_POST`, `$_GET`).
3. Envoyer la requête : `$res = mysqli_query($cnx, $sql);`. Pour INSERT/UPDATE/DELETE, `$res` vaut `true` (succès) ou `false` (échec) — pas un jeu de résultats.
4. Tester le résultat avec `if ($res) ou or die(...)`.
5. Vérifier la syntaxe SQL :
 - `INSERT INTO table (champs) VALUES (valeurs);`
 - `UPDATE table SET champ = valeur WHERE condition;`
 - `DELETE FROM table WHERE condition;`
6. Ne jamais oublier la clause `WHERE` dans UPDATE et DELETE, sous peine de modifier ou supprimer toute la table.

EXERCICE RÉSOLU 12 Gestion des abonnés d'un cybercafé

Dans la base `cybercafe`, la table `abonne(id, nom, forfait, solde)` contient les abonnés. Écrire un script PHP qui : (a) insère l'abonné (1, 'Kamga', 'Mensuel', 5000); (b) passe le solde de Kamga à 3000; (c) supprime l'abonné d'id égal à 4. Chaque opération doit confirmer son succès.

SOLUTION

Raisonnement. Les trois opérations sont des requêtes de manipulation : elles ne ren-

voient pas de lignes, seulement un booléen. On écrit chaque requête dans une variable `$sql`, on l'exécute avec `mysqli_query` et on teste le résultat. Les clauses `WHERE` ciblent précisément les lignes concernées.

```
<?php
$cnx = mysqli_connect("localhost", "root", "")
    or die("Echec de connexion : " . mysqli_connect_error());
mysqli_select_db($cnx, "cybercafe")
    or die("Base introuvable : " . mysqli_error($cnx));

// (a) Insertion : texte entre apostrophes, nombres sans
$sql1 = "INSERT INTO abonne(id, nom, forfait, solde)
        VALUES (1, 'Kamga', 'Mensuel', 5000)";
if (mysqli_query($cnx, $sql1)) {
    echo "Abonne insere avec succes<br>";
} else {
    echo "Echec de l'insertion : " . mysqli_error($cnx) . "<br>";
}

// (b) Modification : WHERE obligatoire pour cibler Kamga
$sql2 = "UPDATE abonne SET solde = 3000 WHERE nom = 'Kamga'";
if (mysqli_query($cnx, $sql2)) {
    echo "Solde mis a jour<br>";
} else {
    echo "Echec MAJ : " . mysqli_error($cnx) . "<br>";
}

// (c) Suppression : WHERE obligatoire pour cibler l'id 4
$sql3 = "DELETE FROM abonne WHERE id = 4";
if (mysqli_query($cnx, $sql3)) {
    echo "Abonne supprime<br>";
} else {
    echo "Echec suppression : " . mysqli_error($cnx) . "<br>";
}

mysqli_close($cnx);
?>
```

Vérification. (a) La requête insère une ligne ; `mysqli_query` renvoie `true` ⇒ « Abonne insere avec succes ». (b) Seule la ligne où `nom = 'Kamga'` est modifiée : le solde passe de 5000 à 3000. (c) Seule la ligne `id = 4` est supprimée ; les autres abonnés sont conservés.

Conclusion. Le script réalise les trois manipulations (insertion, modification, suppression) en confirmant chaque succès, grâce à des requêtes SQL correctement ciblées par `WHERE`.

10.6 Lecture et affichage des résultats d'une requête SELECT

MÉTHODE Exploiter un SELECT avec mysqli

1. Exécuter : `$res = mysqli_query($cnx, $sql)` ; — pour un `SELECT`, `$res` est un jeu de résultats (pointeur sur les lignes), ou `false` en cas d'erreur SQL.

2. Compter les lignes si besoin : `$nb = mysqli_num_rows($res)`; (ne fonctionne que sur un objet résultat SELECT).
3. Parcourir les lignes une à une dans une boucle `while`, avec au choix :
 - `mysqli_fetch_assoc($res)` : tableau associatif (`$l["nom"]`) — le plus lisible et robuste (insensible à l'ordre des colonnes).
 - `mysqli_fetch_row($res)` : tableau à indices numériques (`$l[0]`, `$l[1]` ...).
 - `mysqli_fetch_array($res)` : les deux à la fois (consomme plus de mémoire).

Chaque appel renvoie la ligne suivante, puis `false` quand il n'y en a plus : c'est ce qui arrête naturellement la boucle `while`.
4. Afficher les champs avec `echo`, idéalement dans un tableau HTML (`<table>`).
5. Fermer la connexion avec `mysqli_close($cnx)` ;.

EXERCICE RÉSOLU 13**Affichage des livres d'une bibliothèque scolaire**

La table `livre(code, titre, auteur, prix)` de la base `biblio` contient les livres de la bibliothèque d'un lycée de Bafoussam. Écrire un script PHP qui affiche le nombre total de livres, puis la liste de tous les livres (titre, auteur, prix) sous forme de tableau HTML.

SOLUTION

Raisonnement. Le `SELECT` renvoie un jeu de résultats. On compte d'abord les lignes avec `mysqli_num_rows`, puis on parcourt le jeu avec `while` et `mysqli_fetch_assoc`, qui donne accès aux champs par leur nom — code plus lisible et moins sensible à l'ordre des colonnes. L'affichage se fait ligne par ligne dans un tableau HTML ouvert avant la boucle et fermé après.

```
<?php
$cnx = mysqli_connect("localhost", "root", "")
    or die("Echec de connexion : " . mysqli_connect_error());
mysqli_select_db($cnx, "biblio") or die("Base introuvable : " . mysqli_error($cnx));

$sql = "SELECT * FROM livre";
$res = mysqli_query($cnx, $sql) or die("Requete invalide : " . mysqli_error($cnx));

// Nombre de lignes retournees
$nb = mysqli_num_rows($res);
echo "La bibliotheque contient " . $nb . " livre(s).<br>";

// Affichage dans un tableau HTML
echo "<table border='1' cellpadding='5'>";
echo "<tr><th>Titre</th><th>Auteur</th><th>Prix (FCFA)</th></tr>";
while ($ligne = mysqli_fetch_assoc($res)) {
    echo "<tr>";
    echo "<td>" . htmlspecialchars($ligne["titre"]) . "</td>";
```

```

        echo "<td>" . htmlspecialchars($ligne["auteur"]) . "</td>";
        echo "<td>" . number_format($ligne["prix"], 0, ',', '') . "</td>";
        echo "</tr>";
    }
    echo "</table>";

    mysqli_close($cnx);
?>

```

Vérification. Supposons la table contenant 3 livres. `mysqli_num_rows` renvoie 3 ⇒ « La bibliothèque contient 3 livre(s). ». La boucle `while` s'exécute 3 fois : aux itérations 1, 2, 3, `mysqli_fetch_assoc` renvoie un tableau associatif (valeur vraie) ; à la 4^e tentative, il renvoie `false` et la boucle s'arrête. Le tableau HTML comporte donc 1 ligne d'en-tête et 3 lignes de données. `htmlspecialchars()` protège l'affichage (XSS), `number_format` formate le prix (ex : 15 000).

Conclusion. Le script affiche le nombre de livres puis le catalogue complet dans un tableau HTML, en parcourant le jeu de résultats avec `mysqli_fetch_assoc` jusqu'à épuisement des lignes.

10.7 Synthèse : mini-application web PHP/MySQL complète

MÉTHODE Assembler une application d'enregistrement et de consultation (CRUD simplifié)

1. Analyser le besoin : identifier le formulaire HTML (champs de saisie), la table MySQL cible et les traitements (enregistrer, lister).
2. Créer le formulaire HTML avec `method="post"` et un `action` pointant vers le script de traitement. Ajouter `required` sur les champs obligatoires.
3. Dans le script PHP (ordre strict) :
 - (a) Connexion à la BDD (4 étapes : `connect`, `die`, `select_db`, `die`).
 - (b) Récupération et nettoyage des données : `$var = mysqli_real_escape_string($cnx, $_POST["champ"] ?? '');`. L'opérateur `??` (null coalescing) évite les notices "Undefined array key".
 - (c) Construction de la requête INSERT avec les variables nettoyées.
 - (d) Exécution et test du résultat.
 - (e) Requête SELECT pour afficher la liste actualisée.
 - (f) Fermeture de la connexion.
4. Tester chaque étape séparément : formulaire seul, connexion seule, insertion seule, avant d'assembler.

EXERCICE RÉSOLU 14 Application d'enregistrement des élèves (Version sécurisée)

Le proviseur d'un lycée de Garoua veut une page web permettant d'enregistrer les élèves (matricule, nom, classe) dans la table `eleve(matricule, nom, classe)` de la base `lycee`, puis d'afficher la liste des inscrits. Produire le formulaire HTML et le script PHP complet sécurisé contre les injections SQL.

SOLUTION

Raisonnement. L'application comporte deux fichiers. Le script PHP suit l'ordre : Connexion → Nettoyage entrées → INSERT → SELECT → Affichage → Fermeture. L'utilisation de `mysqli_real_escape_string` est la méthode procédurale standard pour neutraliser les apostrophes et caractères spéciaux dans les données utilisateur.

Fichier `inscription.html` :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Inscription des eleves - Lycee de Garoua</title>
  <style> body {font-family : sans-serif ; max-width : 600px ; margin : 2em auto ;} label {display : block ;
margin-top : 1em ;} input[type=text] {width : 100% ; padding : 0.5em ;} button {margin-top : 1em ;
padding : 0.5em 1em ;} table {width : 100% ; border-collapse : collapse ; margin-top : 1em ;} th, td {
border : 1px solid #ccc ; padding : 0.5em ; text-align : left ;} </style>
</head>
<body>
  <h2>Enregistrement d'un eleve</h2>
  <form method="post" action="enregistrer.php">
    <label>Matricule : <input type="text" name="matricule" required></label>
    <label>Nom complet : <input type="text" name="nom" required></label>
    <label>Classe : <input type="text" name="classe" required></label>
    <button type="submit">Enregistrer</button>
  </form>
</body>
</html>
```

Fichier `enregistrer.php` :

```
<?php
// 1. Connexion a la base (en premier pour disposer de $cnx pour l'echappement)
$cnx = mysqli_connect("localhost", "root", "")
    or die("Echec de connexion : " . mysqli_connect_error());
mysqli_select_db($cnx, "lycee") or die("Base introuvable : " . mysqli_error($cnx));

// 2. Recuperation ET securisation des donnees du formulaire
// ?? ' evite le warning "Undefined array key" si page appelee directement en GET
$mat = mysqli_real_escape_string($cnx, $_POST["matricule"] ?? '');
$nom = mysqli_real_escape_string($cnx, $_POST["nom"] ?? '');
$classe = mysqli_real_escape_string($cnx, $_POST["classe"] ?? '');

// 3. Insertion de l'eleve saisi (uniquement si formulaire soumis)
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    // Validation basique cote serveur
    if ($mat && $nom && $classe) {
```



```

        $sql = "INSERT INTO eleve (matricule, nom, classe)
                VALUES ('$mat', '$nom', '$classe')";
        if (mysqli_query($cnx, $sql)) {
            echo "<p style='color:green;'>Eleve enregistre avec succes.</p>";
        } else {
            echo "<p style='color:red;'>Echec de l'enregistrement : " . mysqli_error($cnx) . "</p>";
        }
    } else {
        echo "<p style='color:red;'>Tous les champs sont obligatoires.</p>";
    }
}

// 4. Affichage de tous les inscrits (SELECT)
$res = mysqli_query($cnx, "SELECT * FROM eleve ORDER BY matricule");
if ($res) {
    $nb = mysqli_num_rows($res);
    echo "<h3>Liste des inscrits ($nb)</h3>";
    if ($nb > 0) {
        echo "<table>";
        echo "<tr><th>Matricule</th><th>Nom</th><th>Classe</th></tr>";
        while ($e = mysqli_fetch_assoc($res)) {
            echo "<tr>";
            echo "<td>" . htmlspecialchars($e["matricule"]) . "</td>";
            echo "<td>" . htmlspecialchars($e["nom"]) . "</td>";
            echo "<td>" . htmlspecialchars($e["classe"]) . "</td>";
            echo "</tr>";
        }
        echo "</table>";
    } else {
        echo "<p>Aucun eleve inscrit pour l'instant.</p>";
    }
}

// 5. Fermeture
mysqli_close($cnx);
?>
</body>
</html>

```

Vérification. L'utilisateur saisit 24G001, Abena, Tle TI puis clique sur « Enregistrer ». Le navigateur envoie les données en POST à `enregistrer.php`. La connexion s'ouvre, les données sont échappées (ex : si nom = 0'Neil, il devient 0Ñeil dans le SQL), la requête INSERT s'exécute, le message de succès s'affiche, puis le SELECT parcouru en boucle affiche la ligne « 24G001 – Abena – Tle TI » dans le tableau HTML.

Conclusion. L'application complète enregistre chaque élève saisi via le formulaire et affiche la liste actualisée des inscrits : formulaire HTML → connexion → nettoyage (`mysqli_real_escape_string`) → INSERT → SELECT → affichage sécurisé (`htmlspecialchars`) → fermeture.

Au Cameroun, l'interfaçage PHP/MySQL est au cœur des solutions Fintech. Les API MTN Mobile Money (MoMo) et Orange Money permettent aux développeurs d'initier des paiements (Collection) et des transferts (Disbursement) depuis un serveur PHP.

- **Architecture type** : L'application PHP (hébergée sur un VPS ou via CAMTEL Fibre) reçoit une commande, enregistre la transaction en base (statut `EN_ATTENTE`), appelle l'API MoMo (HTTP/JSON, OAuth2), met à jour le statut (`REUSSI/ECHEC`) selon la réponse callback (IPN).
- **Sécurité** : Les clés API (API Key, Subscription ID) ne doivent jamais être dans le code source (fichier `.env` ou variables d'environnement serveur).
- **Exemple local** : La plateforme `impots.cm` (DGI) ou les portails `ANTIC / CENADI` reposent sur des stacks LAMP (Linux, Apache, MySQL/MariaDB, PHP) robustes avec journalisation (logs) et pare-feu applicatif (WAF).

Maîtriser `mysqli` et les structures de contrôle, c'est poser la première brique de ces systèmes critiques.

ATTENTION Les 6 erreurs qui coûtent des points au Bac

1. Confondre `=` et `==` dans une condition. `if ($moy = 10)` affecte 10 à `$moy` et le test est toujours vrai ! La comparaison s'écrit `==` (valeur) ou `===` (valeur + type). C'est l'erreur de logique la plus fréquente.
2. Oublier le `break` ; dans un `switch`. Sans `break`, PHP exécute tous les cas qui suivent le cas trouvé (chute en cascade) : le résultat affiché est faux alors que la condition était juste.
3. Oublier la clause `WHERE` dans `UPDATE` ou `DELETE`. `DELETE FROM abonne` ; vide toute la table. Au Bac, une requête de modification ou de suppression sans condition de ciblage est sanctionnée.
4. Traiter le résultat d'un `SELECT` comme un booléen, ou celui d'un `INSERT` comme un jeu de lignes. `mysqli_query` renvoie un jeu de résultats pour `SELECT` (à parcourir avec `mysqli_fetch_assoc` et `mysqli_num_rows`) mais un simple booléen pour `INSERT/UPDATE/DELETE`. Appeler `mysqli_num_rows` sur un `INSERT` provoque une erreur fatale.
5. Oublier la gestion d'erreur et la fermeture de la connexion. Une connexion sans `or die(...)` laisse le script planter silencieusement, et l'absence de `mysqli_close($cnx)` ; montre une démarche incomplète. Le squelette attendu est : `connect` → `die` → `select_db` → requêtes → `close`.
6. Injection SQL : insérer des variables `$_POST` brutes dans la requête. `VALUES ('$nom')` sans `mysqli_real_escape_string` permet à un attaquant d'injecter du SQL (ex : `' OR '1'='1`). Systématiser l'échappement ou utiliser les requêtes préparées (`mysqli_prepare`) en production.

EXERCICE 7 Entraînement : Gestion de stock d'une boutique

On considère la table `produit(id, designation, prix, quantite)` dans la base `boutique`.

1. Écrire le script PHP `ajouter.php` qui reçoit `designation`, `prix`, `quantite` par POST, les sécurise, insère le produit et affiche « Produit ajouté » ou l'erreur MySQL.
2. Écrire le script `lister.php` qui affiche tous les produits dans un tableau HTML avec colonnes : Désignation, Prix (formaté avec espace millier), Quantité, Valeur stock (Prix × Quantité). Afficher le total de la valeur du stock en bas du tableau.
3. Proposer la requête SQL pour augmenter de 10% le prix des produits dont la quantité est inférieure à 5.

CORRIGÉ Exercice : Gestion de stock d'une boutique1. Fichier `ajouter.php`

```
<?php
$cnx = mysqli_connect("localhost", "root", "") or die("Connexion echouee");
mysqli_select_db($cnx, "boutique") or die("Base introuvable");

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $des = mysqli_real_escape_string($cnx, $_POST["designation"] ?? '');
    $prix = (float)($_POST["prix"] ?? 0);
    $qte = (int)($_POST["quantite"] ?? 0);

    if ($des && $prix > 0 && $qte >= 0) {
        $sql = "INSERT INTO produit (designation, prix, quantite)
              VALUES ('$des', $prix, $qte)";
        if (mysqli_query($cnx, $sql)) {
            echo "Produit ajoute avec succes.";
        } else {
            echo "Erreur SQL : " . mysqli_error($cnx);
        }
    } else {
        echo "Donnees invalides.";
    }
}
mysqli_close($cnx);
?>
```

2. Fichier `lister.php`

```
<?php
$cnx = mysqli_connect("localhost", "root", "") or die("Connexion echouee");
mysqli_select_db($cnx, "boutique") or die("Base introuvable");

$res = mysqli_query($cnx, "SELECT * FROM produit");
$totalStock = 0.0;

echo "<table border='1' cellpadding='5'>";
echo "<tr><th>Designation</th><th>Prix (FCFA)</th><th>Qte</th><th>Valeur Stock</th></tr>";
```

```
while ($p = mysqli_fetch_assoc($res)) {
    $valeur = $p["prix"] * $p["quantite"];
    $totalStock += $valeur;
    echo "<tr>";
    echo "<td>" . htmlspecialchars($p["designation"]) . "</td>";
    echo "<td>" . number_format($p["prix"], 0, ',', '') . "</td>";
    echo "<td>" . $p["quantite"] . "</td>";
    echo "<td>" . number_format($valeur, 0, ',', '') . "</td>";
    echo "</tr>";
}
echo "<tr><th colspan='3' style='text-align:right;'>TOTAL</th><th>" . number_format($totalStock, 0, ',', ''
    , ''). "</th></tr>";
echo "</table>";

mysqli_close($cnx);
?>
```

3. Requête SQL de mise à jour (hausse 10% stock faible)

```
UPDATE produit
SET prix = prix * 1.10
WHERE quantite < 5;
```

11 EXERCICES

11.1 Je vérifie mes connaissances

EXERCICE 8 QCM : les structures de contrôle

Pour chaque question, une seule réponse est exacte. Entoure la bonne lettre.

1. En PHP, la structure `switch` est particulièrement adaptée lorsque :
 - a) on veut répéter un bloc d'instructions un nombre connu de fois ;
 - b) on compare une même variable à plusieurs valeurs constantes ;
 - c) on veut tester une condition complexe avec des opérateurs logiques ;
 - d) on veut parcourir un tableau associatif.
2. Dans une boucle `switch`, l'instruction `break` sert à :
 - a) terminer définitivement le script PHP ;
 - b) sortir du `switch` pour éviter d'exécuter les cas suivants ;
 - c) revenir au début du `switch` ;
 - d) fermer la connexion à MySQL.
3. La boucle `foreach` est spécialement conçue pour :
 - a) parcourir les éléments d'un tableau ;
 - b) exécuter une requête SQL ;
 - c) tester plusieurs conditions imbriquées ;
 - d) compter de 1 à 100.
4. La fonction `die()` permet de :
 - a) supprimer une ligne dans une table MySQL ;
 - b) arrêter immédiatement le script en affichant un message d'erreur ;
 - c) fermer la connexion à la base de données ;
 - d) vider un tableau PHP.

EXERCICE 9 Vrai ou faux — justifie ta réponse

Indique si chacune des affirmations suivantes est vraie ou fausse, puis justifie en une ou

deux phrases.

1. La boucle `do...while` exécute son bloc d'instructions au moins une fois, même si la condition est fausse dès le départ.
2. La fonction `mysqli_fetch_row()` retourne un tableau associatif dont les clés sont les noms des colonnes.
3. La fonction `mysqli_num_rows()` permet de connaître le nombre de lignes retournées par une requête `SELECT`.
4. L'instruction `mysqli_close($connexion)` ; doit être appelée avant `mysqli_query()` pour libérer la mémoire.
5. L'appel `date("d/m/Y")` affiche la date du jour au format jour/mois/année.

EXERCICE 10 Définitions

Définis en une phrase claire et précise chacun des termes suivants :

1. structure alternative ;
2. structure répétitive (ou boucle) ;
3. extension `mysqli` ;
4. requête de manipulation des données.

EXERCICE 11 Lecture de code : que vaut le résultat ?

On donne le script PHP suivant :

```
<?php
    $note = 14;
    if ($note < 10){
        echo "Ajourné";
    } elseif ($note < 12){
        echo "Passable";
    } elseif ($note < 14){
        echo "Assez Bien";
    } else {
        echo "Bien ou plus";
    }
    echo "<br>";
    for ($i = 1; $i <= 3; $i++){
        echo $i * 7 . " ";
    }
?>
```

1. Quel message est affiché par la structure `if...elseif...else` ? Justifie.

2. Quelle suite de nombres est affichée par la boucle `for` ?
3. Que se passerait-il si l'on remplaçait `$i++` par `$i--` dans la boucle `for` ?

11.2 Je m'entraîne

EXERCICE 12 La mention du Baccalauréat

Un lycée de Yaoundé souhaite automatiser l'affichage des mentions du Baccalauréat TI. La moyenne générale `$moyenne` (sur 20) est récupérée depuis un formulaire par `$_POST['moyenne']`. Le barème est le suivant :

Moyenne	Mention
Inférieure à 10	Ajourné
De 10 à moins de 12	Passable
De 12 à moins de 14	Assez Bien
De 14 à moins de 16	Bien
16 et plus	Très Bien

1. Écris un script PHP complet qui affiche la mention correspondant à la moyenne saisie, en utilisant une structure `if...elseif...else`.
2. Réécris le même traitement en utilisant une structure `switch` portant sur le quotient entier de la division de la moyenne par 2 (utilise `intval($moyenne / 2)`).
3. Ajoute un test qui vérifie que la moyenne saisie est bien comprise entre 0 et 20, et affiche un message d'erreur dans le cas contraire.

EXERCICE 13 Table de multiplication et boucle infinie

1. Écris un script PHP utilisant une boucle `for` qui affiche la table de multiplication de 7 (de 7×1 à 7×10) dans un tableau HTML à deux colonnes : « opération » et « résultat ».
2. Réécris ce script en utilisant une boucle `while`.
3. Un élève a écrit le code suivant, qui provoque une boucle infinie :

```
<?php
    $i = 1;
    while ($i <= 10) {
        echo "7 x " . $i . " = " . (7 * $i) . "<br>";
    }
?>
```

Identifie la cause de la boucle infinie et corrige le script.

EXERCICE 14 Script à trous : connexion et insertion

La direction du Lycée de Ngaoundéré gère ses élèves dans une base MySQL nommée `ecole`, hébergée sur `localhost` (utilisateur `root`, mot de passe vide). La table `eleve` possède les colonnes `id`, `nom`, `classe` et `moyenne`. Complète le script suivant en remplaçant les pointillés par les fonctions `mysqli` appropriées :

```
<?php
// 1. Connexion au serveur MySQL avec arrêt en cas d'échec
$cnx = .....( "localhost", "root", "" )
      or ...("Echec de connexion au serveur MySQL");

// 2. Selection de la base de donnees
.....( $cnx, "ecole")
      or die("Base de donnees introuvable");

// 3. Insertion d'un nouvel eleve
$sql = "INSERT INTO eleve (nom, classe, moyenne)
      VALUES ('MBALLA Amina', 'Tle TI', 15.5)";
$resultat = .....( $cnx, $sql);

if ($resultat) {
    echo "Eleve enregistré avec succes.";
} else {
    echo "Erreur lors de l'insertion.";
}

// 4. Fermeture de la connexion
.....( $cnx);

?>
```

EXERCICE 15 Produits en rupture de stock

La boutique en ligne d'une startup doualaïse utilise une table `produit(id, designation, prix, quantite)` dans la base `boutique`. Écris un script PHP complet qui :

1. se connecte au serveur MySQL et sélectionne la base `boutique` (avec `die()` en cas d'échec);
2. exécute une requête `SELECT` qui récupère tous les produits dont la quantité est égale à 0;
3. affiche le nombre de produits en rupture de stock grâce à `mysqli_num_rows()`;
4. affiche ces produits dans un tableau HTML (colonnes : identifiant, désignation, prix) en parcourant le résultat avec `mysqli_fetch_assoc()`;

5. ferme proprement la connexion.

11.3 Je me prépare au Bac

EXERCICE 16 Type Bac — Gestion des candidats au Baccalauréat TI

Le centre d'examen du Lycée Bilingue de Bafoussam informatise le suivi de ses candidats au Baccalauréat TI. La base de données MySQL `candidats_bac` contient la table `candidat(id, nom, date_naiss, etablissement, moyenne)`. Le serveur est `localhost`, l'utilisateur `root` et le mot de passe est vide.

1. Écris le script `liste.php` qui affiche tous les candidats dans un tableau HTML (nom, date de naissance, établissement, moyenne), en utilisant `mysqli_fetch_array()`.
2. Écris le script `admissibles.php` qui n'affiche que les candidats ayant une moyenne supérieure ou égale à 10, et indique en bas du tableau le nombre d'admissibles grâce à `mysqli_num_rows()`.
3. Écris le script `supprimer.php` qui :
 - a) vérifie que le paramètre `id` est bien présent dans l'URL avec `isset($_GET['id'])` et affiche un message d'erreur sinon ;
 - b) supprime le candidat correspondant à l'aide d'une requête `DELETE` ;
 - c) affiche un message de confirmation indiquant que la suppression a réussi, ou un message d'erreur dans le cas contraire.
4. Le directeur des examens souhaite afficher en haut de chaque page la date du jour au format `jour/mois/année`. Donne l'instruction PHP à insérer.
5. Cite deux précautions de sécurité à prendre avant d'utiliser la valeur de `$_GET['id']` dans une requête SQL.

EXERCICE 17 Type Bac — Modification des données d'un élève

L'administration du Collège Vogt de Yaoundé dispose d'une base `ecole` contenant la table `eleve(id, nom, classe, moyenne)`. On te demande de réaliser la page `modifier.php` qui permet de corriger les informations d'un élève.

1. La page reçoit l'identifiant de l'élève par l'URL (`modifier.php?id=3`). Écris le code PHP qui vérifie la présence de ce paramètre avec `isset()`, puis exécute la requête `SELECT ... WHERE` permettant de récupérer les données de cet élève.
2. Écris le code qui pré-remplit un formulaire HTML (champs `nom`, `classe`, `moyenne`) avec les valeurs lues dans la base, en utilisant `mysqli_fetch_assoc()`.

3. Lors de la validation du formulaire (méthode POST), écris le code PHP qui construit et exécute la requête `UPDATE ... WHERE` pour enregistrer les modifications, puis affiche le message « Les informations de l'élève ont été mises à jour. » en cas de succès.
4. Un camarade propose de ne pas mettre de clause `WHERE` dans la requête `UPDATE` « pour aller plus vite ». Explique pourquoi c'est une grave erreur et ce qu'elle provoquerait dans la table.

EXERCICE 18 Type Bac — Cybercafé et fonctions de lecture des résultats

Le gérant d'un cybercafé de Buea fait appel à toi pour développer une petite application de suivi des connexions des clients. La base `cybercafe` contient la table `session_nav(id, client, poste, duree, montant)`.

1. Explique clairement la différence entre les fonctions `mysqli_fetch_row()`, `mysqli_fetch_assoc()` et `mysqli_fetch_array()`, en précisant le type de tableau retourné par chacune et la manière d'accéder aux colonnes.
2. Le gérant a écrit le script suivant, mais il affiche des erreurs « Undefined array key » :

```
<?php
$cnx = mysqli_connect("localhost", "root", "");
mysqli_select_db($cnx, "cybercafe");
$res = mysqli_query($cnx, "SELECT client, montant FROM session_nav");
while ($ligne = mysqli_fetch_row($res)) {
    echo $ligne['client'] . " : " . $ligne['montant'] . "<br>";
}
mysqli_close($cnx);
?>
```

Explique la cause exacte de l'erreur, puis propose deux corrections différentes : l'une en conservant `mysqli_fetch_row()`, l'autre en utilisant `mysqli_fetch_assoc()`.

3. Écris un script qui calcule et affiche le montant total encaissé dans la journée, en parcourant le résultat de la requête `SELECT montant FROM session_nav` avec une boucle `while` et un accumulateur.
4. Le gérant veut afficher la date du jour en toutes lettres, par exemple « vendredi 14 juin 2025 ». Sachant que `date()` retourne les noms en anglais, propose une méthode simple (tableaux PHP de noms de jours et de mois en français, combinés avec `date("w")` et `date("n")`) pour obtenir cet affichage.

12 CORRIGÉS DES EXERCICES

CORRIGÉ Exercice 1 — QCM : les structures de contrôle

1. Réponse b. Le `switch` compare la valeur d'une seule variable (ou expression) à plusieurs **constantes** (les `case`). Pour répéter un bloc, on utilise une boucle ; pour des conditions complexes avec `&&` ou `||`, on préfère `if...elseif` ; pour parcourir un tableau, on utilise `foreach`.
2. Réponse b. Sans `break`, PHP poursuit l'exécution dans les `case` suivants (phénomène appelé `fall-through`). Le `break` fait donc sortir du bloc `switch` dès qu'un cas a été traité. Il n'arrête pas le script (c'est le rôle de `die()`) et ne ferme aucune connexion.
3. Réponse a. `foreach ($tableau as $valeur)` parcourt automatiquement tous les éléments d'un **tableau** (indexé ou associatif), sans qu'on ait à gérer de compteur.
4. Réponse b. `die("message")` affiche le message passé en argument puis **arrête immédiatement** l'exécution du script. On l'utilise classiquement après un échec de connexion : `or die("Echec de connexion");`.

CORRIGÉ Exercice 2 — Vrai ou faux

1. Vrai. Dans une boucle `do...while`, le test de la condition est effectué après l'exécution du bloc : le corps de la boucle est donc exécuté **au moins une fois**, même si la condition est fausse dès le départ. C'est la différence fondamentale avec `while`, qui teste avant.
2. Faux. C'est `mysqli_fetch_assoc()` qui retourne un tableau associatif (clés = noms des colonnes). `mysqli_fetch_row()` retourne un tableau à indices numériques : `$ligne[0]`, `$ligne[1]`, etc.
3. Vrai. Appliquée au résultat d'une requête `SELECT`, `mysqli_num_rows($resultat)` retourne le nombre de lignes (enregistrements) que la requête a renvoyées.
4. Faux. C'est l'inverse : `mysqli_close($connexion)` doit être appelée en dernier, après toutes les requêtes. Fermer la connexion avant `mysqli_query()` rendrait toute requête impossible, puisque celle-ci a besoin de la connexion active.
5. Vrai. Dans le format de `date()`, `d` représente le jour sur deux chiffres, `m` le mois sur deux chiffres et `Y` l'année sur quatre chiffres : on obtient par exemple 14/06/2025.

CORRIGÉ Exercice 3 — Définitions

1. Structure alternative : structure de contrôle qui permet d'exécuter un bloc d'instructions ou un autre selon qu'une condition est vraie ou fausse (ex. : `if...else`, `switch`).
2. Structure répétitive (boucle) : structure de contrôle qui permet d'exécuter un même bloc d'instructions plusieurs fois, tant qu'une condition reste vraie ou pour chaque élément d'un ensemble (ex. : `for`, `while`, `do...while`, `foreach`).
3. Extension `mysqli` : bibliothèque de fonctions PHP (MySQL improved) qui permet à un script de se connecter à un serveur MySQL, de sélectionner une base, d'envoyer des requêtes SQL et d'en exploiter les résultats.
4. Requête de manipulation des données : instruction SQL qui agit sur le contenu des tables : insertion (`INSERT INTO`), modification (`UPDATE`), suppression (`DELETE`) et sélection/lecture (`SELECT`).

CORRIGÉ Exercice 4 — Lecture de code : que vaut le résultat ?

1. La variable `$note` vaut 14. On évalue les conditions dans l'ordre :

- `$note < 10` : `14 < 10` est faux ;
- `$note < 12` : `14 < 12` est faux ;
- `$note < 14` : `14 < 14` est faux (l'inégalité est stricte !);
- on entre donc dans le `else`.

Le message affiché est : « Bien ou plus ». Le piège classique est de croire que 14 vérifie `$note < 14` : une inégalité stricte exclut la valeur elle-même.

2. La boucle `for` s'exécute pour $i = 1, 2, 3$ et affiche $i \times 7$:

$$1 \times 7 = 7, \quad 2 \times 7 = 14, \quad 3 \times 7 = 21.$$

Affichage obtenu : 7 14 21 (suivi d'un espace après chaque nombre).

3. Si l'on remplace `$i++` par `$i--`, la variable `$i` décroît : 1, 0, -1, -2, ... La condition `$i <= 3` resterait alors toujours vraie : on obtient une **boucle infinie** qui affiche 7, 0, -7, -14, ... sans jamais s'arrêter (jusqu'à l'interruption par le serveur au bout du temps maximal d'exécution). C'est l'erreur classique du compteur qui évolue dans le mauvais sens.

CORRIGÉ Exercice 5 — La mention du Baccalauréat

1. Script avec `if...elseif...else` :

```
<?php
    $moyenne = $_POST['moyenne'];

    if ($moyenne < 10){
        echo "Mention : Ajourné";
    } elseif ($moyenne < 12){
        echo "Mention : Passable";
    } elseif ($moyenne < 14){
        echo "Mention : Assez Bien";
    } elseif ($moyenne < 16){
        echo "Mention : Bien";
    } else {
        echo "Mention : Très Bien";
    }
?>
```

L'ordre des tests est essentiel : chaque `elseif` n'est évalué que si les précédents sont faux, ce qui garantit que les bornes inférieures sont déjà exclues.

2. Avec `switch:intval($moyenne / 2)` donne le quotient entier de la moyenne par 2. On vérifie les cas : moyenne dans `[0;10[` donne un quotient de 0 à 4 ; `[10;12[` donne 5 ; `[12;14[` donne 6 ; `[14;16[` donne 7 ; 16 et plus donne 8, 9 ou 10.

```
<?php
    $moyenne = $_POST['moyenne'];
    $q = intval($moyenne / 2);

    switch ($q){
        case 0 :
        case 1 :
        case 2 :
        case 3 :
        case 4 :
            echo "Mention : Ajourné";
            break;
        case 5 :
            echo "Mention : Passable";
            break;
        case 6 :
            echo "Mention : Assez Bien";
            break;
        case 7 :
            echo "Mention : Bien";
            break;
        default :
            echo "Mention : Très Bien";
    }
?>
```

Remarque : les cas 0 à 4 « tombent » volontairement les uns sur les autres (absence de `break`) pour partager le même traitement ; le `default` couvre les quotients 8, 9 et 10.

3. On encadre le traitement par un test de validité :

```
<?php
    $moyenne = $_POST['moyenne'];

    if ($moyenne < 0 || $moyenne > 20){
        echo "Erreur : la moyenne doit etre comprise entre 0 et 20.";
    } elseif ($moyenne < 10){
        echo "Mention : Ajourné";
    } elseif ($moyenne < 12){
        echo "Mention : Passable";
    } elseif ($moyenne < 14){
        echo "Mention : Assez Bien";
    } elseif ($moyenne < 16){
        echo "Mention : Bien";
    } else {
        echo "Mention : Très Bien";
    }
?>
```

Le test de validité est placé en premier : une moyenne de -3 ou de 25 ne peut plus déclencher une mention absurde.

CORRIGÉ Exercice 6 — Table de multiplication et boucle infinie

1. Avec une boucle `for` :

```
<?php
    echo "<table border='1'>";
    echo "<tr><th>Opération</th><th>Résultat</th></tr>";
    for ($i = 1; $i <= 10; $i++){
        echo "<tr>";
        echo "<td>7 x " . $i . " </td>";
        echo "<td>" . (7 * $i) . " </td>";
        echo "</tr>";
    }
    echo "</table>";
?>
```

2. Avec une boucle `while` :

```
<?php
    echo "<table border='1'>";
    echo "<tr><th>Opération</th><th>Résultat</th></tr>";
    $i = 1; // initialisation AVANT la boucle
    while ($i <= 10) {
        echo "<tr>";
        echo "<td>7 x " . $i . " </td>";
        echo "<td>" . (7 * $i) . " </td>";
        echo "</tr>";
        $i++; // incrementation DANS la boucle
    }
    echo "</table>";
?>
```

3. Cause de la boucle infinie : l'instruction d'incrémentation `$i++`, a été oubliée dans le corps de la boucle. La variable `$i` reste donc bloquée à 1, et la condition `$i <= 10` demeure vraie éternellement. Correction :

```
<?php
    $i = 1;
    while ($i <= 10) {
        echo "7x" . $i . " = " . (7 * $i) . "<br>";
        $i++; // ne jamais oublier de faire progresser le compteur
    }
?>
```

Règle à retenir : dans une boucle `while`, il faut toujours vérifier les trois ingrédients : initialisation avant la boucle, condition d'arrêt, et modification du compteur dans le corps.

CORRIGÉ Exercice 7 — Script à trous : connexion et insertion

Le script complété :

```
<?php
// 1. Connexion au serveur MySQL avec arrêt en cas d'echec
$cnx = mysqli_connect("localhost", "root", "")
    or die("Echec de connexion au serveur MySQL");

// 2. Selection de la base de donnees
mysqli_select_db($cnx, "ecole")
    or die("Base de donnees introuvable");

// 3. Insertion d'un nouvel eleve
$sql = "INSERT INTO eleve (nom, classe, moyenne)
    VALUES ('MBALLA Amina', 'Tle TI', 15.5)";
$resultat = mysqli_query($cnx, $sql);

if ($resultat) {
    echo "Eleve enregistré avec succes.";
} else {
    echo "Erreur lors de l'insertion.";
}

// 4. Fermeture de la connexion
mysqli_close($cnx);
?>
```

Justifications :

- `mysqli_connect(hote, utilisateur, mot_de_passe)` établit la connexion et retourne **false** en cas d'échec, ce qui déclenche le `or die(...)`;
- `mysqli_select_db($cnx, "ecole")` choisit la base sur laquelle porteront les requêtes;
- `mysqli_query($cnx, $sql)` envoie la requête SQL au serveur; pour un `INSERT`, elle re-

tourne true en cas de succès, false sinon — d'où le test `if ($resultat);`

- `mysqli_close($cnx)` libère la connexion en fin de script.

CORRIGÉ Exercice 8 — Produits en rupture de stock

```
<?php
// 1. Connexion et selection de la base
$cnx = mysqli_connect("localhost", "root", "")
    or die("Echec de connexion au serveur MySQL");
mysqli_select_db($cnx, "boutique")
    or die("Base boutique introuvable");

// 2. Requete : produits dont la quantite est nulle
$sql = "SELECT id, designation, prix FROM produit WHERE quantite = 0";
$resultat = mysqli_query($cnx, $sql)
    or die("Erreur dans la requete SQL");

// 3. Nombre de produits en rupture
$nb = mysqli_num_rows($resultat);
echo "<p>Nombre de produits en rupture de stock : " . $nb . "</p>";

// 4. Affichage dans un tableau HTML
echo "<table border='1'>";
echo "<tr><th>Id</th><th>Désignation</th><th>Prix (FCFA)</th></tr>";
while ($ligne = mysqli_fetch_assoc($resultat)) {
    echo "<tr>";
    echo "<td>" . $ligne['id'] . " </td>";
    echo "<td>" . $ligne['designation'] . " </td>";
    echo "<td>" . $ligne['prix'] . " </td>";
    echo "</tr>";
}
echo "</table>";

// 5. Fermeture de la connexion
mysqli_close($cnx);
?>
```

Points de vigilance :

- la clause `WHERE quantite = 0` est le cœur de la requête : c'est elle qui filtre les produits en rupture ;
- `mysqli_fetch_assoc()` retourne un tableau associatif : on accède aux colonnes par leur nom (`$ligne['designation']`) ; à chaque passage dans la boucle, elle avance d'une ligne et retourne false (en fait null) quand il n'y a plus rien à lire, ce qui arrête le `while` ;
- si aucun produit n'est en rupture, `$nb` vaut 0 et le tableau HTML reste vide : le script ne plante pas.

CORRIGÉ Exercice 9 — Type Bac : gestion des candidats au Baccalauréat TI

Barème indicatif (sur 20) : question 1 : 4 pts ; question 2 : 4 pts ; question 3 : 6 pts ; question 4 : 2 pts ; question 5 : 4 pts.

1. Script `liste.php` :

```
<?php
    $cnx = mysqli_connect("localhost", "root", "")
        or die("Echec de connexion");
    mysqli_select_db($cnx, "candidats_bac")
        or die("Base introuvable");

    $res = mysqli_query($cnx, "SELECT * FROM candidat")
        or die("Erreur SQL");

    echo "<table border='1'>";
    echo "<tr><th>Nom</th><th>Date de naissance</th>";
    echo "<th>Etablissement</th><th>Moyenne</th></tr>";
    while ($ligne = mysqli_fetch_array($res)) {
        echo "<tr>";
        echo "<td>". $ligne['nom'] . "</td>";
        echo "<td>". $ligne['date_naiss'] . "</td>";
        echo "<td>". $ligne['etablissement'] . "</td>";
        echo "<td>". $ligne['moyenne'] . "</td>";
        echo "</tr>";
    }
    echo "</table>";
    mysqli_close($cnx);
?>
```

`mysqli_fetch_array()` retourne un tableau à la fois numérique et associatif : on peut donc écrire `$ligne['nom']` ou `$ligne[1]`.

2. Script `admissibles.php` : même structure, avec la requête filtrante et le comptage :

```
<?php
    $cnx = mysqli_connect("localhost", "root", "")
        or die("Echec de connexion");
    mysqli_select_db($cnx, "candidats_bac")
        or die("Base introuvable");

    $sql = "SELECT * FROM candidat WHERE moyenne >= 10";
    $res = mysqli_query($cnx, $sql) or die("Erreur SQL");

    echo "<table border='1'>";
    echo "<tr><th>Nom</th><th>Etablissement</th><th>Moyenne</th></tr>";
    while ($ligne = mysqli_fetch_assoc($res)) {
        echo "<tr><td>". $ligne['nom'] . "</td><td>";
        echo "<td>". $ligne['etablissement'] . "</td><td>";
        echo "<td>". $ligne['moyenne'] . "</td></tr>";
    }
    echo "</table>";

    echo "<p>Nombre d'admissibles : ";
    echo mysqli_num_rows($res) . "</p>";
    mysqli_close($cnx);
```

```
?>
```

3. Script `supprimer.php` :

```
<?php
// a) Verification du parametre id dans l'URL
if (!isset($_GET['id'])) {
    die("Erreur : aucun identifiant fourni.");
}
$id = intval($_GET['id']); // securisation (voir question 5)

$cnx = mysqli_connect("localhost", "root", "")
    or die("Echec de connexion");
mysqli_select_db($cnx, "candidats_bac")
    or die("Base introuvable");

// b) Requete de suppression ciblee
$sql = "DELETE FROM candidat WHERE id = " . $id;
$ok = mysqli_query($cnx, $sql);

// c) Message de confirmation ou d'erreur
if ($ok) {
    echo "Le candidat a ete supprime avec succes.";
} else {
    echo "Erreur : la suppression a echoue.";
}
mysqli_close($cnx);
?>
```

La clause `WHERE id = ...` est indispensable : sans elle, `DELETE` viderait toute la table.

4. Instruction à insérer en haut de chaque page :

```
<?php echo "Date du jour : " . date("d/m/Y"); ?>
```

5. Deux précautions de sécurité avant d'utiliser `$_GET['id']` :

- Contrôler le type de la valeur, par exemple avec `intval($_GET['id'])`, qui force la conversion en entier : toute chaîne malveillante est neutralisée ;
- Se prémunir contre l'injection SQL : une valeur saisie dans l'URL peut contenir du code SQL hostile (ex. `?id=1 OR 1=1`). Il faut échapper la valeur avec `mysqli_real_escape_string($cnx, $_GET['id'])` ou, mieux, utiliser des requêtes préparées. On peut aussi vérifier que l'identifiant existe réellement dans la base avant de supprimer.

Points de vigilance au Bac : toujours justifier le rôle de `die()` (arrêt du script avec message), citer la condition d'application de `mysqli_num_rows()` (résultat d'un `SELECT`), et mentionner l'injection SQL dès qu'une donnée provient de l'utilisateur.

CORRIGÉ Exercice 10 — Type Bac : modification des données d'un élève

Barème indicatif (sur 20) : question 1 : 5 pts ; question 2 : 5 pts ; question 3 : 6 pts ; question 4 : 4 pts.

1. Vérification du paramètre et lecture de l'élève :

```
<?php
    if (!isset($_GET['id'])) {
        die("Erreur : aucun eleve selectionne.");
    }
    $id = intval($_GET['id']);

    $cnx = mysqli_connect("localhost", "root", "")
        or die("Echec de connexion");
    mysqli_select_db($cnx, "ecole") or die("Base introuvable");

    $sql = "SELECT * FROM eleve WHERE id = " . $id;
    $res = mysqli_query($cnx, $sql) or die("Erreur SQL");

    if (mysqli_num_rows($res) == 0) {
        die("Aucun eleve ne porte cet identifiant.");
    }
?>
```

2. Pré-remplissage du formulaire avec `mysqli_fetch_assoc()` :

```
<?php
    $eleve = mysqli_fetch_assoc($res);
?>
<form method="post" action="modifier.php?id=<?php echo $id; ?>">
    Nom : <input type="text" name="nom"
        value="<?php echo $eleve['nom']; ?>"><br>
    Classe : <input type="text" name="classe"
        value="<?php echo $eleve['classe']; ?>"><br>
    Moyenne : <input type="text" name="moyenne"
        value="<?php echo $eleve['moyenne']; ?>"><br>
    <input type="submit" value="Enregistrer">
</form>
```

L'attribut `value` de chaque champ reçoit la valeur lue dans la base : le formulaire apparaît ainsi pré-rempli.

3. Traitement de la validation (méthode POST) :

```
<?php
    if (isset($_POST['nom'])) {
        $nom = $_POST['nom'];
        $classe = $_POST['classe'];
        $moyenne = $_POST['moyenne'];
        $id = intval($_GET['id']);

        $cnx = mysqli_connect("localhost", "root", "")
            or die("Echec de connexion");
        mysqli_select_db($cnx, "ecole") or die("Base introuvable");
```

```

// Attention : en production, échapper $nom et $classe
// avec mysqli_real_escape_string($cnx, $nom) etc.
// ou utiliser des requêtes préparées (PDO/mysqli).
$sql = "UPDATE eleve
      SET nom = '$nom', classe = '$classe',
          moyenne = $moyenne
      WHERE id = $id";
$ok = mysqli_query($cnx, $sql);

if ($ok){
    echo "Les informations de l'élève ont été mises à jour.";
} else {
    echo "Erreur lors de la mise à jour.";
}
mysqli_close($cnx);
}
?>

```

4. Pourquoi la clause `WHERE` est vitale : une requête `UPDATE` sans `WHERE` s'applique à toutes les lignes de la table. Ici, `UPDATE eleve SET nom = ..., classe = ..., moyenne = ...` écraserait les données de tous les élèves du Collège Vogt : chacun se retrouverait avec le même nom, la même classe et la même moyenne que l'élève modifié. C'est une perte de données massive et irréversible (sauf sauvegarde). La clause `WHERE id = $id` garantit que seule la ligne de l'élève concerné est modifiée. Même remarque pour `DELETE` : sans `WHERE`, toute la table est vidée.

Points de vigilance au Bac : exiger la justification du rôle de `isset()` (éviter l'erreur « Undefined array key » et les accès sans paramètre), et la citation explicite du danger d'un `UPDATE` ou d'un `DELETE` sans `WHERE`.

CORRIGÉ Exercice 11 — Type Bac : cybercafé et fonctions de lecture des résultats

Barème indicatif (sur 20) : question 1 : 6 pts ; question 2 : 6 pts ; question 3 : 4 pts ; question 4 : 4 pts.

1. Différences entre les trois fonctions de lecture :

Fonction	Type de tableau retourné	Accès aux colonnes
<code>mysqli_fetch_row()</code>	tableau à indices numériques	<code>\$ligne[0]</code> , <code>\$ligne[1]</code> , ... dans l'ordre du <code>SELECT</code>
<code>mysqli_fetch_assoc()</code>	tableau associatif	<code>\$ligne['client']</code> , <code>\$ligne['montant']</code> (noms des colonnes)
<code>mysqli_fetch_array()</code>	tableau mixte (numérique et associatif)	les deux formes ci-dessus sont utilisables

Les trois fonctions lisent une ligne du résultat, avancent le curseur interne d'une position, et retournent `null` lorsqu'il n'y a plus de ligne — ce qui permet de les utiliser

comme condition d'une boucle `while`.

2. Cause exacte de l'erreur : le script utilise `mysqli_fetch_row()`, qui retourne un tableau à indices numériques, mais le code accède aux valeurs par des clés associatives (`$ligne['client']` et `$ligne['montant']`). Ces clés n'existent pas dans le tableau retourné : PHP signale donc « Undefined array key ». Deux corrections :

Correction 1 — on conserve `mysqli_fetch_row()` et on utilise les indices numériques (`client` est la 1^{re} colonne du `SELECT`, donc l'indice 0 ; `montant` l'indice 1) :

```
<?php
$cnx = mysqli_connect("localhost", "root", "");
mysqli_select_db($cnx, "cybercafe");
$res = mysqli_query($cnx,
    "SELECT client, montant FROM session_nav");
while ($ligne = mysqli_fetch_row($res)) {
    echo $ligne[0] . " : " . $ligne[1] . "<br>";
}
mysqli_close($cnx);
?>
```

Correction 2 — on remplace par `mysqli_fetch_assoc()`, qui accepte les clés nommées :

```
<?php
$cnx = mysqli_connect("localhost", "root", "");
mysqli_select_db($cnx, "cybercafe");
$res = mysqli_query($cnx,
    "SELECT client, montant FROM session_nav");
while ($ligne = mysqli_fetch_assoc($res)) {
    echo $ligne['client'] . " : " . $ligne['montant'] . "<br>";
}
mysqli_close($cnx);
?>
```

3. Montant total encaissé avec un accumulateur :

```
<?php
$cnx = mysqli_connect("localhost", "root", "")
    or die("Echec de connexion");
mysqli_select_db($cnx, "cybercafe") or die("Base introuvable");

$res = mysqli_query($cnx, "SELECT montant FROM session_nav");
$total = 0; // accumulateur initialise à 0
while ($ligne = mysqli_fetch_assoc($res)) {
    $total = $total + $ligne['montant'];
}
echo "Montant total encaissé : " . $total . " FCFA";
mysqli_close($cnx);
?>
```

Principe de l'accumulateur : on initialise `$total` à 0 avant la boucle, puis on y ajoute le montant de chaque ligne lue. (En pratique, on obtiendrait le même résultat en une seule requête : `SELECT SUM(montant) FROM session_nav.`)

4. Date du jour en français : `date("w")` donne le numéro du jour de la semaine (0 = dimanche, ..., 6 = samedi) et `date("n")` le numéro du mois (1 à 12). On utilise ces numéros comme indices de deux tableaux de noms français :

```
<?php
$jours = array("dimanche", "lundi", "mardi", "mercredi",
               "jeudi", "vendredi", "samedi");
$mois = array(1 => "janvier", "février", "mars", "avril",
              "mai", "juin", "juillet", "août", "septembre",
              "octobre", "novembre", "décembre");

$nomJour = $jours[date("w")]; // ex. "vendredi"
$nomMois = $mois[date("n")]; // ex. "juin"

echo $nomJour . " " . date("d") . " " . $nomMois
      . " " . date("Y");
// affiche par exemple : vendredi 14 juin 2025
?>
```

Attention : le tableau `$mois` est démarré à l'indice 1 (notation `1 => "janvier"`) car `date("n")` retourne 1 pour janvier, alors qu'un tableau PHP commence par défaut à l'indice 0.

Points de vigilance au Bac : savoir associer chaque fonction de lecture au type de tableau qu'elle retourne ; savoir diagnostiquer l'erreur « Undefined array key » comme une incohérence entre la fonction de lecture et le mode d'accès ; initialiser l'accumulateur avant la boucle.

13 FICHE BILAN

PHP avancé et MySQL

Alternatives

- if / else / elseif
- switch / case

Boucles

- for, foreach
- while, do...while

Fonction date()

- Formats : d, m, Y, H, i

Connexion mysqli

- mysqli_connect()
- die() en cas d'échec

Requêtes SQL

- INSERT, UPDATE, DELETE
- SELECT via
mysqli_query()

Lecture des résultats

- fetch_row / assoc /
array
- num_rows, close

Mini-application

- Formulaire + BDD

Carte mentale de la leçon : PHP avancé et interfaçage avec MySQL.

FICHE BILAN

L'essentiel à connaître pour l'examen

1. Définitions clés

- **Structure alternative** : exécute un bloc d'instructions selon qu'une condition est vraie ou fausse (if, else, elseif, switch).
- **Structure répétitive** (boucle) : répète un bloc tant qu'une condition est vraie (while, do...while) ou un nombre connu de fois (for, foreach).
- **Extension mysqli** : bibliothèque PHP permettant de dialoguer avec le SGBD **MySQL** (connexion, requêtes, lecture des résultats).
- **Jeu de résultats** : ensemble de lignes retourné par une requête SELECT, parcouru ligne par ligne avec les fonctions mysqli_fetch_*.

2. Syntaxes à connaître par cœur

Notion	Syntaxe PHP
Alternative	<code>if (\$cond) {...} elseif (\$c2) {...} else {...}</code>
Choix multiple	<code>switch (\$x) { case 1: ...; break; default: ...; }</code>
Boucle for	<code>for (\$i=0; \$i<10; \$i++) {...}</code>
Boucle foreach	<code>foreach (\$tab as \$val) {...}</code>
Boucle while	<code>while (\$cond) {...}</code>
Boucle do...while	<code>do {...} while (\$cond); (au moins 1 passage)</code>
Date	<code>date("d/m/Y H:i") → 15/03/2025 14:30</code>
Connexion	<code>\$cnx = mysqli_connect("localhost","root","") or die("Erreur");</code>
Base	<code>mysqli_select_db(\$cnx, "mabase");</code>
Requête	<code>\$res = mysqli_query(\$cnx, "SELECT * FROM eleve");</code>
Lecture	<code>while (\$lig = mysqli_fetch_assoc(\$res)) {...}</code>
Comptage	<code>\$n = mysqli_num_rows(\$res);</code>
Fermeture	<code>mysqli_close(\$cnx);</code>

3. Les trois fonctions de lecture (à ne pas confondre)

- `mysqli_fetch_row($res)` : tableau à indices numériques (`$lig[0]`).
- `mysqli_fetch_assoc($res)` : tableau associatif (`$lig["nom"]`).
- `mysqli_fetch_array($res)` : les deux à la fois.

4. Méthode express : interroger MySQL en 5 étapes

1. **Se connecter** : `mysqli_connect(...)` avec `or die(...)`.
2. **Choisir la base** : `mysqli_select_db(...)`.
3. **Envoyer la requête** : `mysqli_query($cnx, $sql)`.
4. **Exploiter** : boucle `while` + `mysqli_fetch_assoc` (ou `num_rows`).
5. **Fermer** : `mysqli_close($cnx)`.

5. Requêtes de manipulation depuis PHP

- **Insertion** : `INSERT INTO eleve VALUES (NULL, 'Amina', 15.5);`
- **Modification** : `UPDATE eleve SET moyenne=16 WHERE id=3;`
- **Suppression** : `DELETE FROM eleve WHERE id=3;`
- **Sélection** : `SELECT nom, moyenne FROM eleve WHERE moyenne>=10;`

À RETENIR Checklist avant le Bac

- ☐ Je sais écrire une structure `if / elseif / else` complète avec les accolades.
- ☐ Je sais quand préférer `switch` (comparaison d'une même variable à plusieurs valeurs) et je n'oublie pas le `break`.
- ☐ Je distingue `while` (test avant) et `do...while` (test après, au moins une exécution).
- ☐ Je sais parcourir un tableau avec `foreach ($tab as $val)`.
- ☐ Je connais les formats de `date()` : d, m, Y, H, i, s.
- ☐ Je sais écrire la connexion complète : `mysqli_connect + or die + mysqli_select_db`.
- ☐ Je sais exécuter une requête avec `mysqli_query($cnx, $sql)`.
- ☐ Je distingue `fetch_row`, `fetch_assoc` et `fetch_array` sans hésiter.
- ☐ Je sais compter les lignes d'un résultat avec `mysqli_num_rows()`.
- ☐ Je pense toujours à fermer la connexion avec `mysqli_close($cnx)`.
- ☐ Je sais écrire les quatre requêtes SQL : INSERT, UPDATE, DELETE, SELECT.
- ☐ Je vérifie les points-virgules, les guillemets autour des chaînes SQL et le \$ devant chaque variable PHP.

**Signé • OnBuch+**

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+